

SIYEENOVE robot arm kit compatible micro:bit

Instruction

2026.3.11

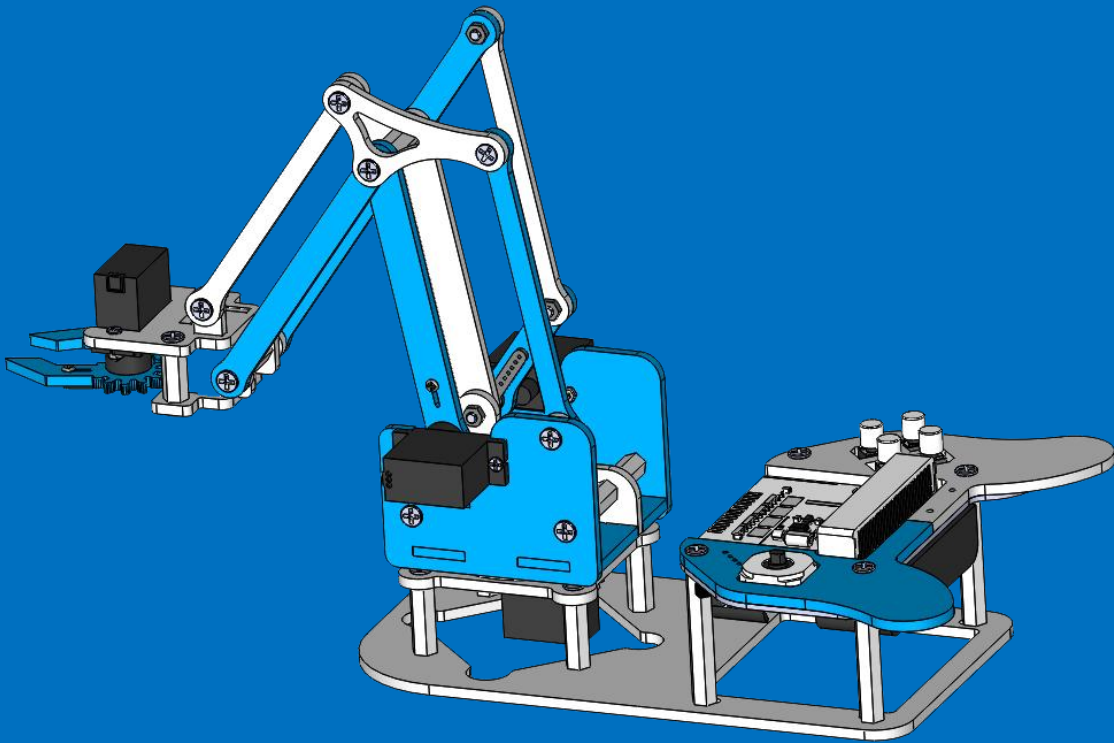


Table of contents

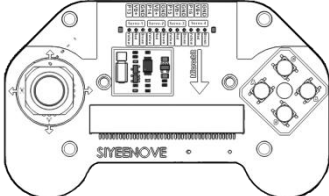
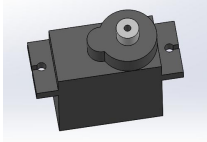
1. 1. Introduction to Robot Arm	4
1.1 Package Contents	4
1.2 Specification	6
1.3 Product Dimensions	6
1.4 SIYEENOVE Joystick Introduction	7
1.4.1 Joystick Specifications	7
1.4.2 Interface Specifications	8
2. Assembly	9
2.1 Assembly Preparation: Initialize the servo angle	9
2.2 Precautions Before Assembling the Robot	18
2.3 Start assembly	19
Step 1 - Assembly Servo-1	19
Step 2 - Assembly Servo-2	20
Step 3 - Assembly Servo-3	21
Step 4 - Assembly Servo-4	23
Step 5 - Power off the Joystick and disconnect servos ...	24
Step6 - Assembly Base (Nylon Standoffs)	25
Step7 - Assembly Base (Steel Ball Bearing Plate)	26
Step 8 - Assembly Base (Servo Structure)	27
Step 9 - Attach Servo Horn to Rotating Base	28
Step 10 - Upper Arm Link Assembly	29
Step 11 - Assemble Rotating Base	30
Step 12 - Mount Rotating Base to Main Frame	32
Step 13 - AAssembly Left Servo Structure on Turntable ...	33
Step 14 - Attach Upper Arm Linkage (Left Side)	34
Step 15 - Assembly Right Servo Structure on Turntable ...	35
Step 16 - Assembly Primary Arm/Elbow Connector(1)	36
Step 17 - Assembly Primary Arm/Elbow Connector(2)	37
Step 18 - Attach Forearm Linkage (First Position)	38
Step 19 - Attach Forearm Linkage (Second Position)	39
Step 20 - Connect Linkage between Upper Arm and Forearm .	40
Step 21 - Mount Left Gripper Jaw	41
Step 22 - Mount Right Gripper Jaw	43
Step 23 - Assembly Wrist Structure	44
Step 24 - Attach Gripper to Wrist	46
Step 25 - Connect Servo-4 with Extension Cable	48
Step 26 - Wire Servos to Joystick Controller	49
Step 27 - fix mJoystick to Base Frame	50
Step 28 - Cable Management	52
3. Control the robotic arm	54

3.1 Upload Control Code	54
3.2 Quick Start Guide for Robotic Arm Control	56
4. Introduction to MicroPython	57
4.1 Create a New Project	57
4.2 Save the project locally	58
4.3 Importing a Local Project	59
4.4 Upload Project	59
4.5 Save the project as a MicroPython file	64
4.6 Basic Syntax Learning for the Python Editor	64
5. Robotic arm programming	65
5.1 Button	66
5.2 Joystick	68
5.3 Vibration Motor	70
5.4 Battery voltage	71
5.5 Servo	72
5.6 mArm	75
6. QA	80
7. Contact Us	81

1. 1. Introduction to Robot Arm

The SIYEENOVE Micro:bit robotic arm is an educational and programmable robotic arm designed to teach students and enthusiasts the fundamentals of robotics, coding, and automation. Powered by the BBC Micro:bit microcontroller, this robotic arm offers a hands-on learning experience in STEM (science, technology, engineering, and mathematics) fields.

1.1 Package Contents

#Part 1 : Electronic components		
Joystick 1PCS	Servo 4PCS	3P cable 1PCS
		

#Part 2 : Fasteners/accessories

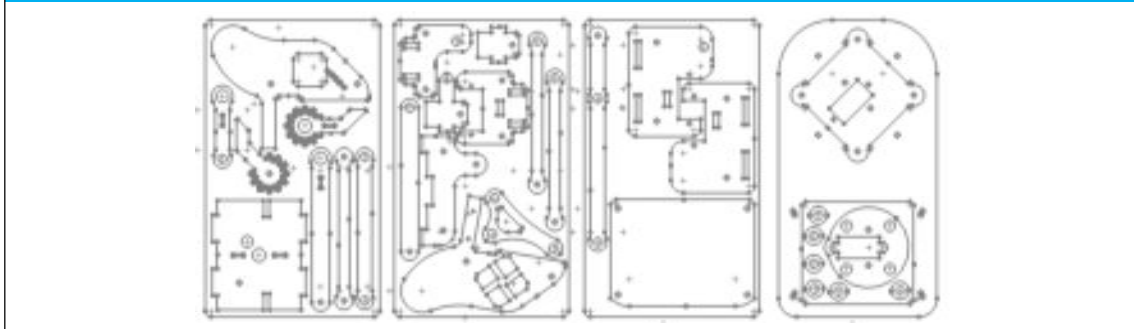
The following accessories are listed in order in a small bag with a label and serial number. You can quickly find them by referring to the serial number and name on the package.

1 M3×12mm Screw 6Pcs 	2 M3×9mm Screw 43Pcs 	3 M2×8mm Screw 10Pcs 
4 M1.4×5mm Screw 7Pcs 	5 M3 Nut 12Pcs 	6 M2 Nut 10Pcs 
7 M3×14mm Standoff 3Pcs 	8 M3×17+6mm Standoff 2Pcs 	9 M3×26mm Standoff 2Pcs 
10 M3×18mm Standoff 4Pcs	11 M3×20mm Standoff 5Pcs	12 M3×38mm Standoff 4Pcs

SIYEENOVE

		
13 5×3.2×3mm Spacer 8Pcs	14 5×3.2×6mm Spacer 6Pcs	15 Spiral Conduit
		
16 Steel Ball 4Pcs	17 Button Cop 4Pcs	
		

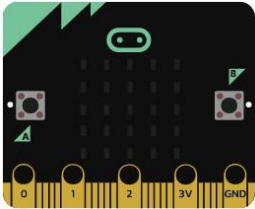
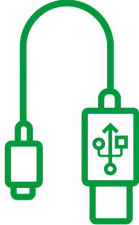
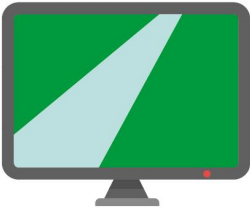
#Part 3 : Acrylic board



#Part 4 : Tools

		
M3 screwdriver 1PCS	M1.5 screwdriver 1PCS	wrench 1PCS

What do you need? (not included)

			
Micro:bit V2 1PCS	Micro USB cable	Computer with Internet and USB port	Four 1.5V AA batteries

SIYEENOVE

1.2 Specification

Control method: SIYEENOVE Joystick (micro:bit not include)

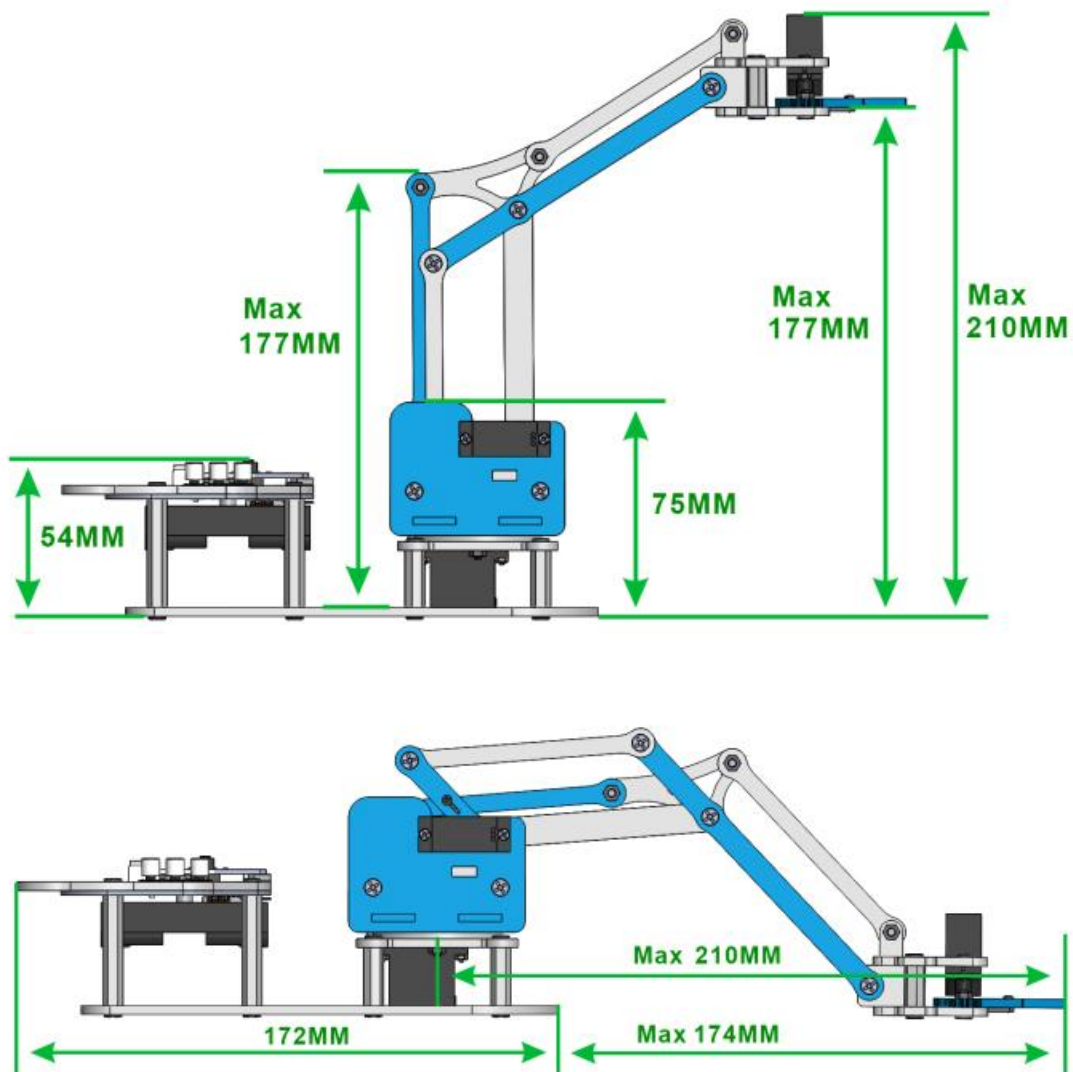
Programming language: MicroPython

Required battery type: 4×AA

Servo type: MG90S 180°

Maximum lifting capacity: $\leq 60g$

1.3 Product Dimensions



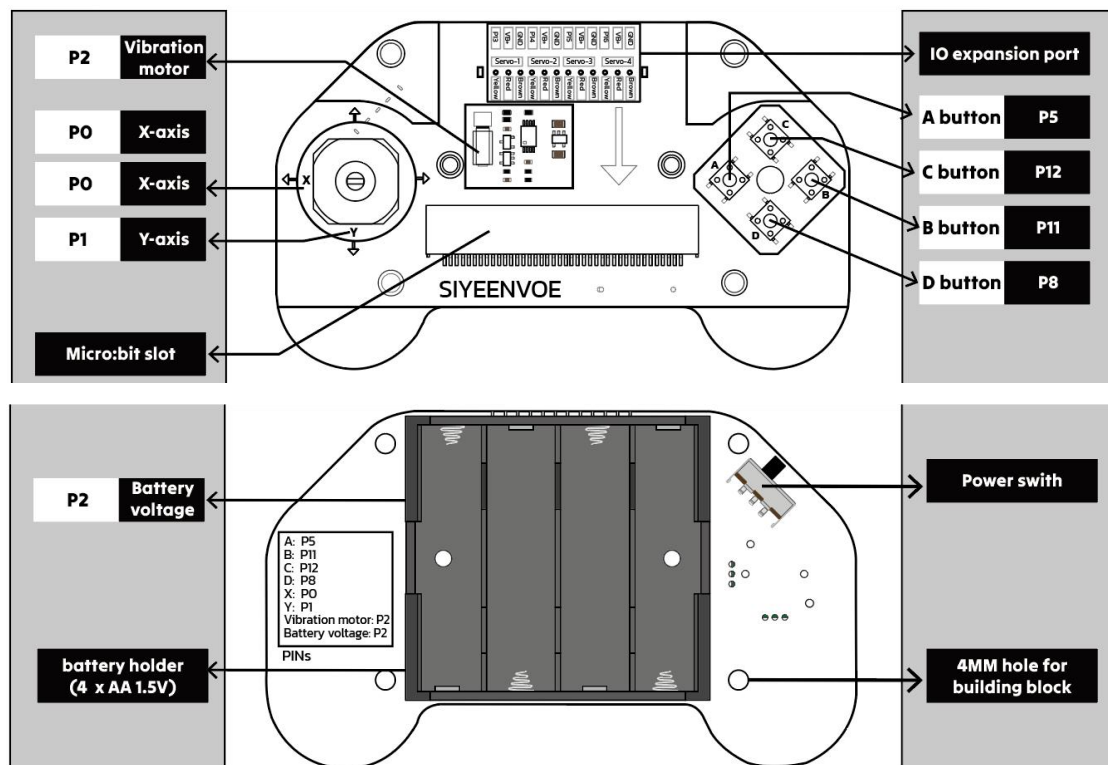
1.4 SIYEENOVE Joystick Introduction

The SIYEENOVE joystick controller is a manual input device designed for robotics, gaming, and educational applications. It features 3-axis (X/Y/Z-axis) motion and integrated buttons, enabling highly precise directional control. It's ideally suited for robotic arms, drones, and other motion-based systems.

1.4.1 Joystick Specifications

Shield	
Name	mJoystick
SKU	M1E0001
USB Port	
Name	Micro USB
Applicable motherboard	
Name	Micro:bit
Pins	
I/O Pins	4 (P13、P14、P15、P16)
Servo pins	4 (P13、P14、P15、P16)
Communication port	
SPI	Yes (P13、P14、P15、P16)
Power	
Input voltage (nominal value)	6V (4 节 AA 电池, 1.5V/PCS)
VB	6V (4 节 AA 电池, 1.5V/PCS)
Vibration motor	
Type	3610 Vibration motor
Rated voltage/current	2.7V/75mA
Rotate speed	14000±2500 RPM
Size	
With	72.5 mm
Length	120 mm
Height	29 mm
Weight	
	50 g

1.4.2 Interface Specifications



Name	Description
Vibration motor	Provides haptic feedback to the handle, controlled by micro:bit's P2 pin.
X axle	The joystick's X-axis, whose analog value can be read through micro:bit's P0 pin.
Y axle	The joystick's Y-axis, whose analog value can be read through micro:bit's P1 pin.
Micro:bit	Designed for inserting the micro:bit main control board.
4 x AA 1.5V battery case	The battery case accommodates 4 batteries, each with standard 1.5V voltage, approximately 14.5mm in diameter and 50.5mm in height.
Battery voltage	When 4 AA batteries are installed, the voltage can be read through micro:bit's P2 pin.
IO extension	For connecting external servos or sensors. The VB pin voltage equals the series voltage of 4 AA batteries ($1.5V \times 4 = 6V$).
A button	Its value can be read through micro:bit's P5 pin.
B button	Its value can be read through micro:bit's P12 pin.
C button	Its value can be read through micro:bit's P11 pin.
D button	Its value can be read through micro:bit's P8 pin.
Power switch	It controls the main power supply of the handle.
4MM hole	The handle features 4x4mm holes compatible with LEGO connectors for expansion.

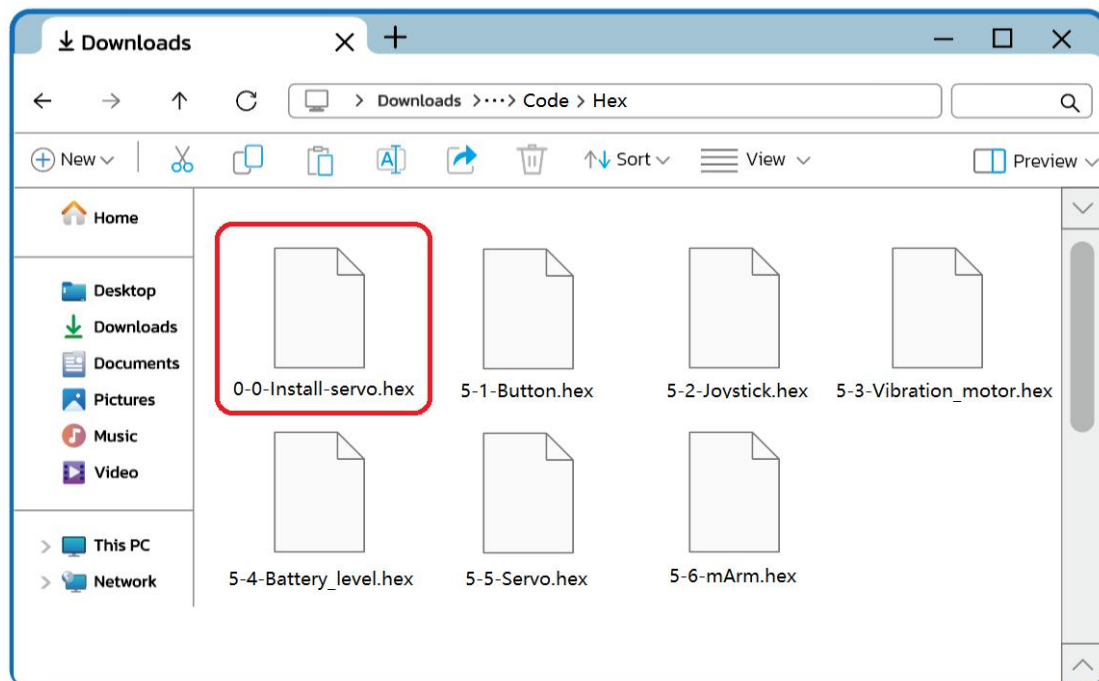
2. Assembly

2.1 Assembly Preparation: Initialize the servo angle

Attention!!

Before assembly, you must upload the "0-0-Install-servo.hex" code, as the servo motor requires angle initialization. If proper servo initialization is not performed during installation, the robotic arm will be unable to execute the preset programs and tasks. In severe cases, powering on and activating the system could cause the servo to jam, potentially damaging the servo motor.

The initialization code is stored in the "**Code>Hex**" folder within the downloaded resource package, as shown in the figure below:



Code:

```
from microbit import *

# Servo class
class Servo:
    def __init__(self, pin):
        self.pin = pin
        # Servo parameters
        self.min_pulse = 500    # Pulse width at 0 degrees (microseconds)
        self.max_pulse = 2400    # Pulse width at 180 degrees (microseconds)
        self.period = 20000     # 20ms period (50Hz)
        self.pin.set_analog_period(20) # Set period to 20ms

    def set_angle(self, angle):
        # Limit angle range
        angle = max(0, min(180, angle))

        # Calculate pulse width
        pulse_width = self.min_pulse + (angle / 180) * (self.max_pulse - self.min_pulse)

        # Convert to analog value (0-1023)
        analog_value = int((pulse_width / self.period) * 1023)

        # Set PWM
        self.pin.write_analog(analog_value)

    def set_pulse(self, pulse_width):
        # Set pulse width directly (microseconds)
        pulse_width = max(self.min_pulse, min(self.max_pulse, pulse_width))
        analog_value = int((pulse_width / self.period) * 1023)
        self.pin.write_analog(analog_value)

def battery_level():
    """
    Read battery level from analog pin P2
    Assumes voltage divider (6:1) for measuring 0-6V battery
    Returns battery percentage (0-100)
    """
    # Read analog value from pin2 (0-1023)
    bat_level = pin2.read_analog()

    # Map the analog reading to voltage (0-1023 corresponds to 0-3.3V)
    # With 6:1 voltage divider: 6V battery -> 1V at ADC input
    # ADC reference: 3.3V, 10-bit (0-1023)
```

```
# Calculate voltage at ADC pin
adc_voltage = bat_level * 3.3 / 1023

# Calculate actual battery voltage (with 6:1 divider)
battery_voltage = adc_voltage * 6

# Define voltage range for 0-100% (adjust based on your battery)
# Typical: 4.5V (0%) to 6.0V (100%) for 6V battery
min_voltage = 4.5 # 0% charge
max_voltage = 6.0 # 100% charge

# Convert to percentage
if battery_voltage >= max_voltage:
    percentage = 100
elif battery_voltage <= min_voltage:
    percentage = 0
else:
    percentage = (battery_voltage - min_voltage) / (max_voltage - min_voltage) * 100

return round(percentage)

# Show happy face
display.show(Image.HAPPY)
sleep(100)

# Create servo object
servo1 = Servo(pin13)
servo2 = Servo(pin14)
servo3 = Servo(pin15)
servo4 = Servo(pin16)

# Initialize the servo to 0 degrees
servo1.set_angle(0)
servo2.set_angle(0)
servo3.set_angle(0)
servo4.set_angle(0)

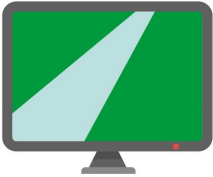
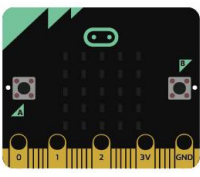
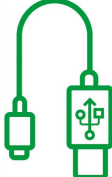
# The four servos slowly turn from 0 degrees to 90 degrees
for i in range(90):
    servo1.set_angle(i)
    servo2.set_angle(i)
    servo3.set_angle(i)
    servo4.set_angle(i)
```

SIYEENOVE

```
sleep(20)

# Cycle read battery voltage
while True:
    display.scroll(battery_level())
    sleep(2000)
```

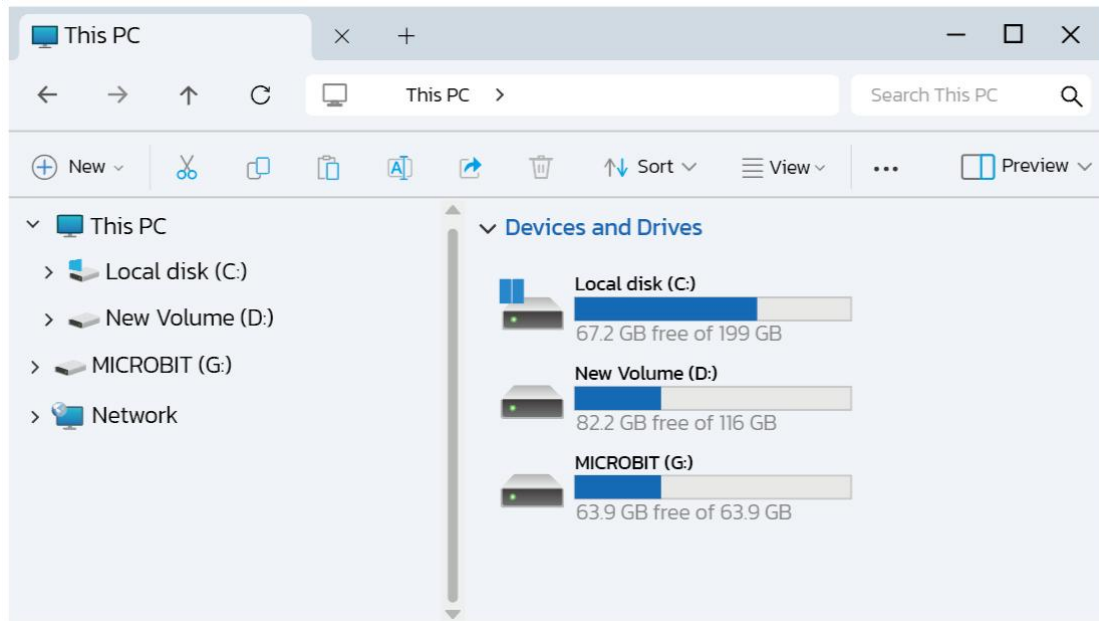
Required tools:

PC	Micro:bit v2.x.x	Micro USB cable
		

Steps:

I. Use a Micro USB cable to connect the Micro:bit to your PC. The PC will detect the Micro:bit as a removable drive (e.g., "MICROBIT").

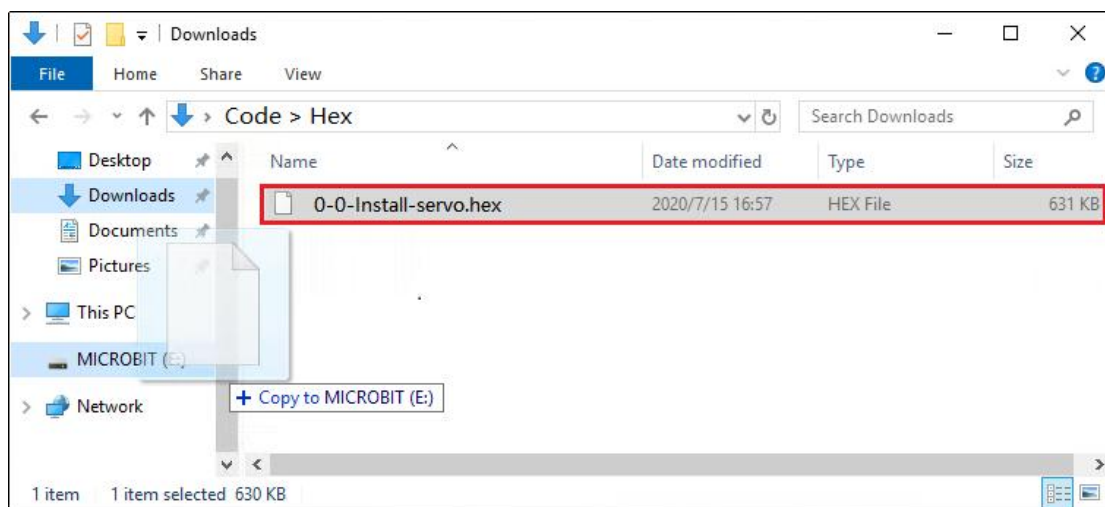




II. Locate the HEX file (0-0-Install-servo). Use one of the following two methods to quickly upload the code to the Micro:bit.

Method 1 (Drag and Drop):

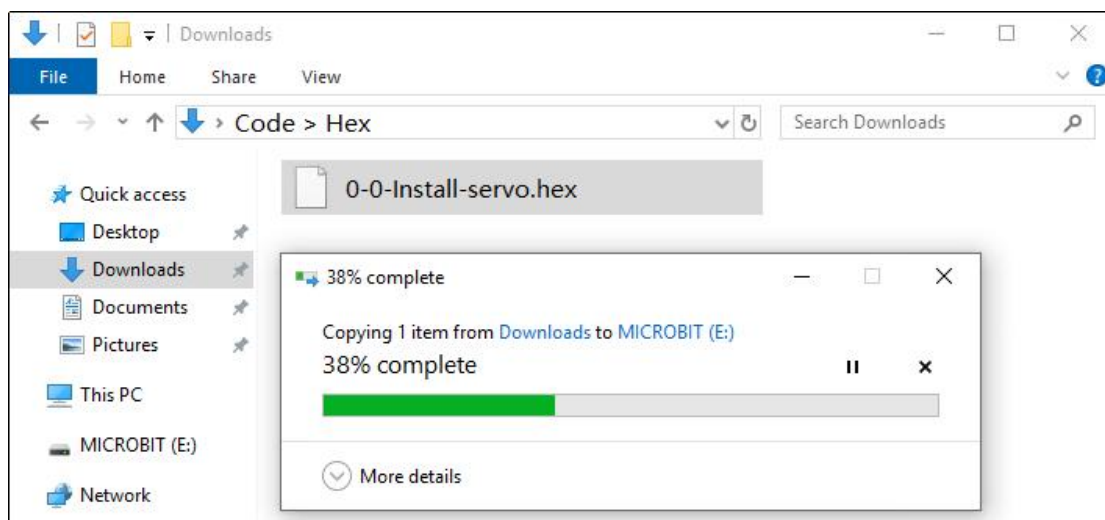
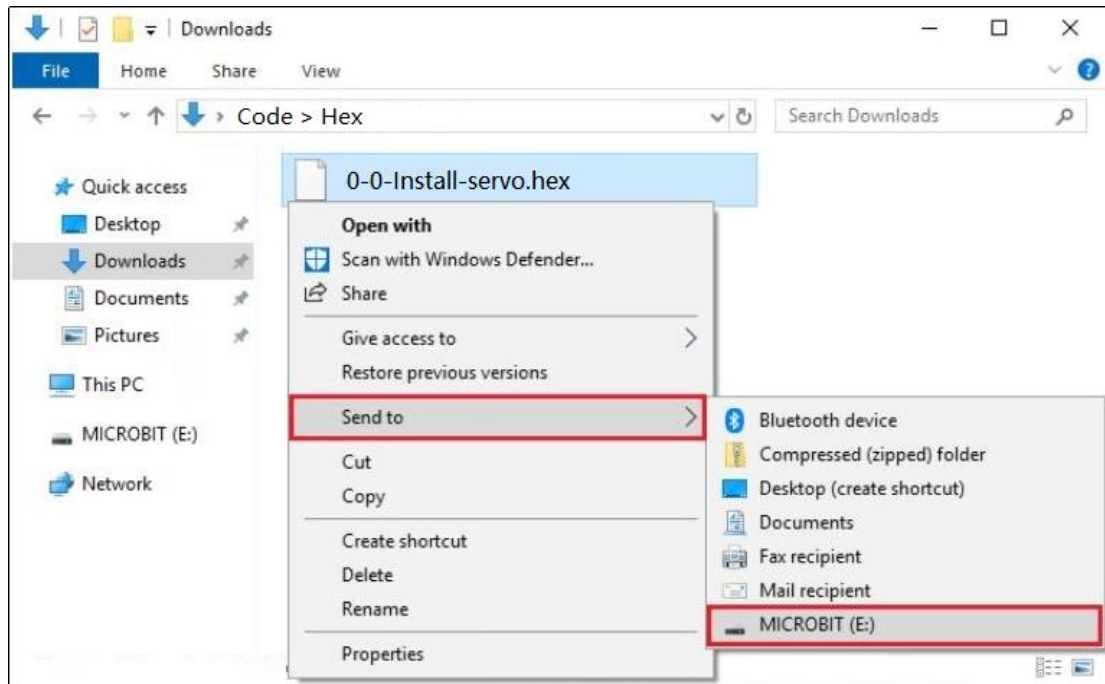
Simply drag and drop the HEX file onto the MICROBIT drive.



SIYEENOVE

Method 2 (Send to):

Right-click the HEX file and select MICROBIT drive → “Send to”

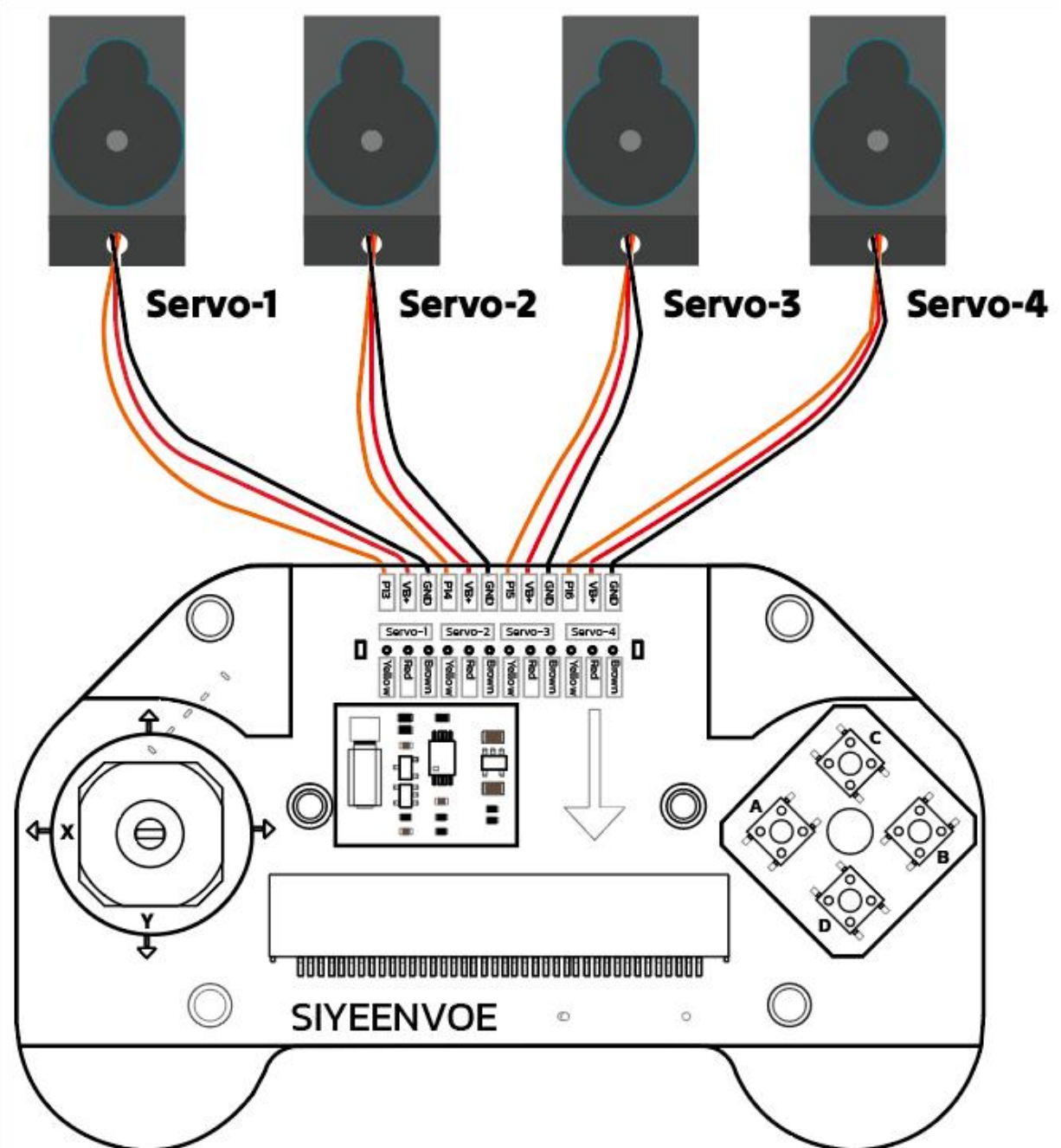


Attention:

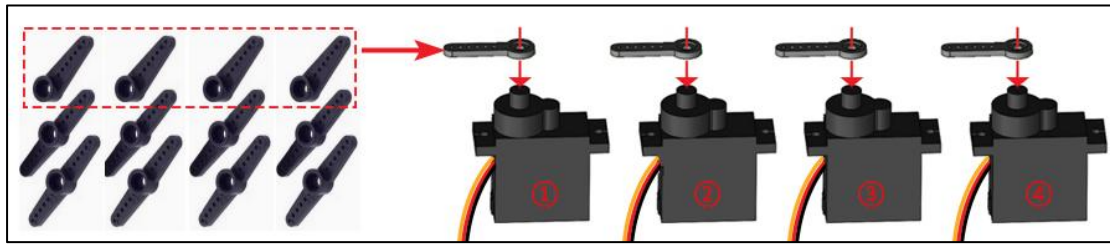
Make sure your Micro:bit is properly connected before uploading.
Do not disconnect the device while the firmware is flashing.

i. Connect the servo to the control stick:

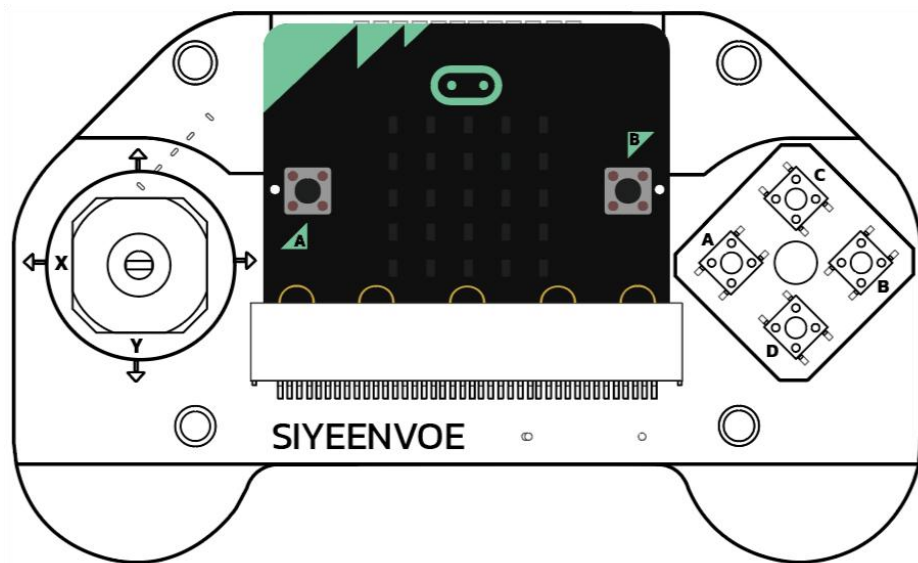
Servo-1			Servo-2			Servo-3			Servo-4		
											
P13	VB+	GND	P14	VB+	GND	P15	VB+	GND	P16	VB+	GND



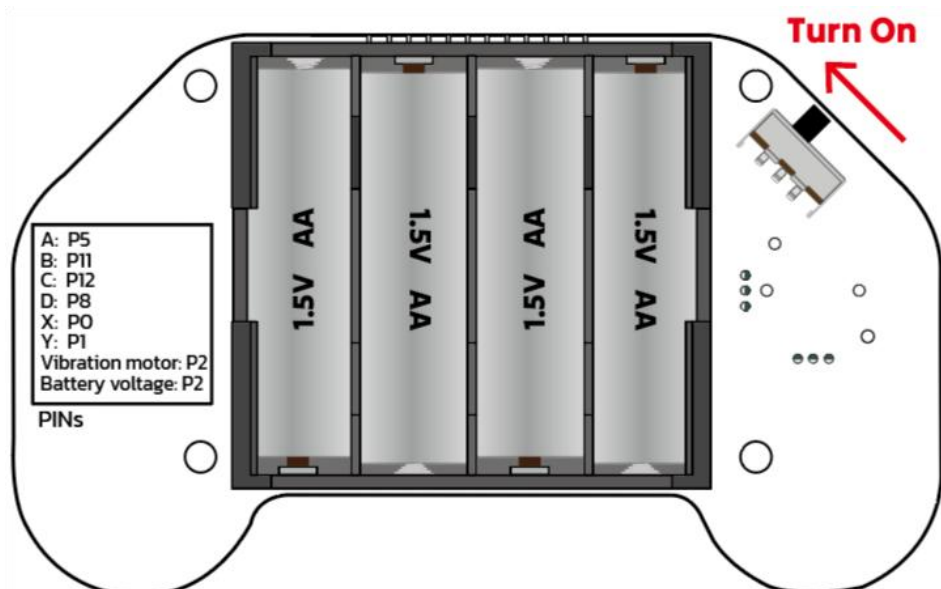
ii. To facilitate observation of the servo shaft's rotation and its ability to stop at the initial position, we recommend temporarily connecting the servo swing arm (arm) to the servo shaft during this process without using screws for fixation.



iii. Insert the Micro:bit into the handle.



6. Install 4 AA batteries and turn on the power switch.



7. Results:

1) Battery Level Indication:

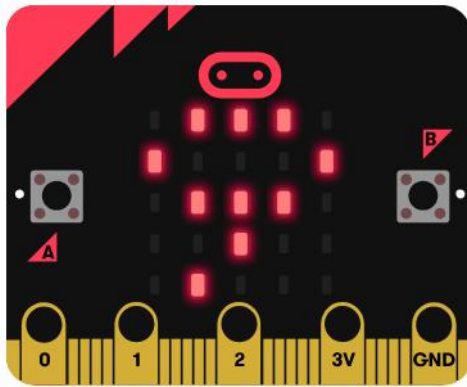
After powering on, the micro:bit's LED matrix will continuously cycle through displaying the battery level (0%-100%).

Important: When the battery level drops below 30%, replace all batteries immediately.

If no battery level is displayed:

Check if the micro:bit is fully inserted into the handle.

Verify correct battery installation.



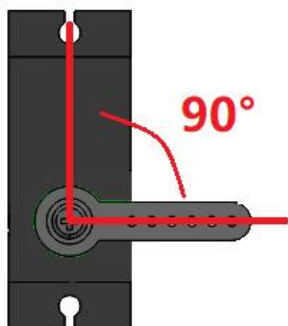
2) Servo Initialization Sequence:

First Power-On:

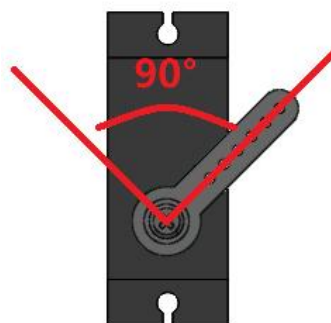
Allow 5 seconds to pass, all 4 servos will rotate from random positions → 0° → slowly to 90° (holding at 90°)

On subsequent power-ups:

Allow 5 seconds to pass, all servos will move from 90° → 0° → slowly back to 90° (holding position).



.....



.....

If servos fail to rotate:

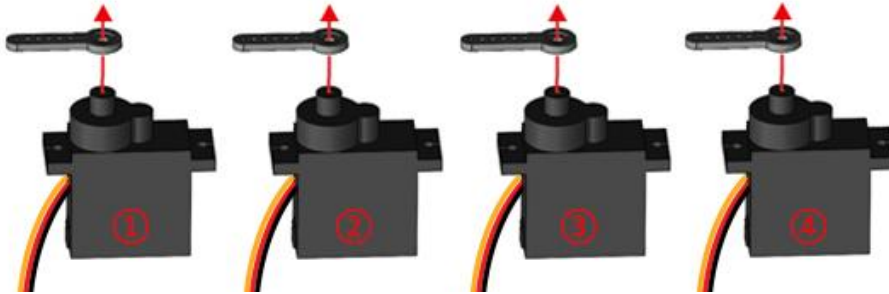
Inspect servo wiring connections

Re-upload the code if wiring is correct but servos remain unresponsive

Clean micro:bit's edge connectors and reinsert firmly

Contact support if issues persist (possible servo defect).

8. After the servo angle initialization is complete, gently remove the servo arm from each servo for subsequent installation. Do not rotate the servo shaft, as this could damage the gear mechanism or disrupt the calibration position.

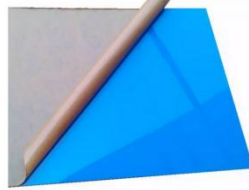


2.2 Precautions Before Assembling the Robot

1) Remove the protective film before assembly! Both the top and bottom surfaces are coated with a protective film.



and



2) Do not empty all the components from the bag at once to avoid mixing and confusion. Open the component bags one by one according to the instructions in the manual to prevent small parts from getting mixed up. Some screws may be included as spare parts.

3) Assemble in the same direction as the chart; the colors of the components may vary.

4) If any parts are missing or damaged, do not assemble or use the product. Please contact us for after-sales support.

2.3 Start assembly

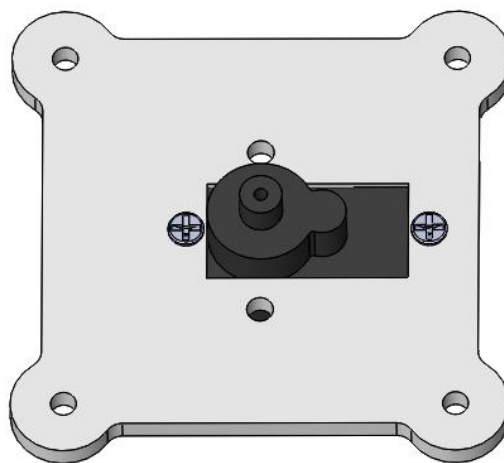
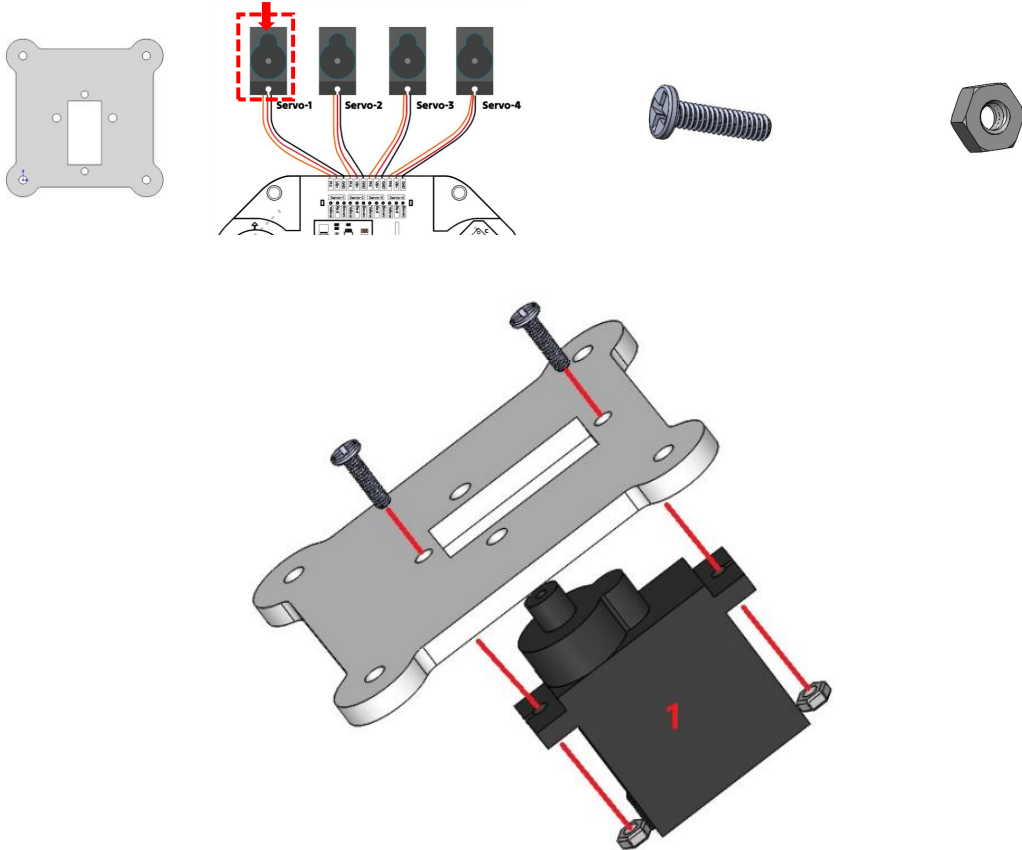
Step 1 - Assembly Servo-1

Acrylic
board 1PCS

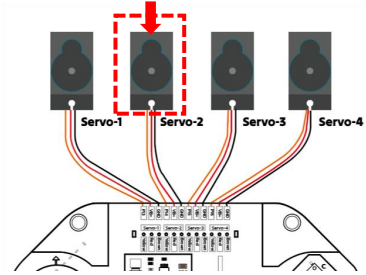
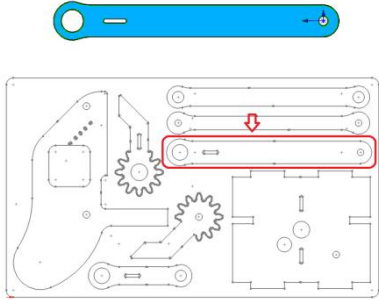
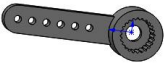
Servo-1

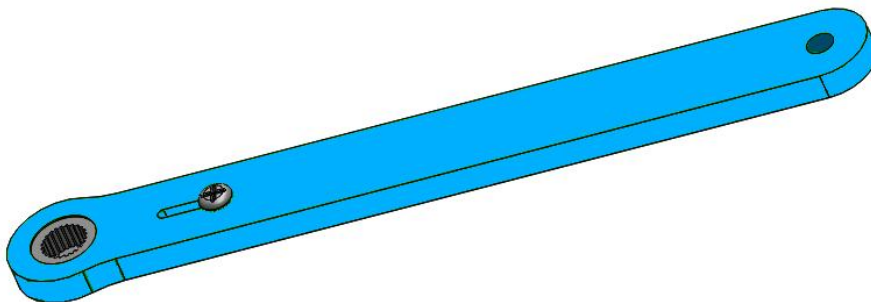
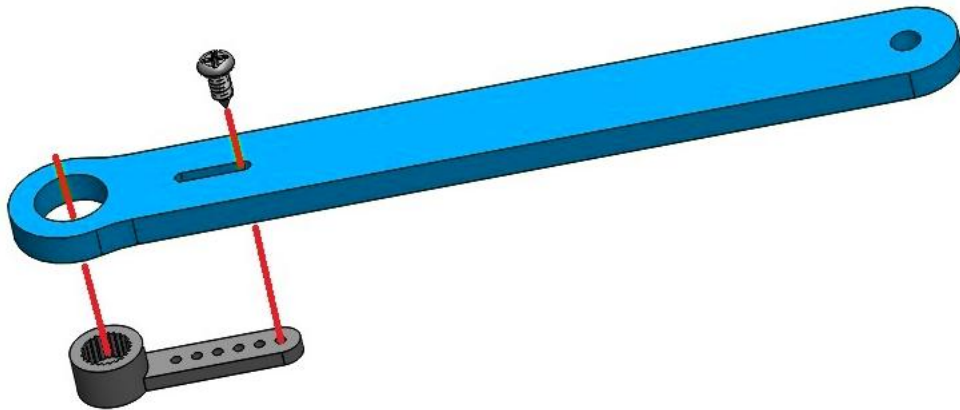
Bag No.③
M2x8mm screw 2PCS

Bag No.⑥
M2 nut 2PCS



Step 2 - Assembly Servo-2

Servo-2		Acrylic board 1PCS	
			
Servo horn 1PCS	Bag No. ④ M1.4X5 Self-tapping screw 1PCS		M2.5X4mm screw 1PCS
			

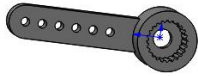


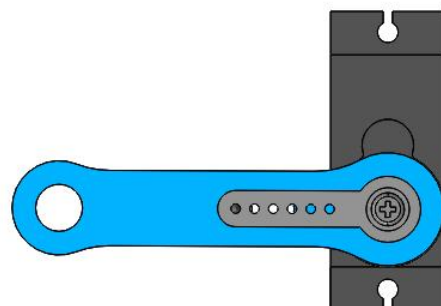
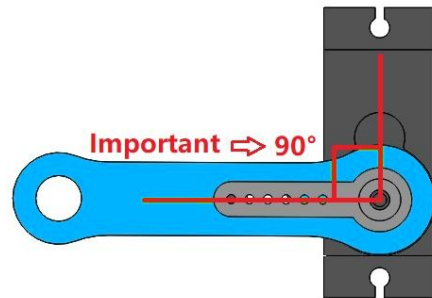
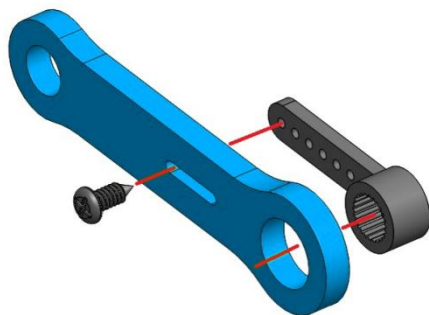


Step 3 - Assembly Servo-3

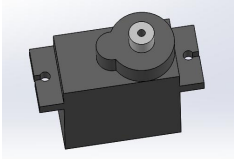
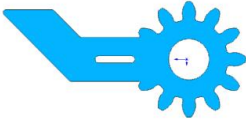
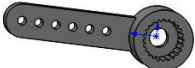
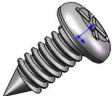
Servo-3	Acrylic board 1PCS

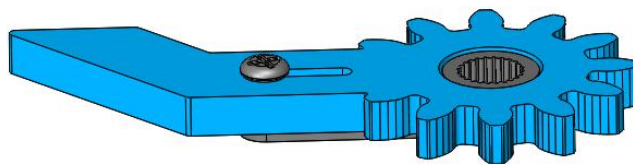
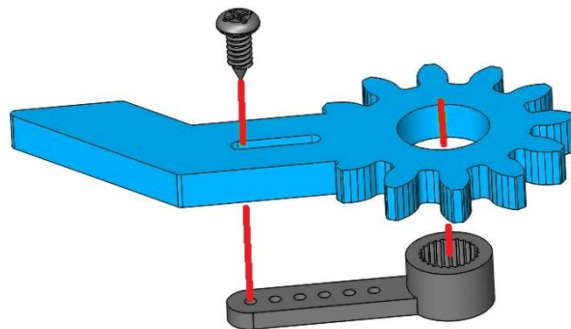
SIYEENOVE

Servo horn 1PCS	Bag No.④ M1.4X5 self-tapping screw 1PCS	M2.5X4mm screw 1PCS
		

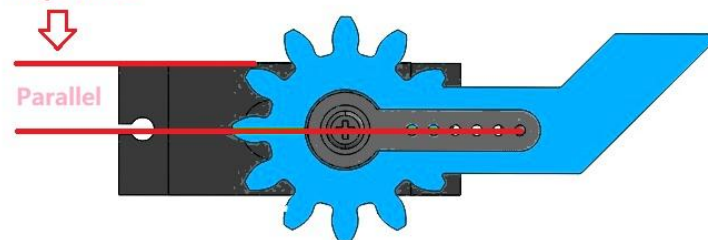


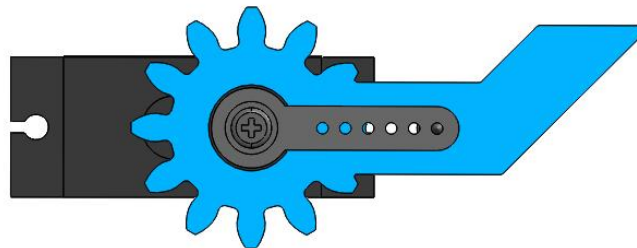
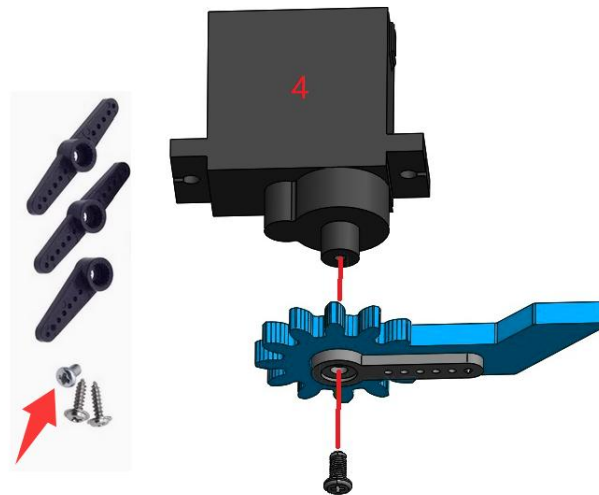
Step 4 - Assembly Servo-4

Servo-4		Acrylic board 1PCS	
			
Servo horn 1PCS	Bag No. ④ M1.4X5 self-tapping screw 1PCS		M2.5X4mm screw 1PCS
			



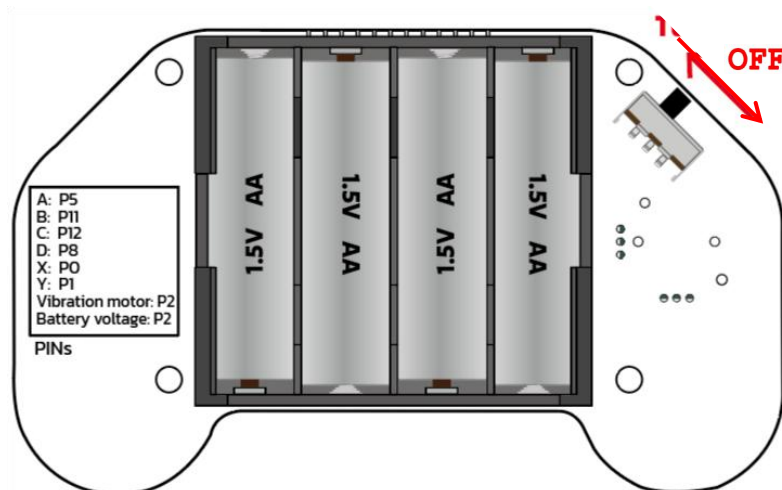
Important



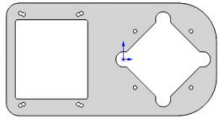


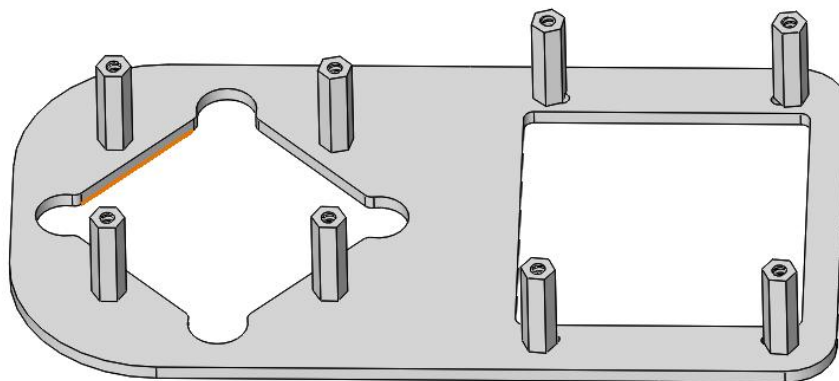
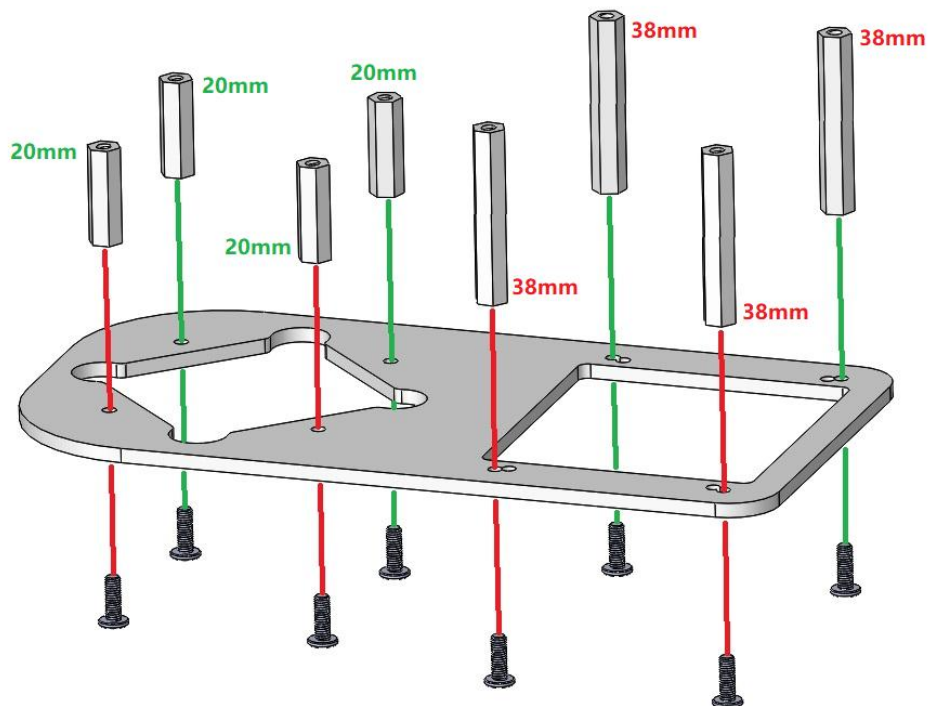
Step 5 - Power off the Joystick and disconnect servos

To prepare for the next assembly steps: Turn off the handle's power and unplug all servo cables from their ports.

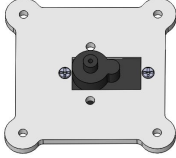
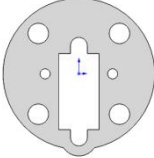
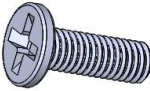
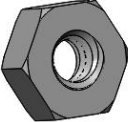


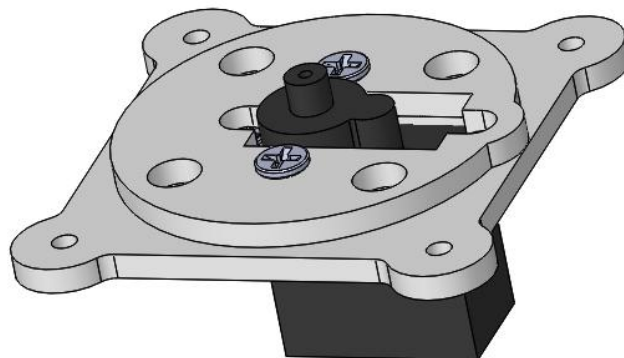
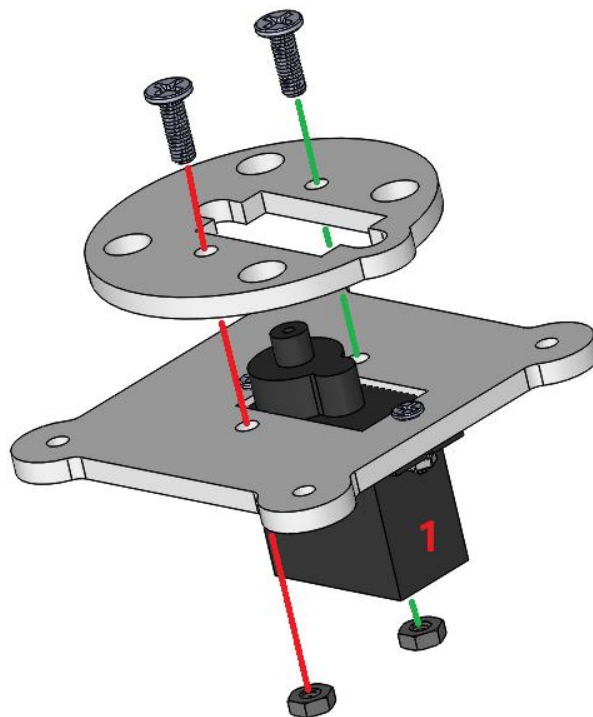
Step6 - Assembly Base (Nylon Standoffs)

Acrylic board	Bag No. ⑪ M3x20MM Nylon Standoff 4PCS	Bag No. ⑫ M3x38MM Nylon Standoff 4PCS	Bag No. ② M3X9MMscrew 8PCS
			

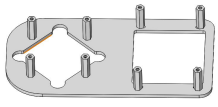
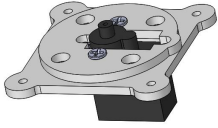
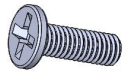


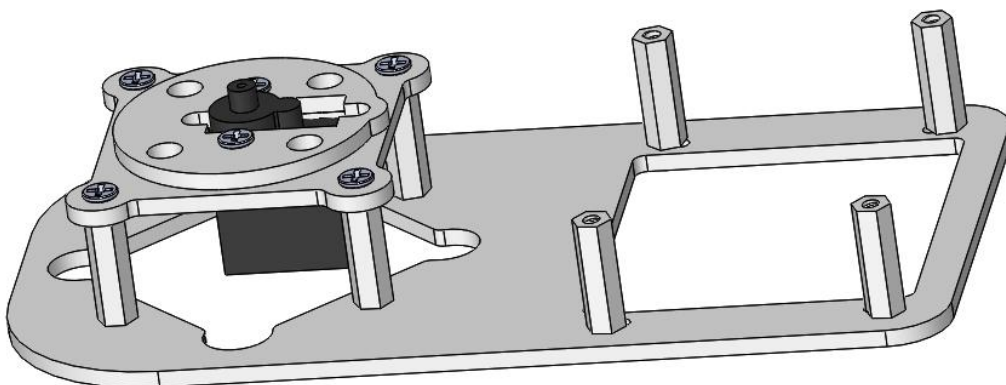
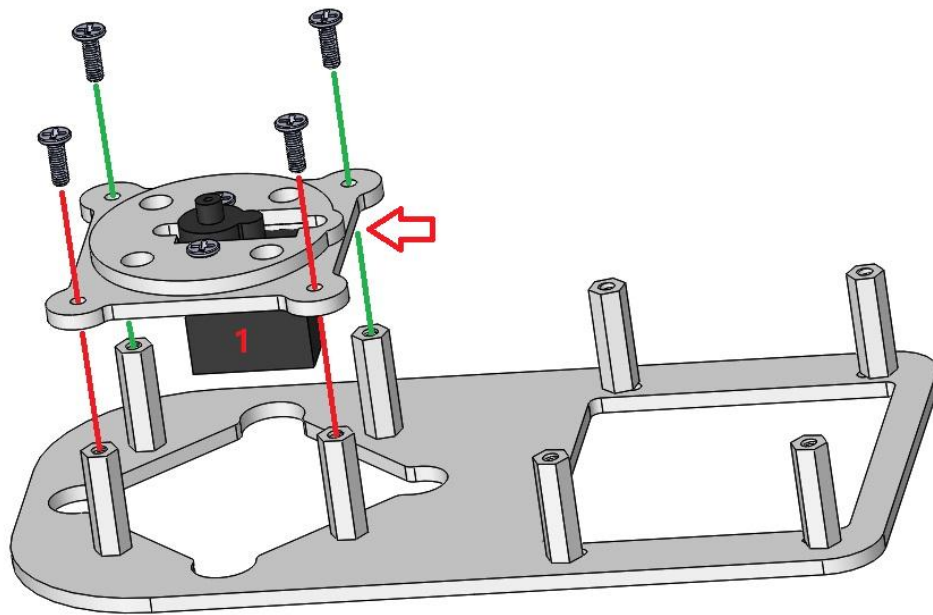
Step7 - Assembly Base (Steel Ball Bearing Plate)

The structure in Step 1	Acrylic board	Bag No.② M3X9MMscrew2PCS	Bag No.⑤ M3 nut 2 PCS
			

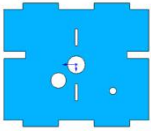


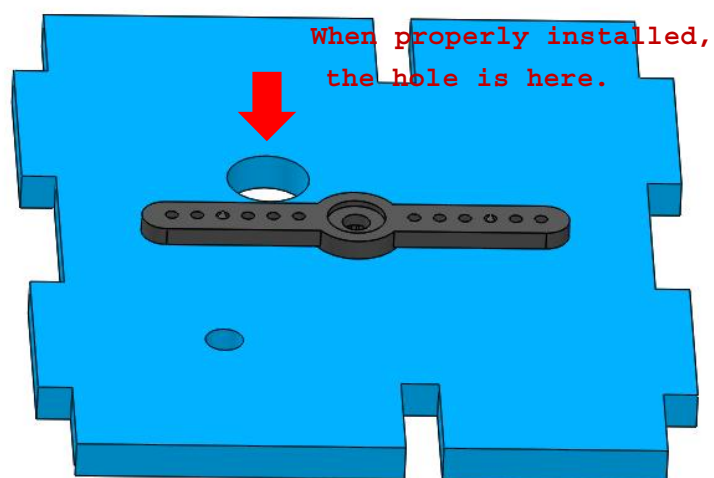
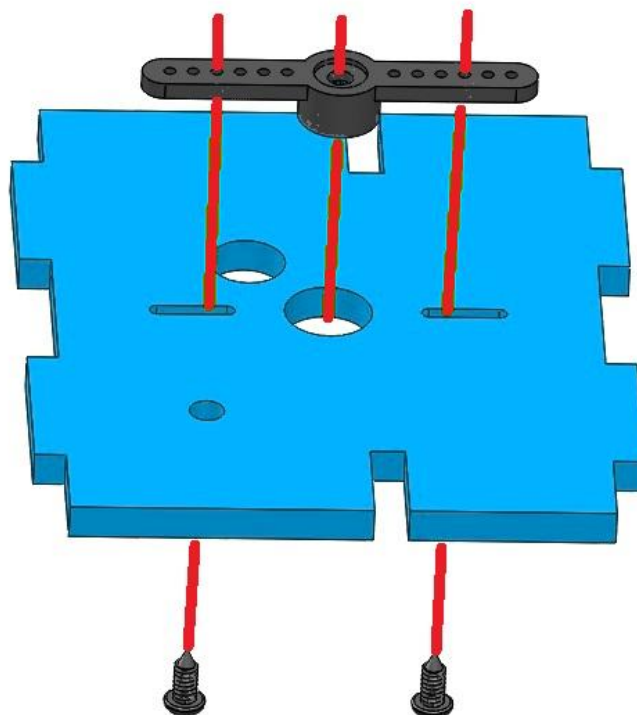
Step 8 - Assembly Base (Servo Structure)

The structure in Step 6	The structure in Step 7	Bag No.② M3X9MMscrew4PCS
		

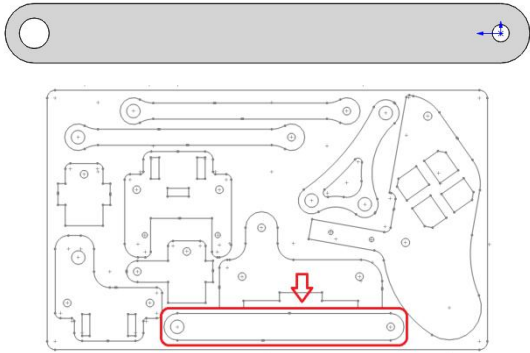
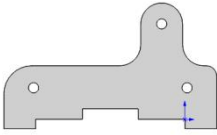


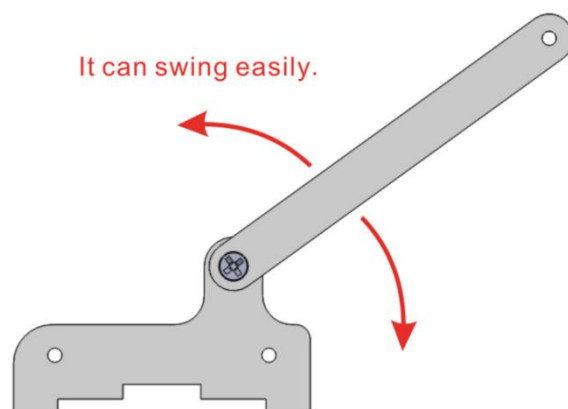
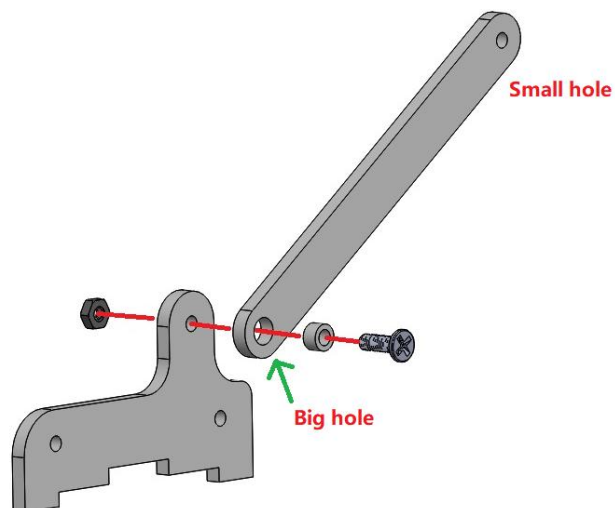
Step 9 - Attach Servo Horn to Rotating Base

Acrylic board	Servo horn 1PCS	Bag No.④ M1.4X5 self-tapping screw 2PCS
		

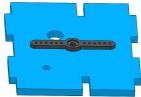
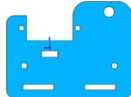
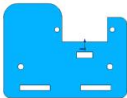
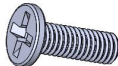


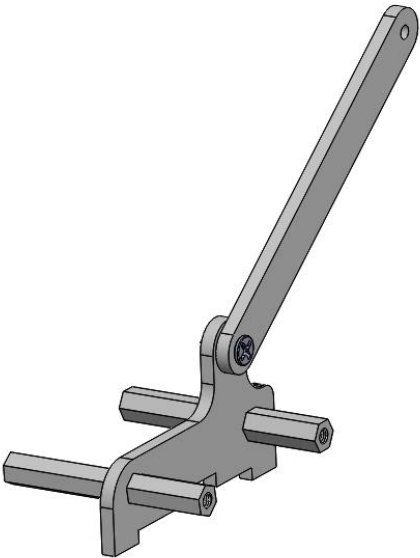
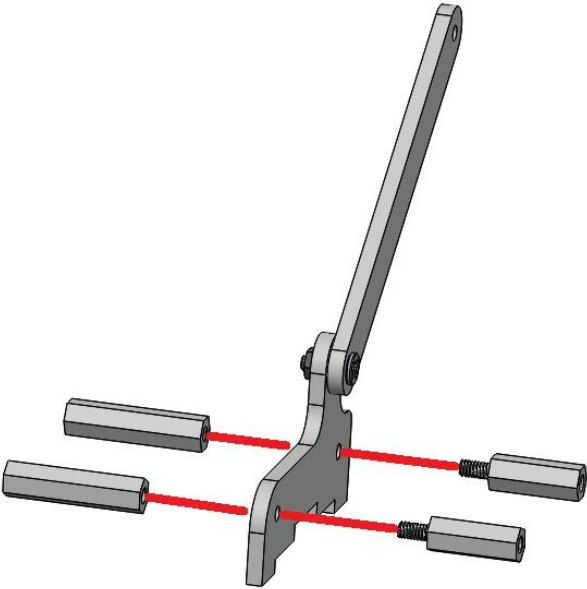
Step 10 - Upper Arm Link Assembly

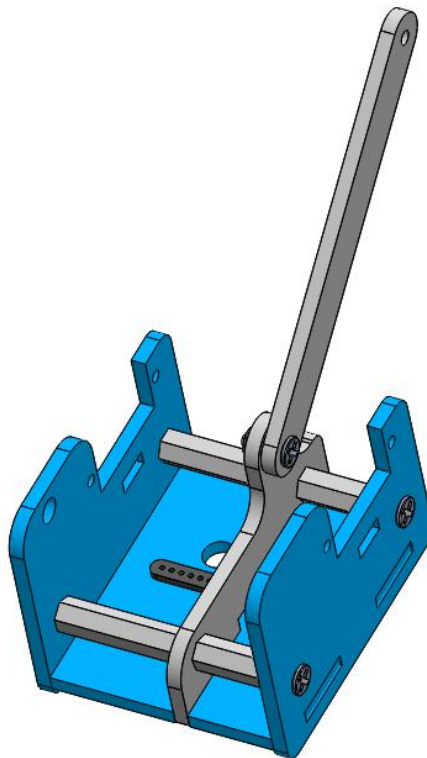
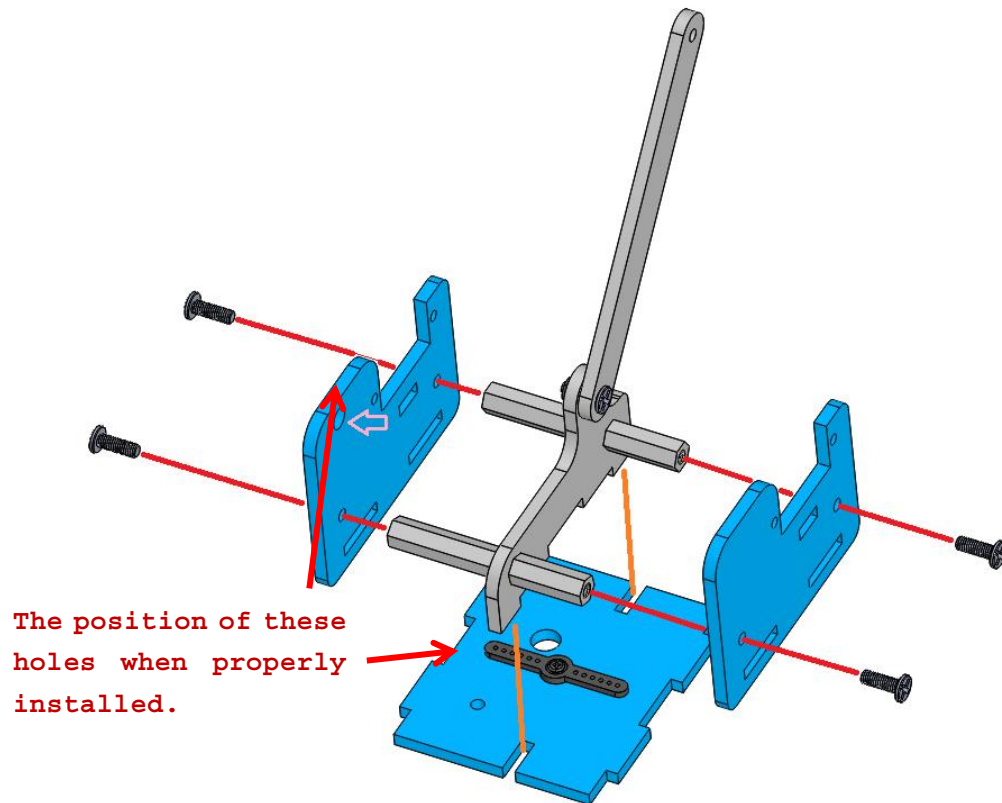
Acrylic board		Acrylic board
		
Bag No. ⑬ 5x3.2x3MM spacer 1PCS	Bag No. ② M3X9MM screw 1PCS	Bag No. ⑤ M3 nut 1PCS
		



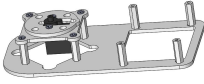
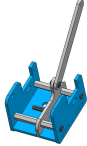
Step 11 - Assemble Rotating Base

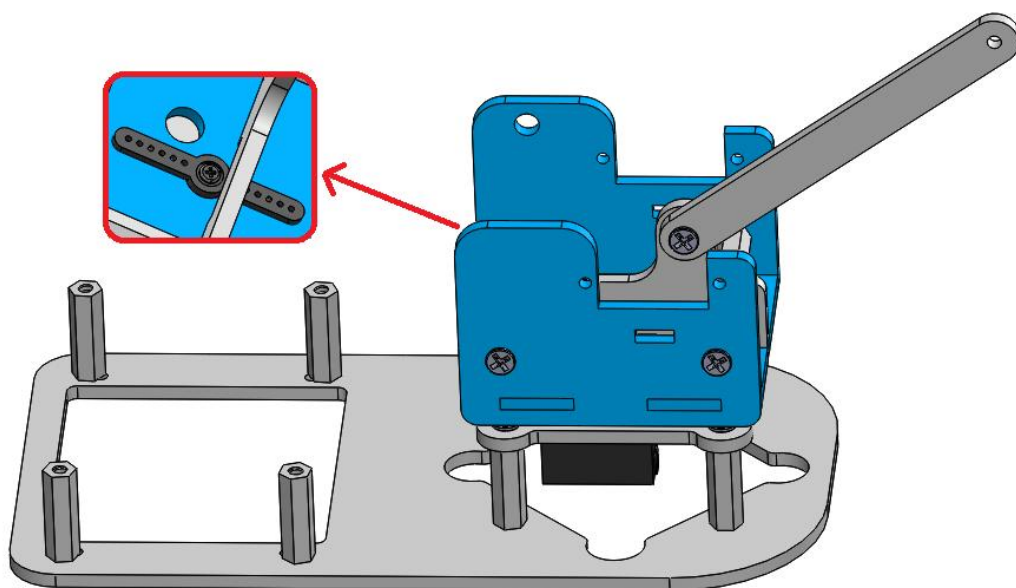
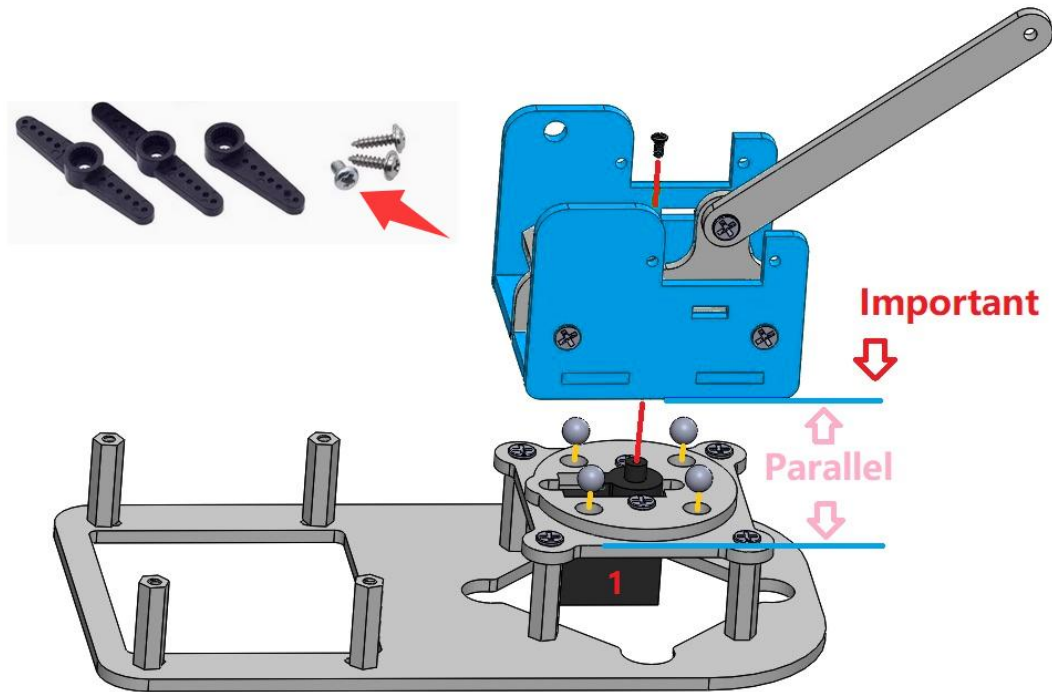
The structure in Step 9	The structure in Step 10		Acrylic board 1PCS
			
Acrylic board 1PCS	Bag No.⑧ M3x17+6MMNylon Standoff 2PCS	Bag No.⑨ M3x26MMNylon Standoff 2PCS	Bag No.② M3X9MMscrew4PCS
			



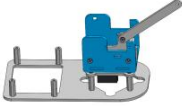


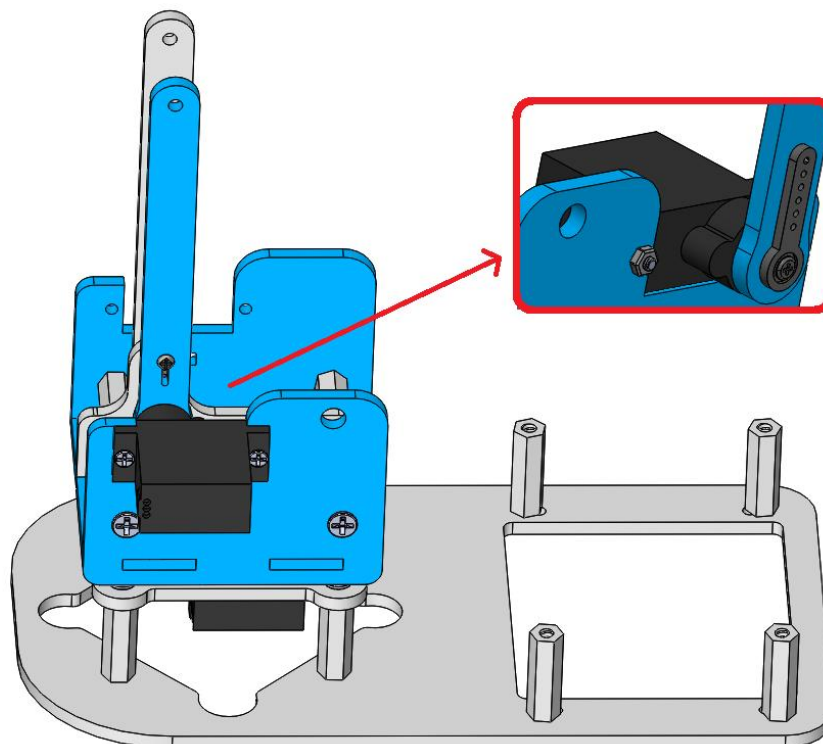
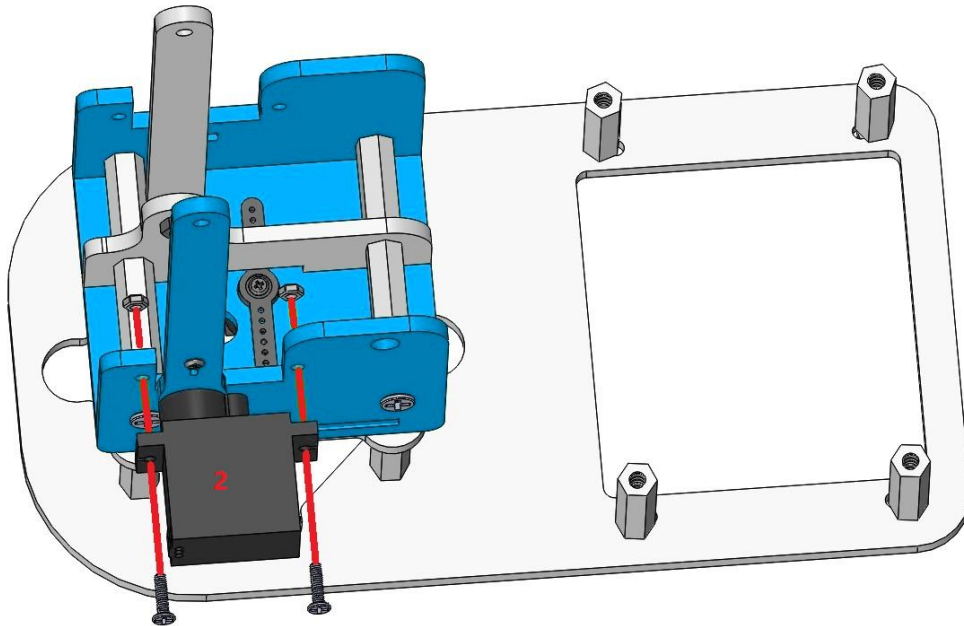
Step 12 - Mount Rotating Base to Main Frame

The structure in Step 8	The structure in Step 11	Steel board 4PCS	M2.5X4mm screw 1PCS
			

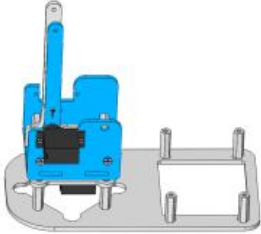
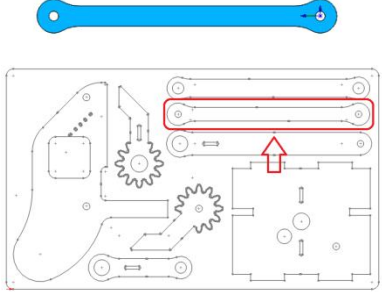
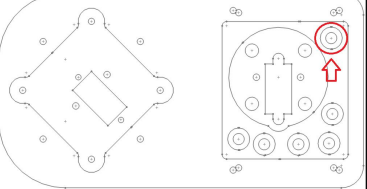


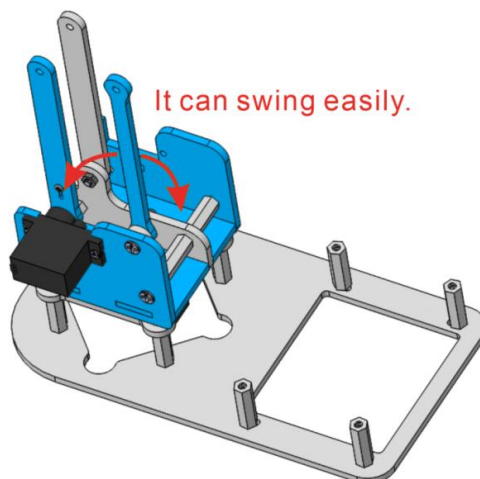
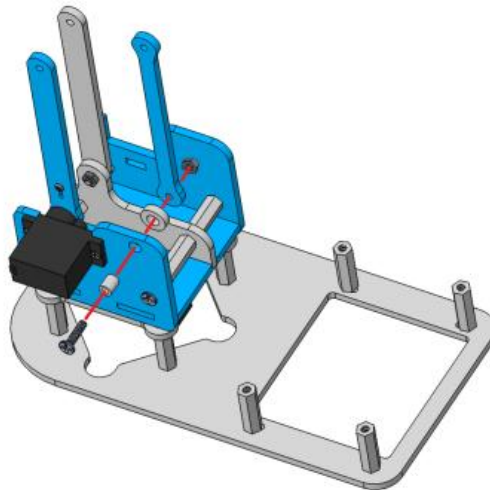
Step 13 - AAssembly Left Servo Structure on Turntable

The structure in Step 2	The structure in Step 12	Bag No.③ M2x8mm screw 2PCS	Bag No.⑥ M2 nut 2PCS
			



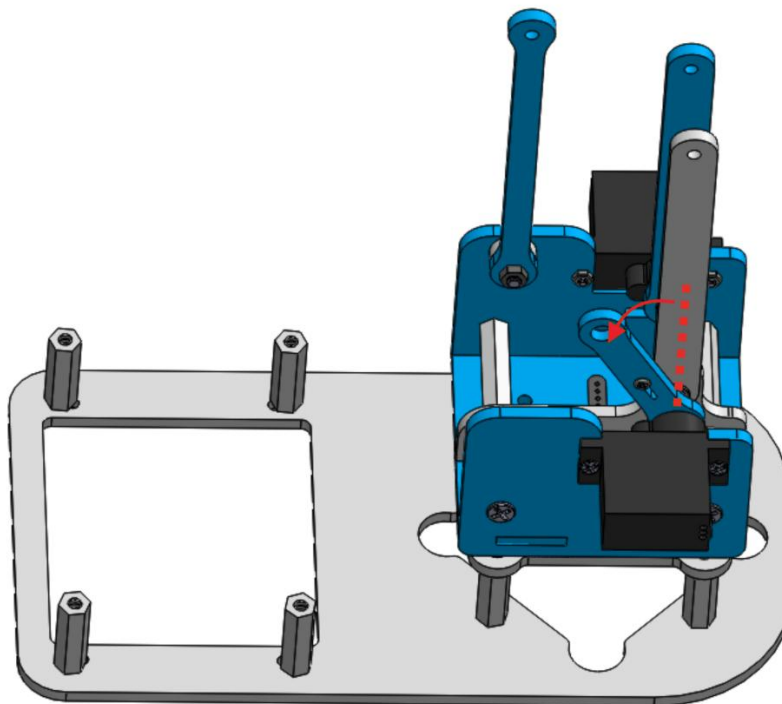
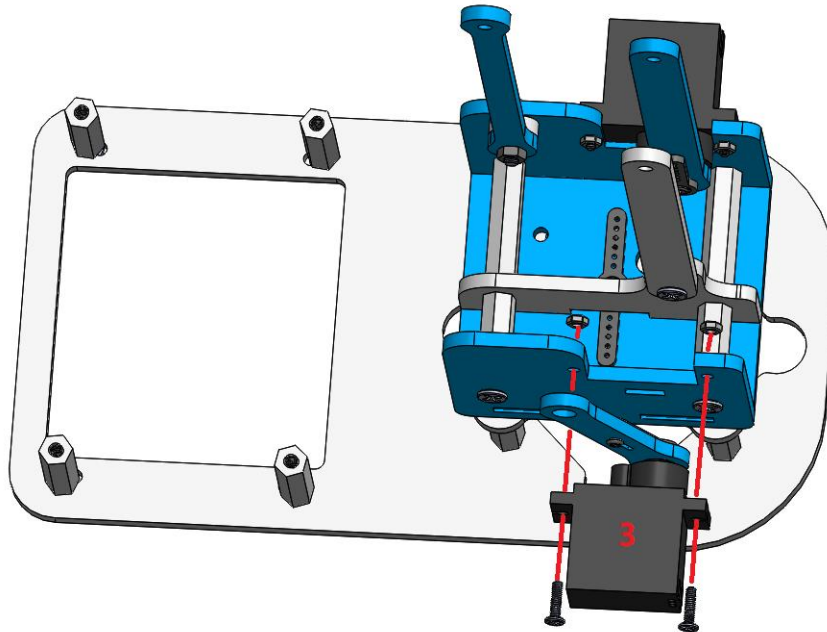
Step 14 - Attach Upper Arm Linkage (Left Side)

The structure in Step 2	The structure in Step 12	Bag No. ③ M2x8mm screw 2PCS
		 
Bag No. ⑭ 5x3.2x6MM spacer 1PCS	Bag No. ① M3X12MM screw 1PCS	Bag No. ⑤ M3 nut 1PCS
		

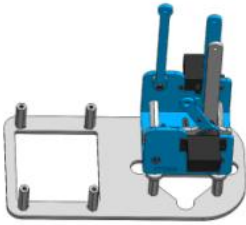
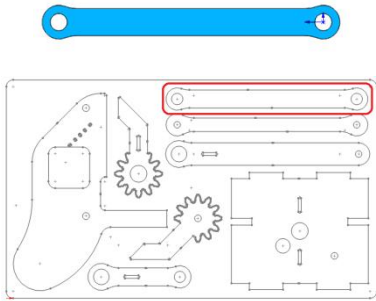
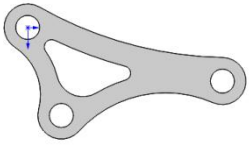


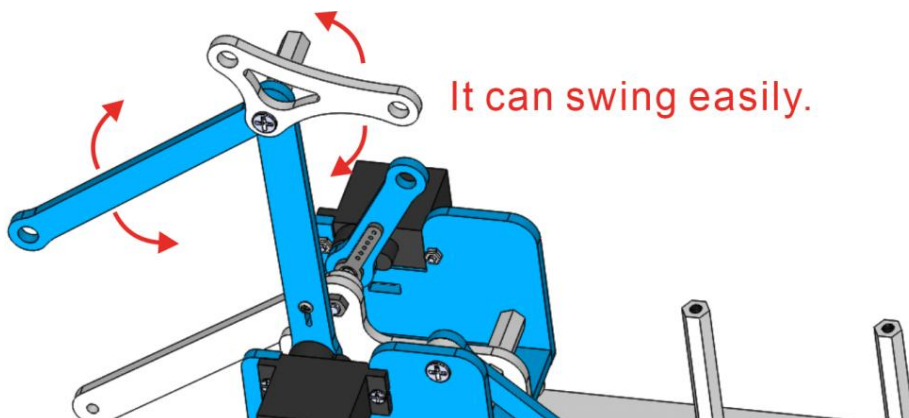
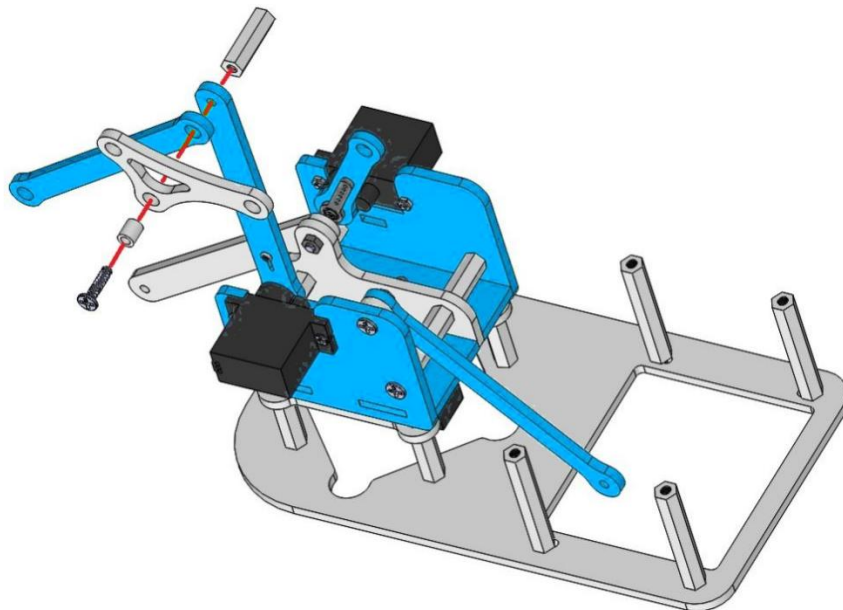
Step 15 - Assembly Right Servo Structure on Turntable

The structure in Step 14	The structure in Step 3	Bag No. ③ M2x8mm screw 2PCS	Bag No. ⑥ M2 nut 2PCS
			

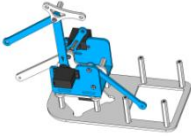


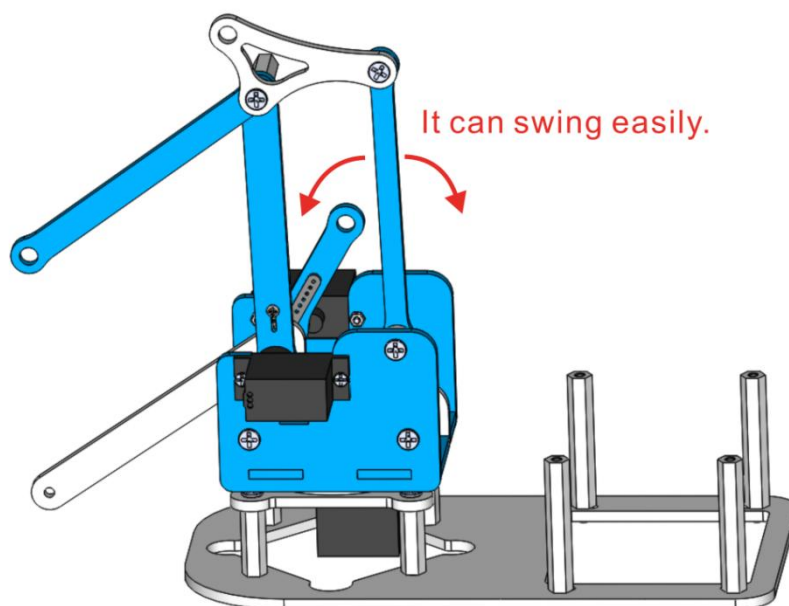
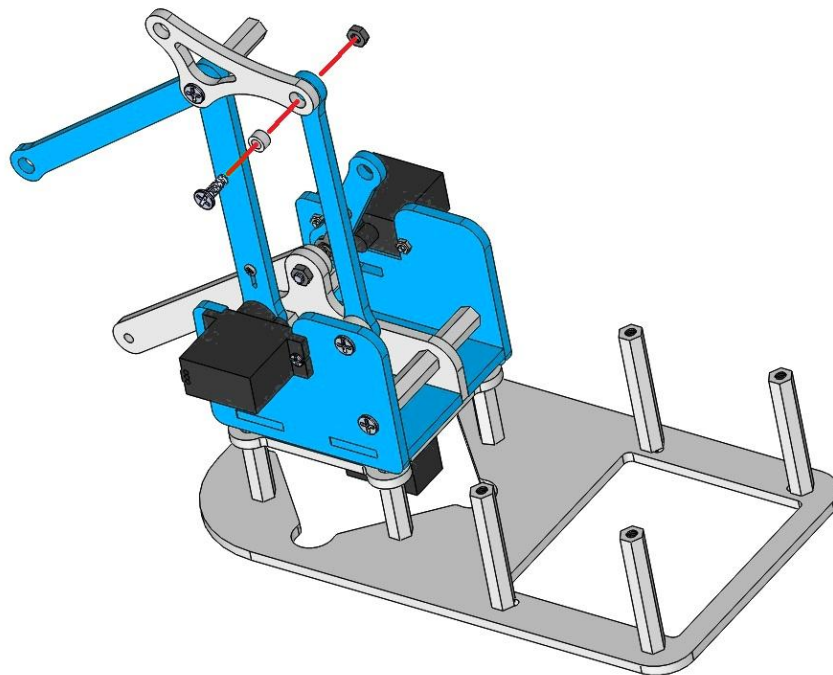
Step 16 - Assembly Primary Arm/Elbow Connector(1)

The structure in Step 15	Acrylic board 1PCS	Acrylic board 1PCS
		
Bag No. ⑭ 5x3.2x6MM spacer 1PCS	Bag No. ⑪ M3x20MM Nylon Standoff 1PCS	Bag No. ① M3X12MM screw 1PCS
		

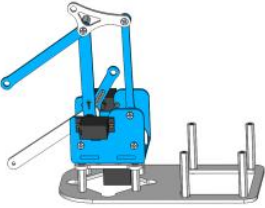
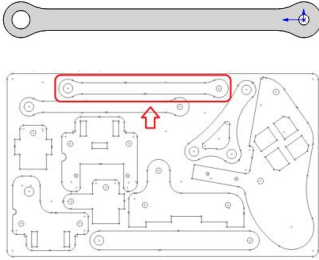
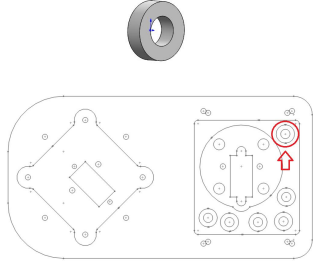


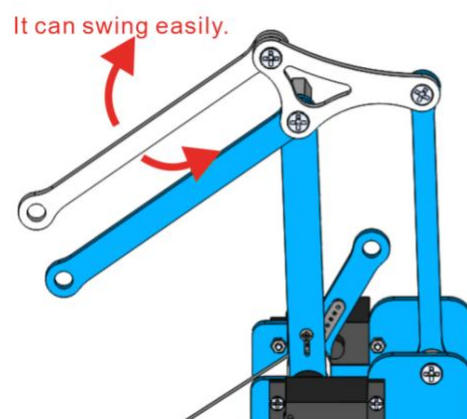
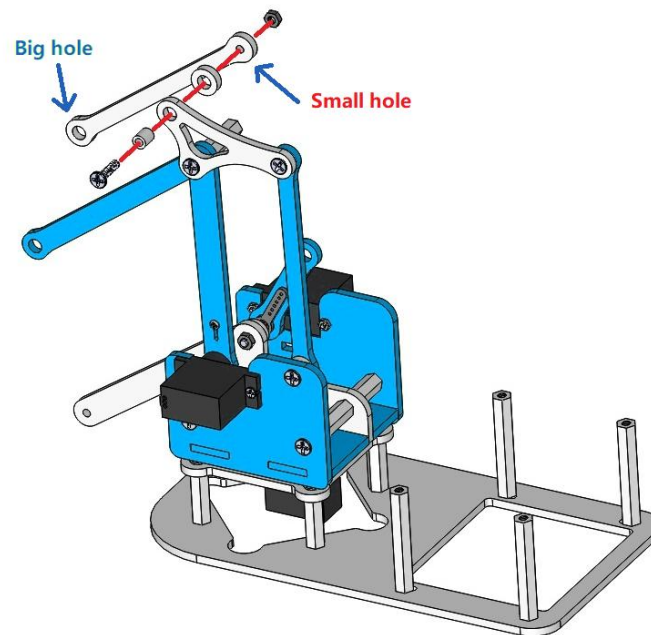
Step 17 - Assembly Primary Arm/Elbow Connector(2)

The structure in Step 16	Bag No. ⑬ 5x3.2x3M Mspacer 1PCS	Bag No. ② M3X9MM screw 1PCS	Bag No. ⑤ M3 nut 1PCS
			

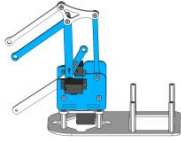


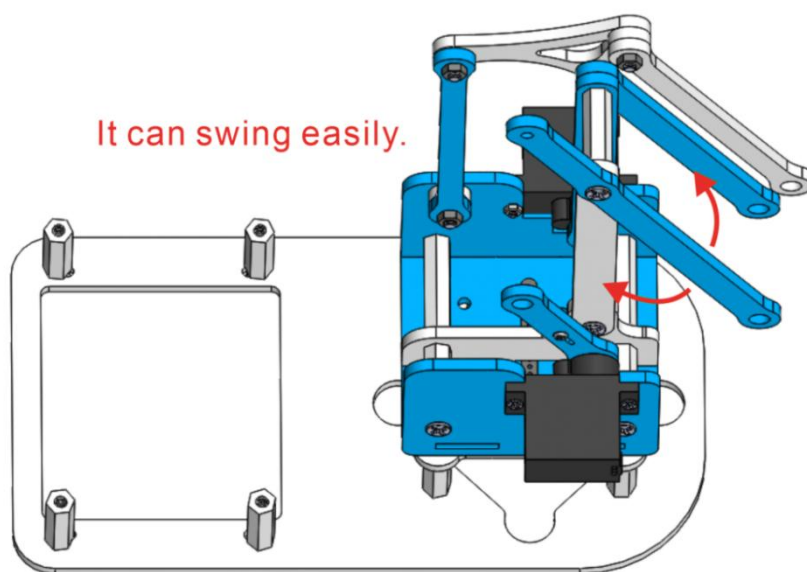
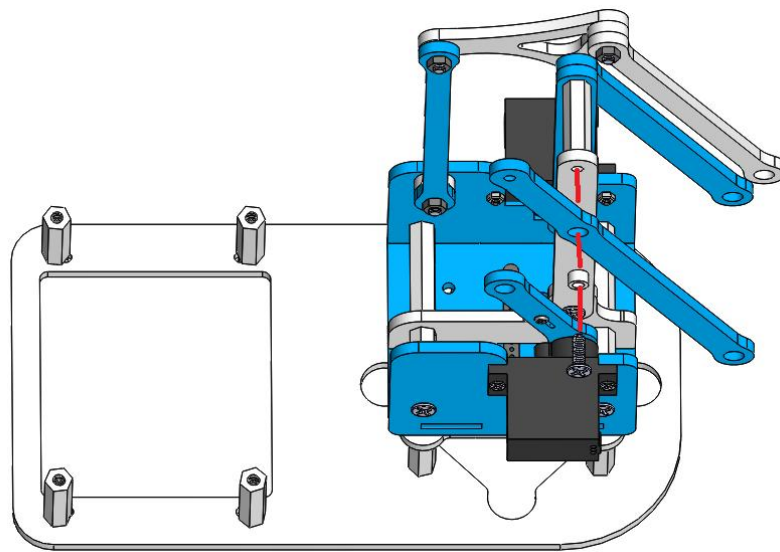
Step 18 - Attach Forearm Linkage (First Position)

The structure in Step 17	Acrylic board 1PCS	Acrylic spacer 1PCS
		
Bag No. ⑭ 5x3.2x6MM spacer 1PCS	Bag No. ① M3X12MM screw 1PCS	Bag No. ⑤ M3 nut 1PCS
		

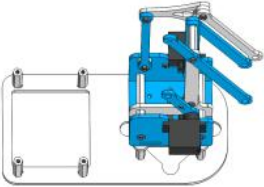
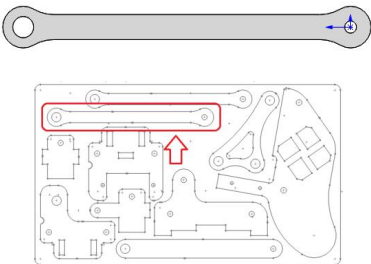


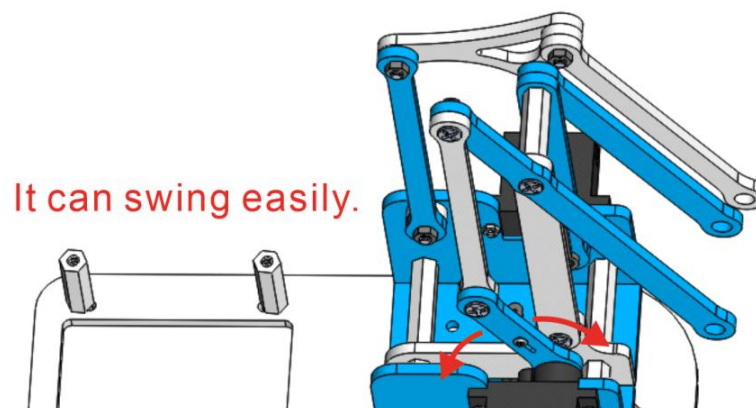
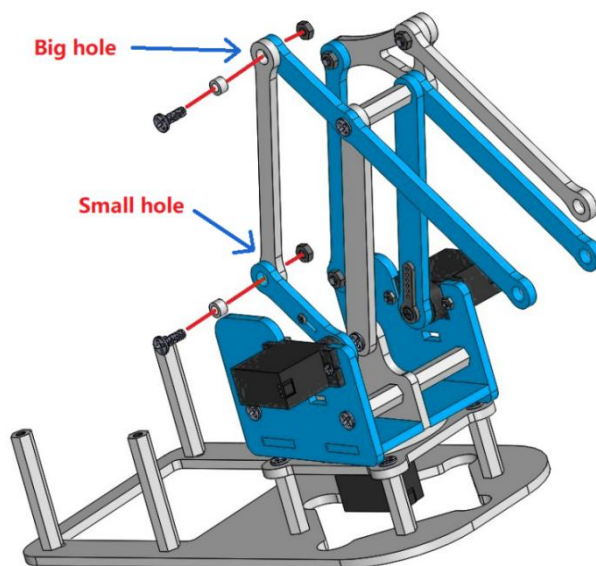
Step 19 - Attach Forearm Linkage (Second Position)

The structure in Step 18	Acrylic board 1PCS
	
Bag No. ⑬ 5x3.2x3MM spacer 1PCS	Bag No. ② M3X9MM screw 1PCS
	

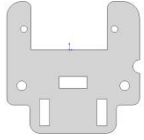


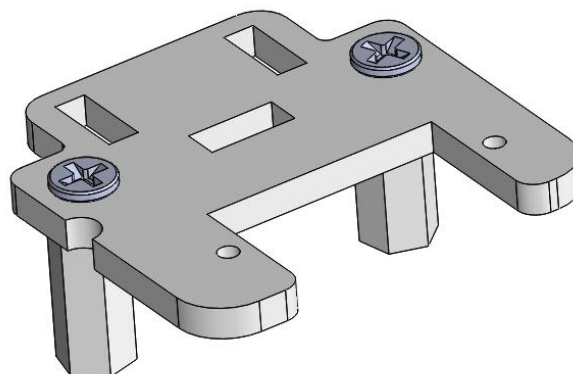
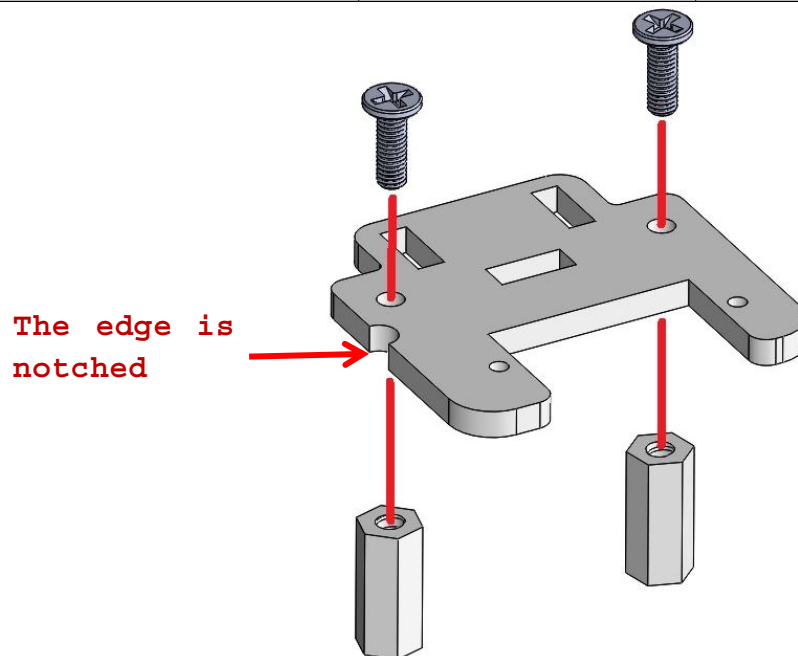
Step 20 - Connect Linkage between Upper Arm and Forearm

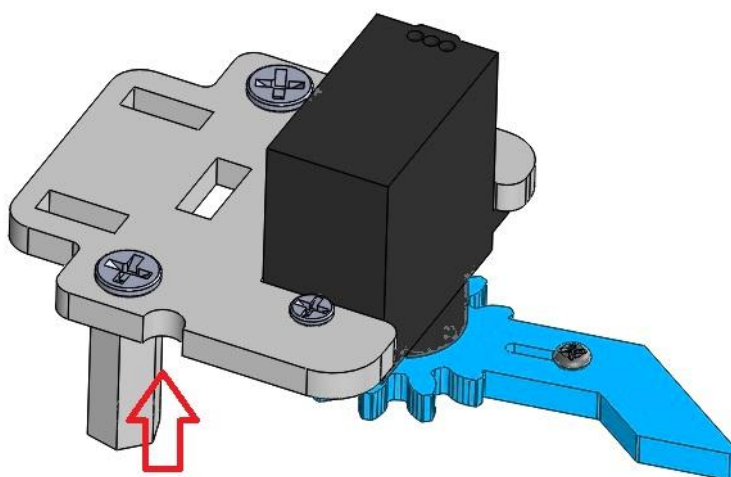
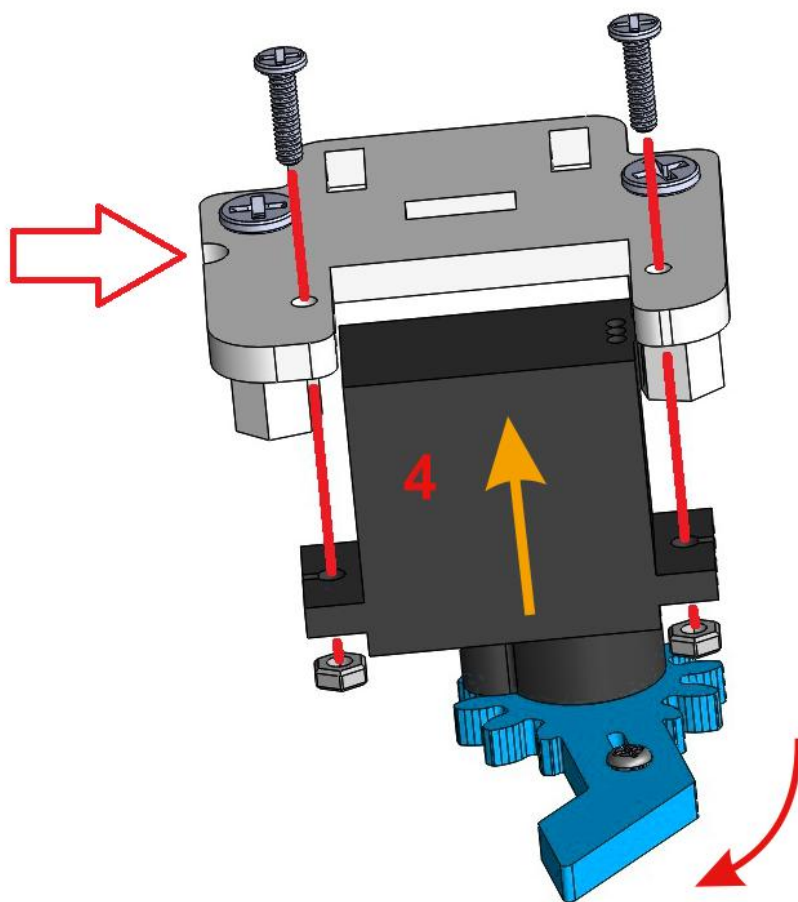
The structure in Step 19		Acrylic board 1PCS	
			
Bag No. ⑬ 5x3.2x3MM spacer 2PCS	Bag No. ② M3X9MM screw 2PCS	Bag No. ⑤ M3 nut 2PCS	
			



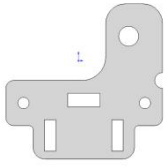
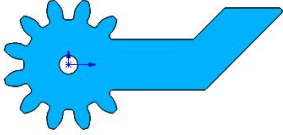
Step 21 - Mount Left Gripper Jaw

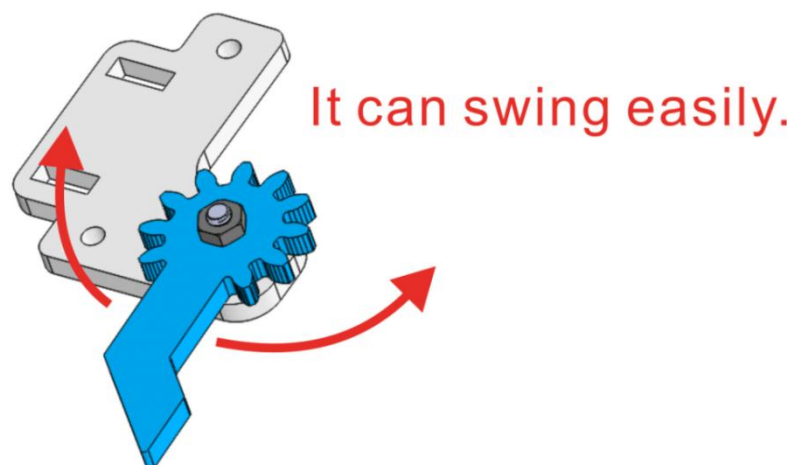
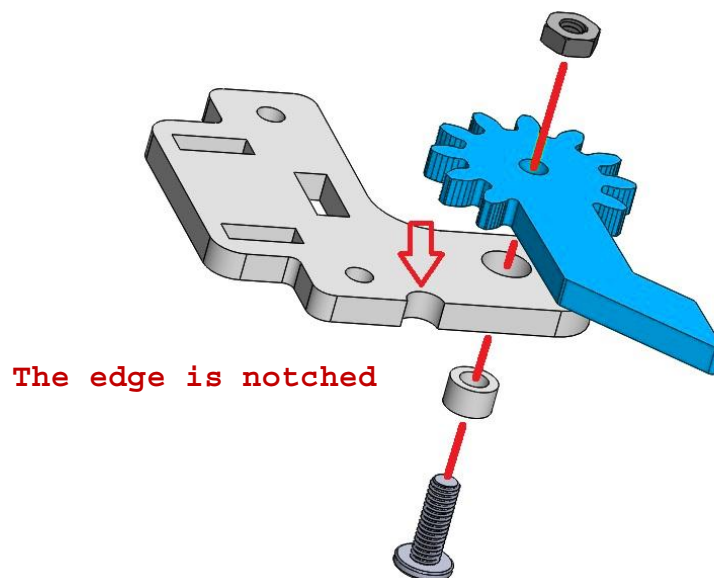
Acrylic board 1PCS	Bag No.② M3X9MM screw 2PCS	Bag No.⑦ M3x14MM Nylon Standoff 2PCS
		
The structure in Step 4	Bag No.③ M2x8mm screw 2PCS	Bag No.⑥ M2 nut 2PCS
		



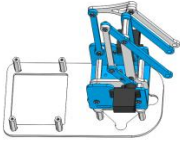
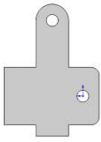
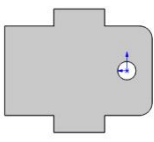


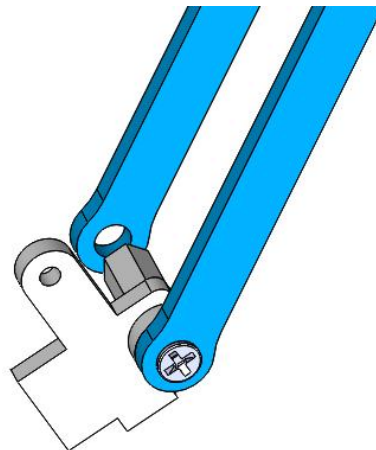
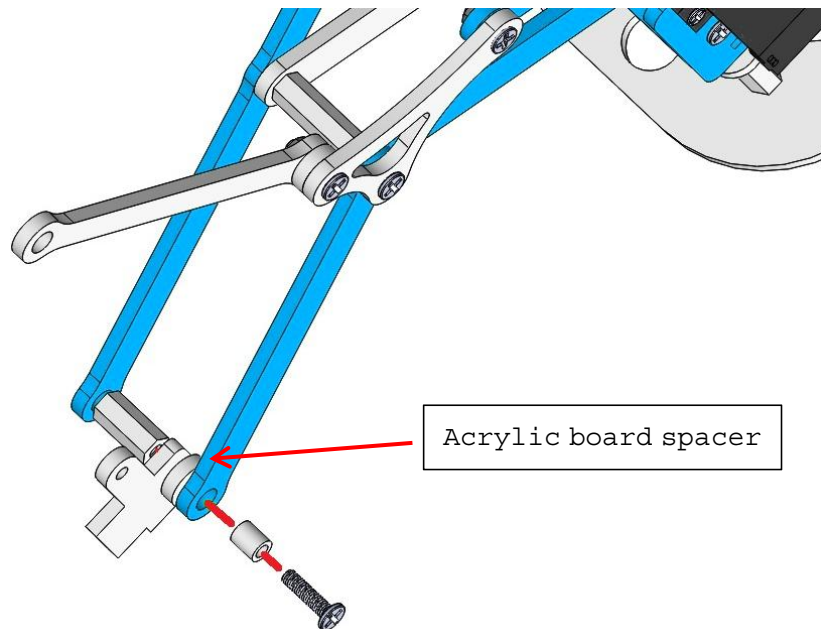
Step 22 - Mount Right Gripper Jaw

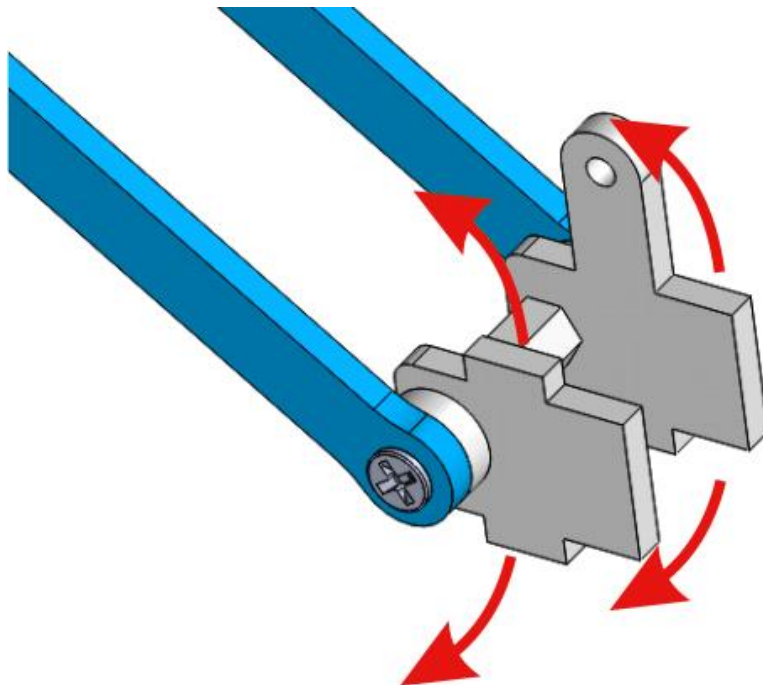
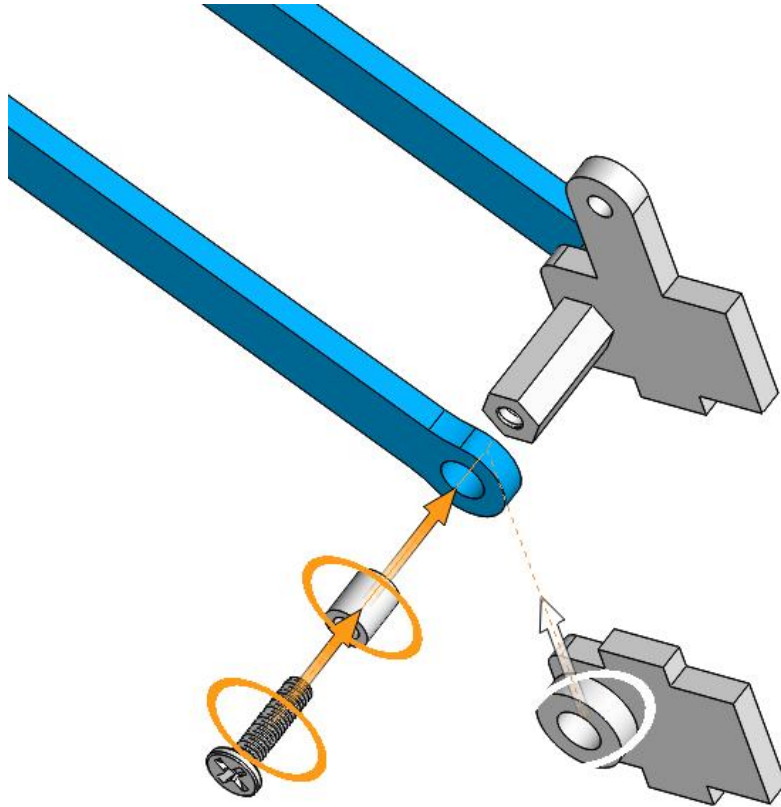
Acrylic board 1PCS		Acrylic board 1PCS	
			
Bag No. ⑬ 5x3.2x3MM spacer 1PCS	Bag No. ② M3X9MM screw 1PCS	Bag No. ⑤ M3 nut 1PCS	
			



Step 23 - Assembly Wrist Structure

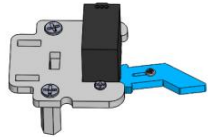
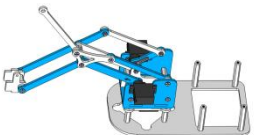
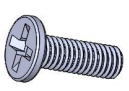
The structure in Step 20	Acrylic board 1PCS	Acrylic board 1PCS	Acrylic board spacer 2PCS
			
Bag No. ① M3X12MM screw 2 PCS	Bag No. ⑦ M3x14MM Nylon Standoff 1PCS		Bag No. ⑭ 5x3.2x6MM spacer 2PCS
			

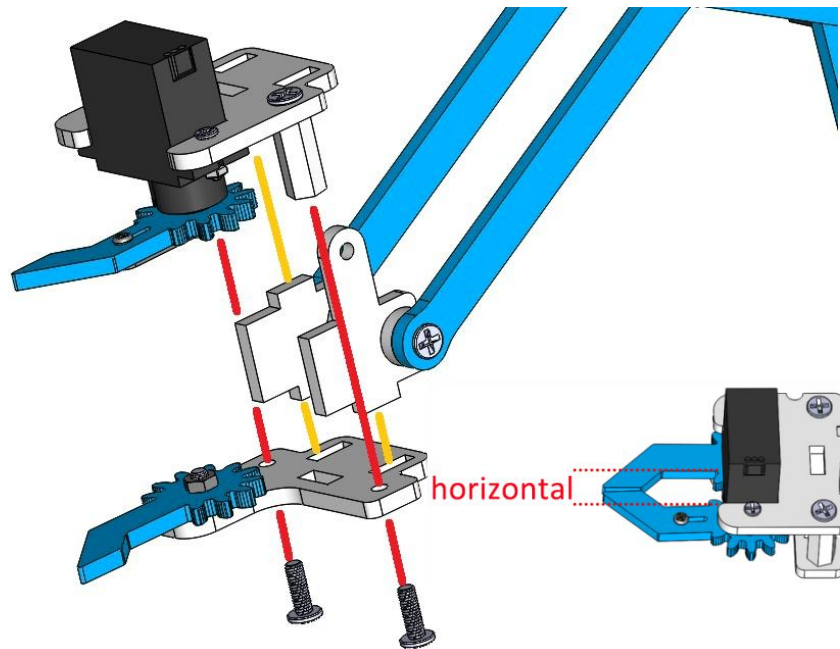




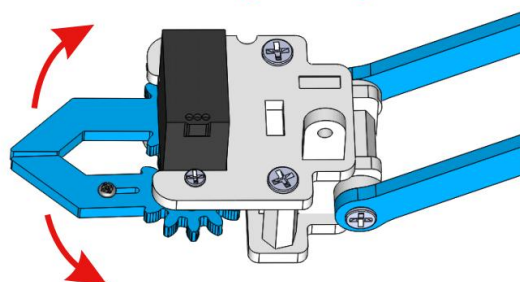
It can swing easily.

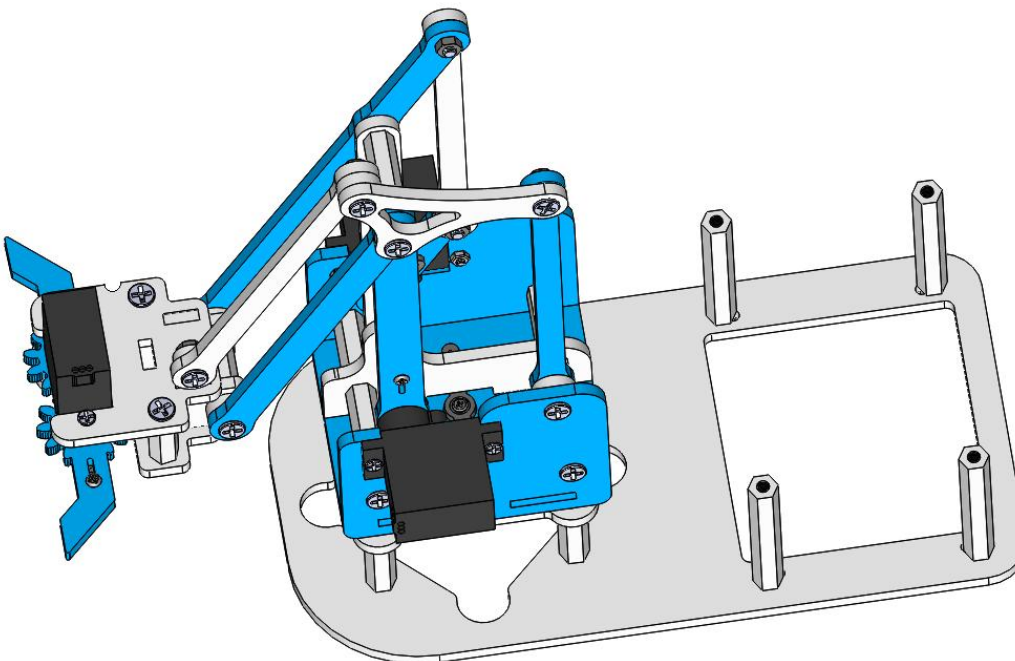
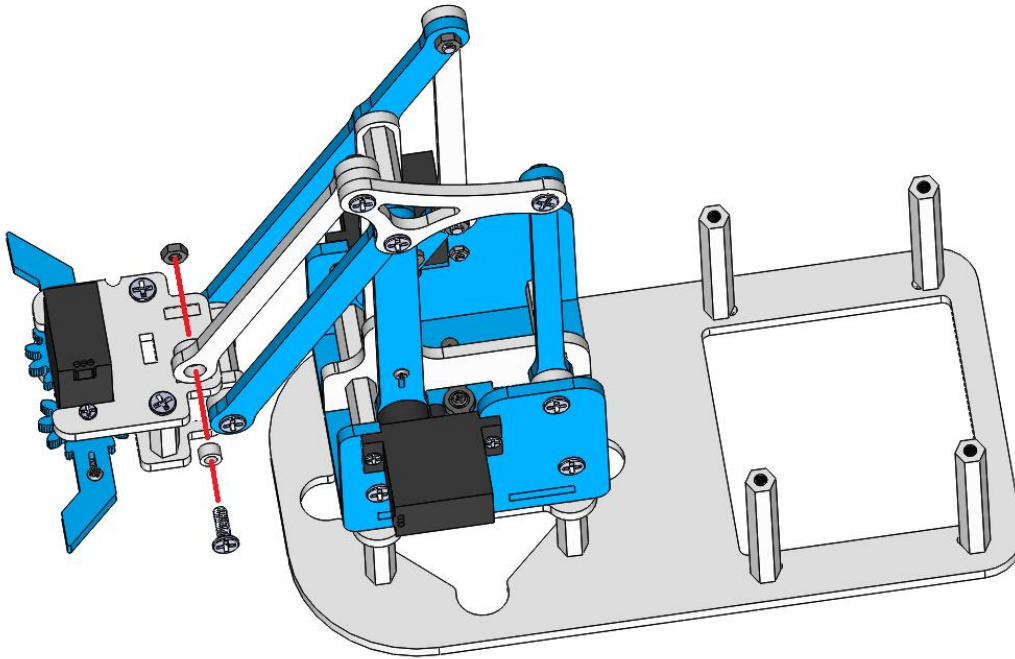
Step 24 - Attach Gripper to Wrist

The structure in Step 21	The structure in Step 22
	
The structure in Step 23	Bag No. ② M3X9MMscrew 3PCS
	
Bag No. ⑤ M3 nut 1PCS	Bag No. ⑬ 5x3.2x3MM spacer 1PCS
	

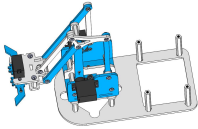


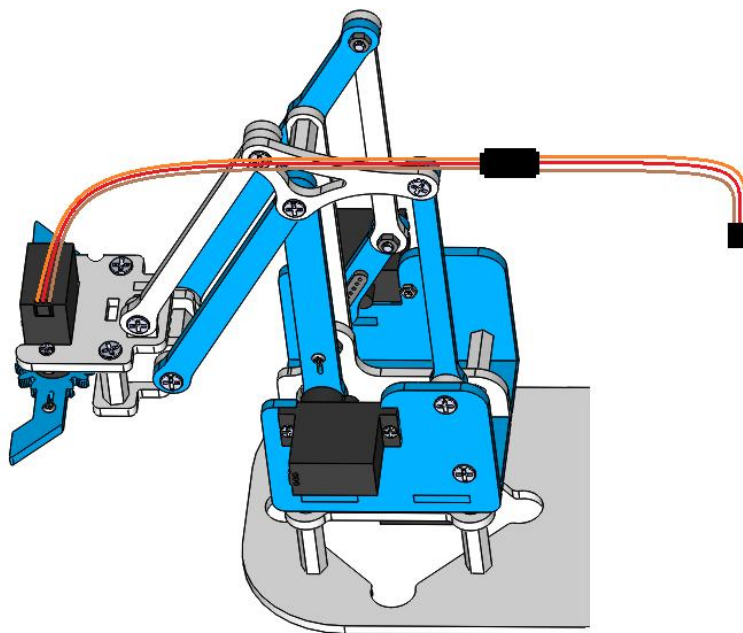
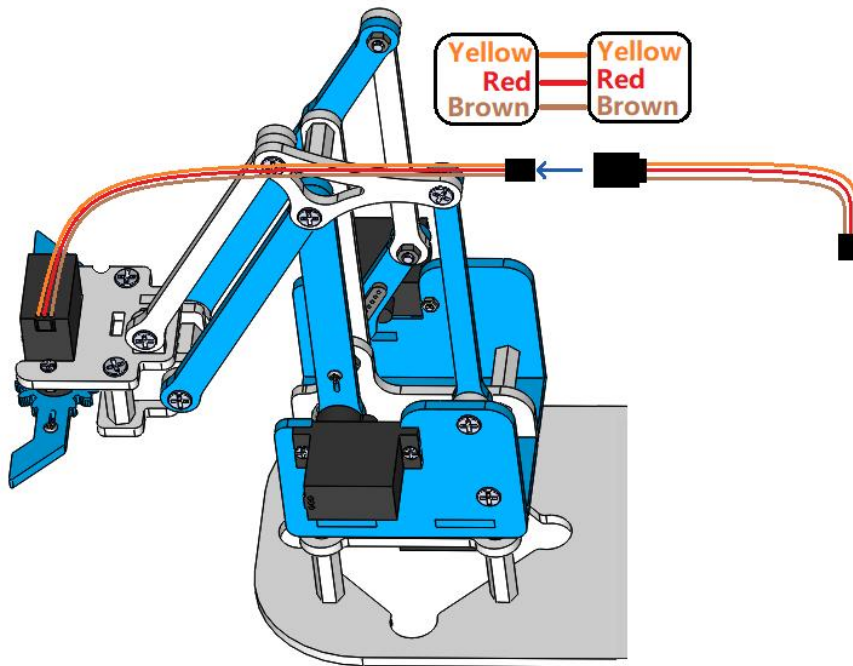
It can swing easily.



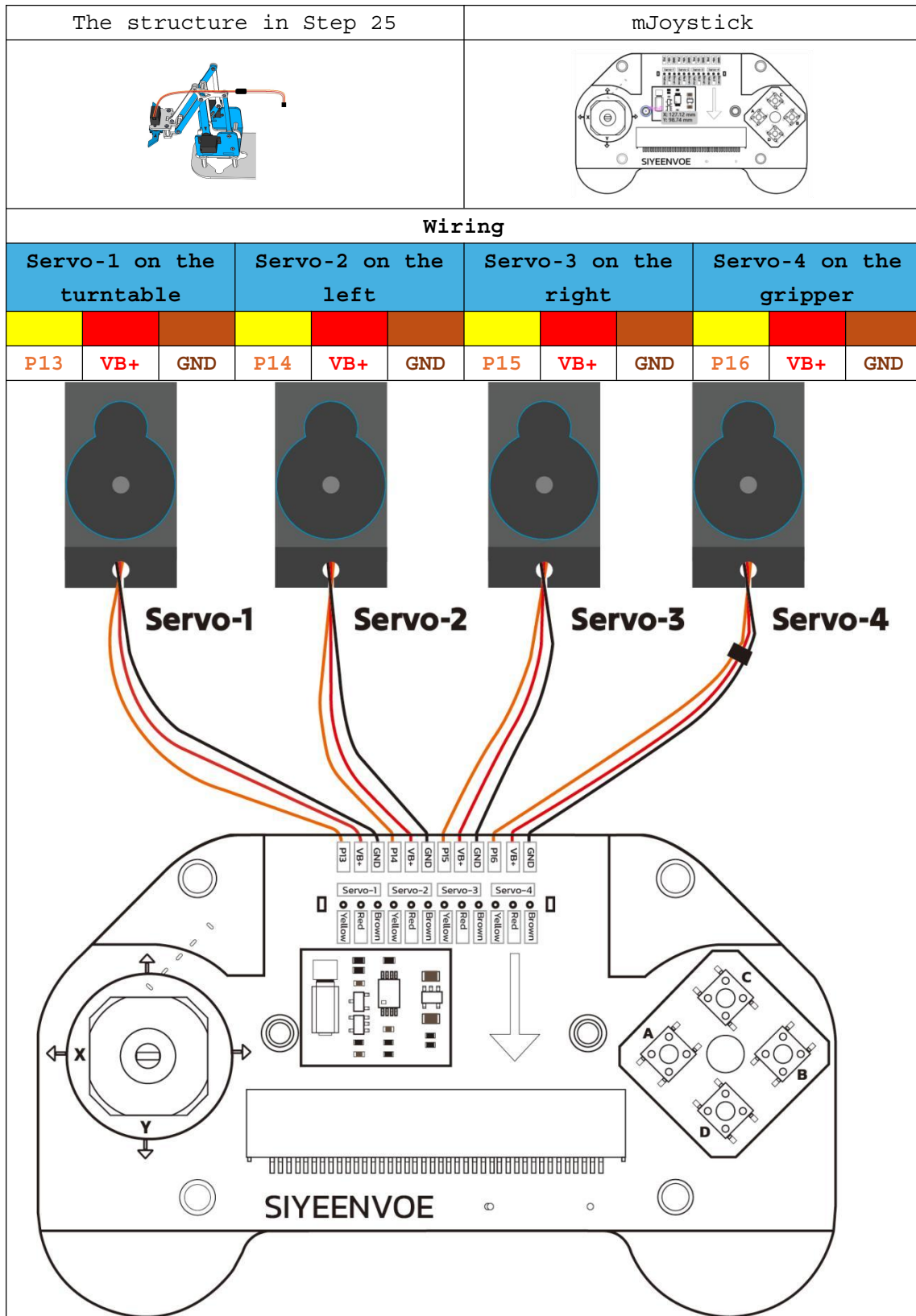


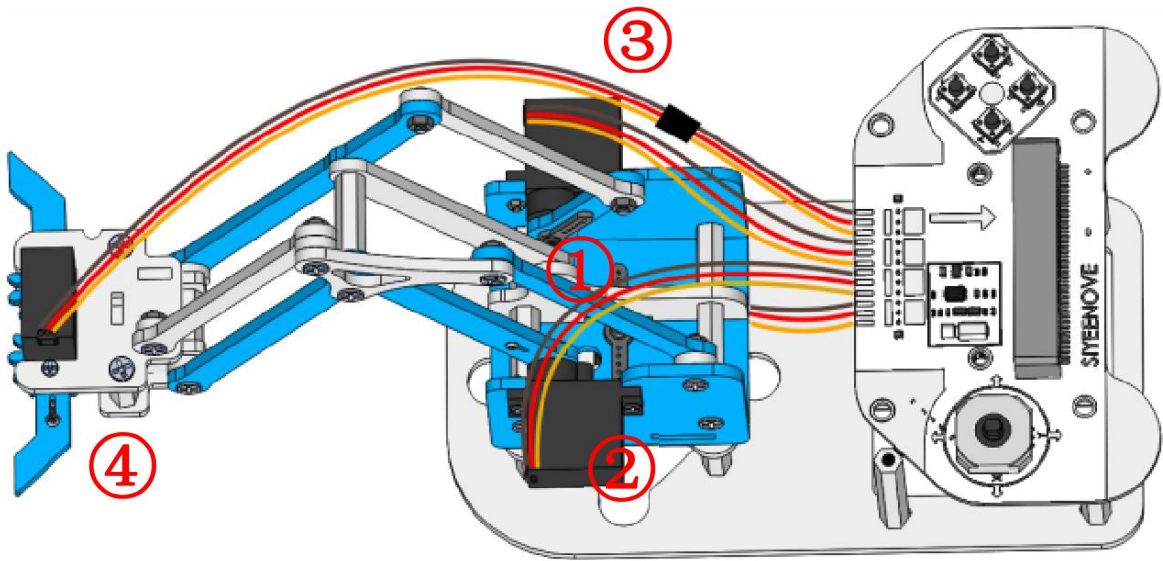
Step 25 - Connect Servo-4 with Extension Cable

The structure in Step 24	3P servo extension cable 1PCS
	

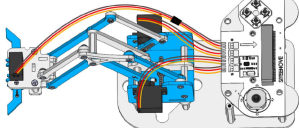
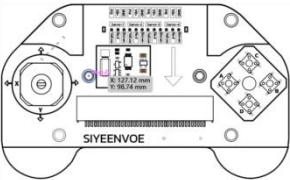


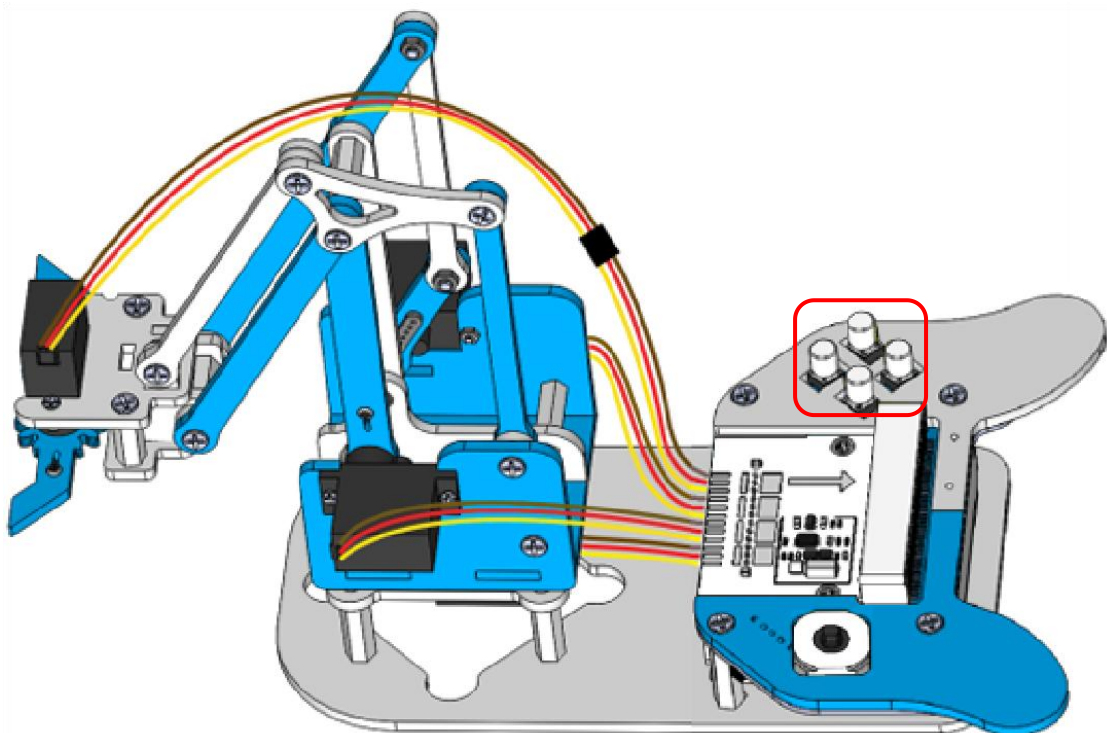
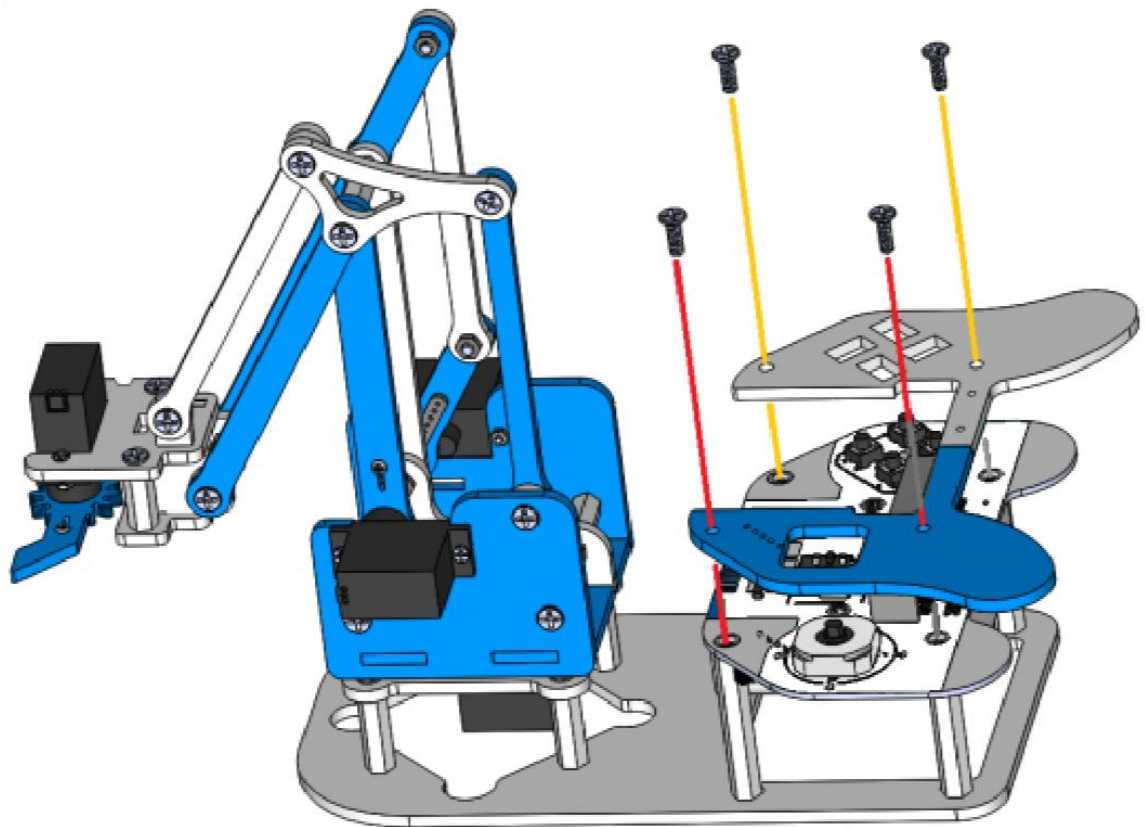
Step 26 - Wire Servos to Joystick Controller





Step 27 - fix mJoystick to Base Frame

The structure in Step 26	mJoystick	Bag No.② M3X9MM screw 4PCS
		
Acrylic board 1PCS	Acrylic board 1PCS	Bag No.⑱ Button cup 4PCS
		



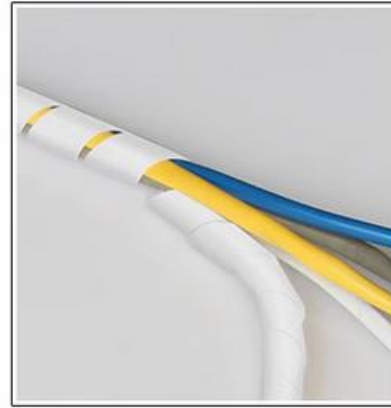
Step 28 - Cable Management

► Spiral Wrap Tube Application:

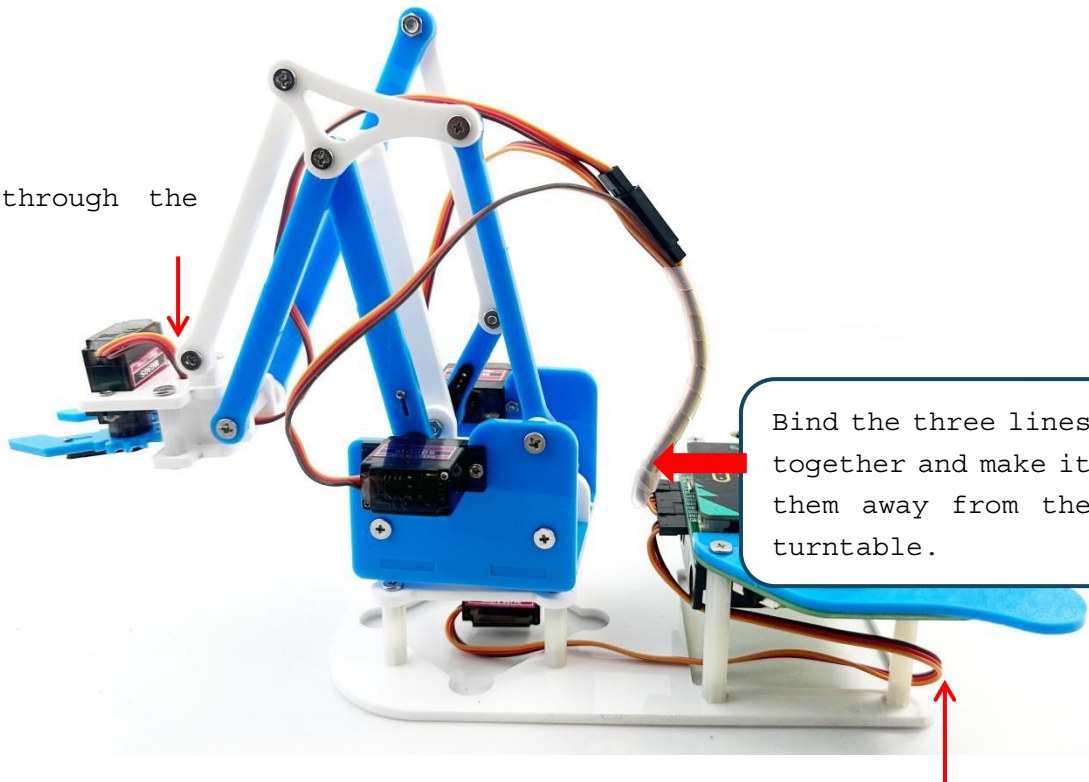
Bundles multiple servo wires for cleaner aesthetics.

Prevents cable interference with moving parts.

Customizable length (trim with scissors as needed).



Pass through the
hole



Bind the three lines
together and make it
them away from the
turntable.

loop it around from
underneath.

Assembly complete!

The function of the remaining acrylic sheets

Once assembled, you' ll have 1 spare acrylic sheet and 4 M3x18mm brackets for the handle. This component plays a crucial role when using the vertical rod independently:

Battery compartment cover

Securely fasten the 4×AA batteries in place

Prevent accidental displacement during operation.

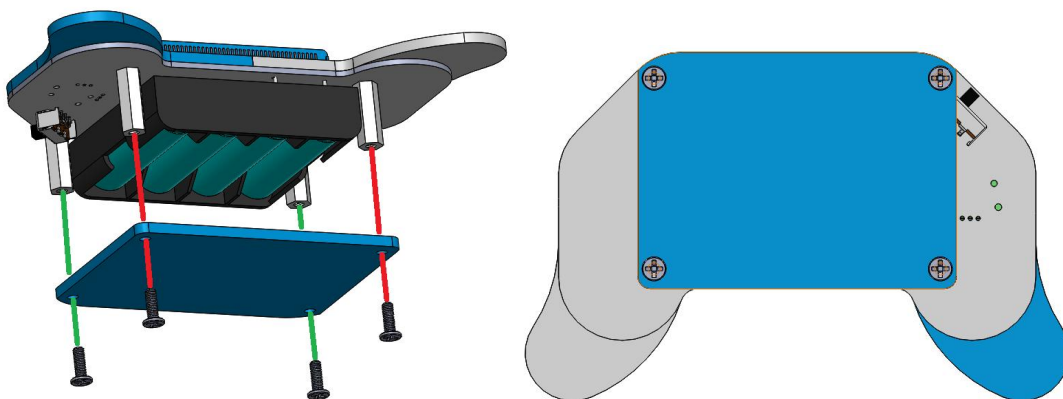
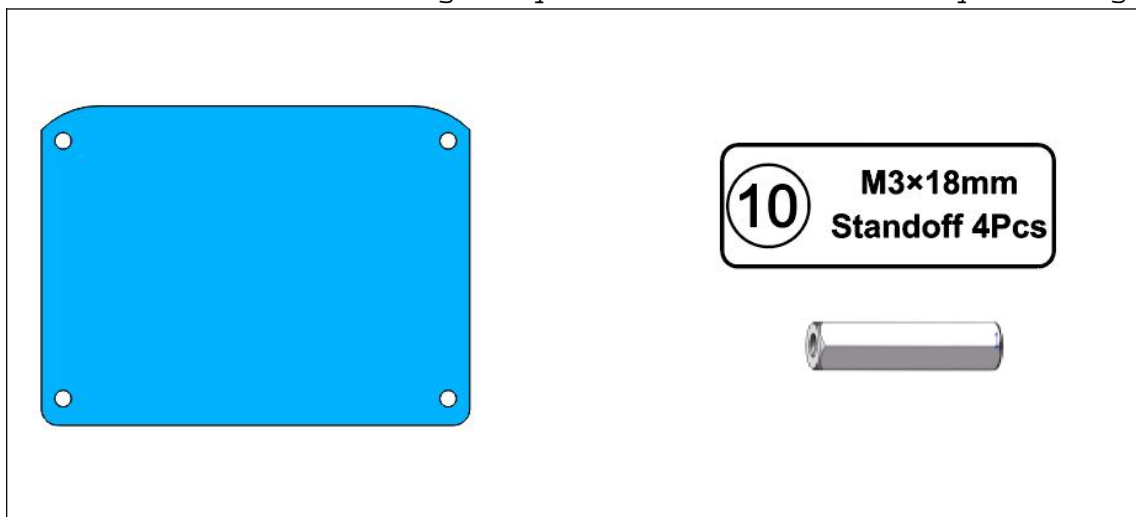
Safety barrier

Prevent fingers from directly contacting battery terminals

Prevent fingers from directly contacting battery terminals

Structural reinforcement

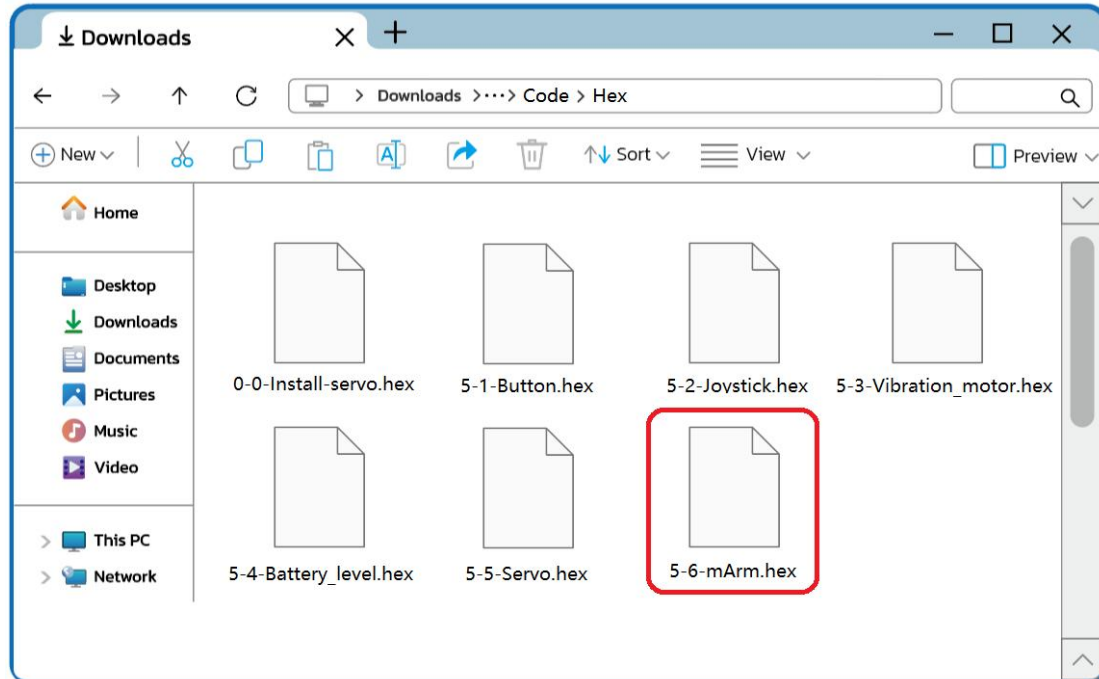
Increase the rigidity of the handle battery housing



3. Control the robotic arm

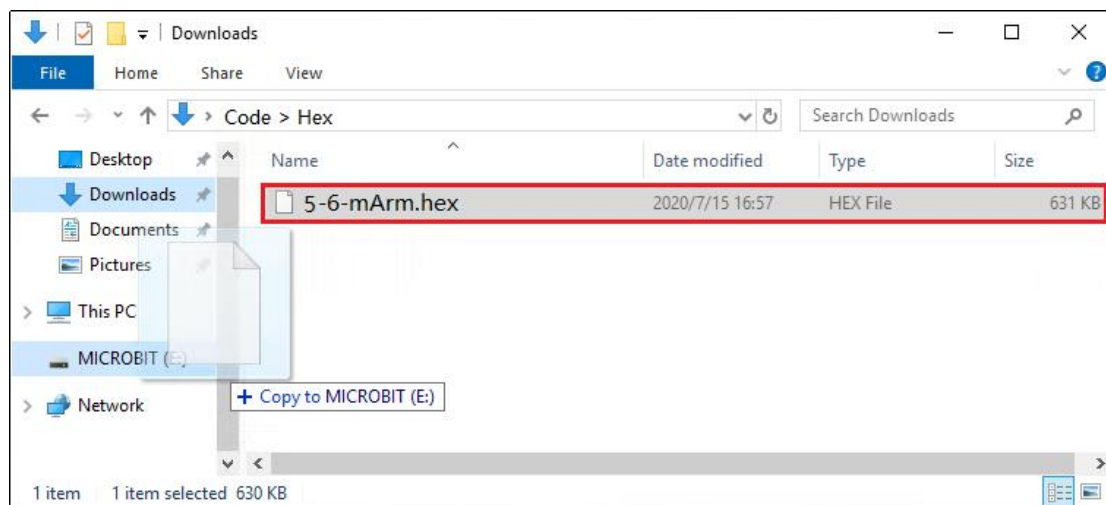
3.1 Upload Control Code

The control code "**5-6-mArm.hex**" for the robotic arm is stored in the "**Code**" folder of the downloaded tutorial package.

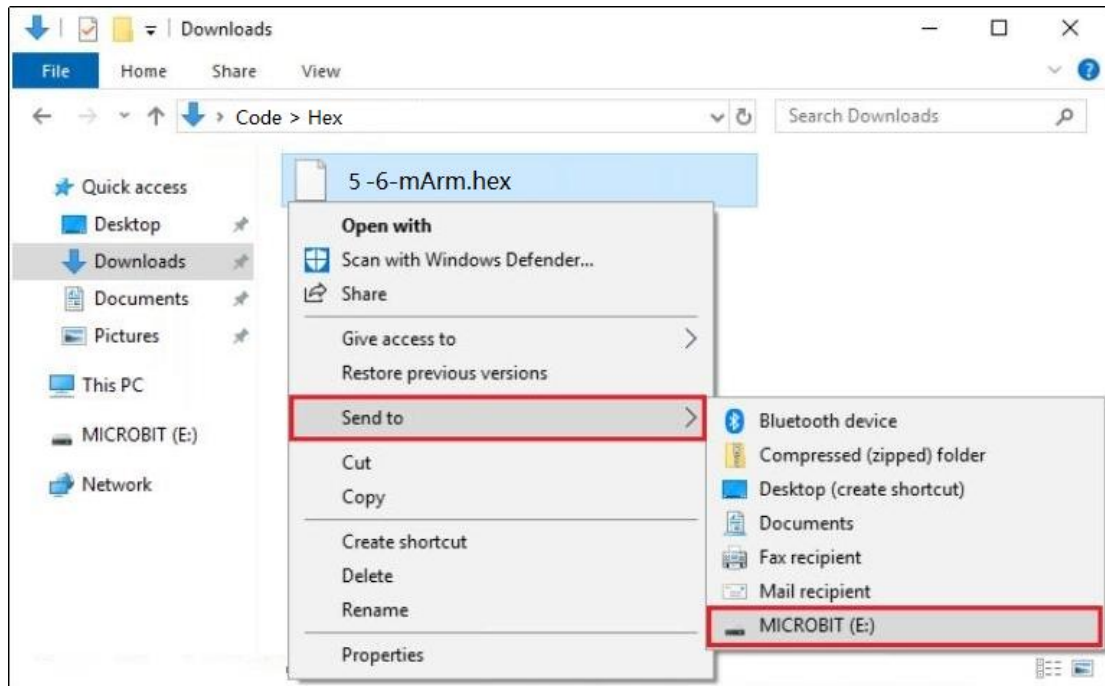


If you are using Micro:bit v1, please refer to Section 5.6 to regenerate the ".hex" file and then upload it.

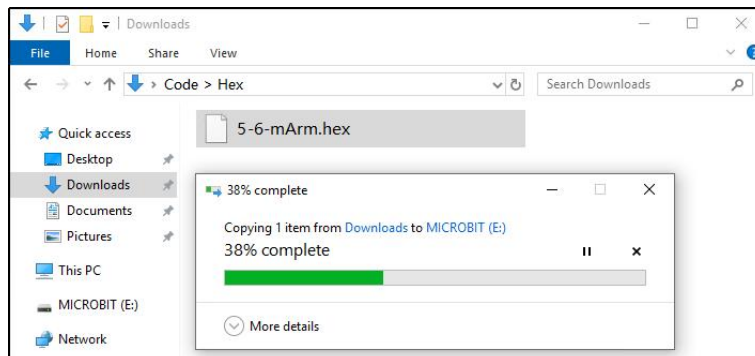
Simply drag and drop the **HEX** file onto the MICROBIT drive.



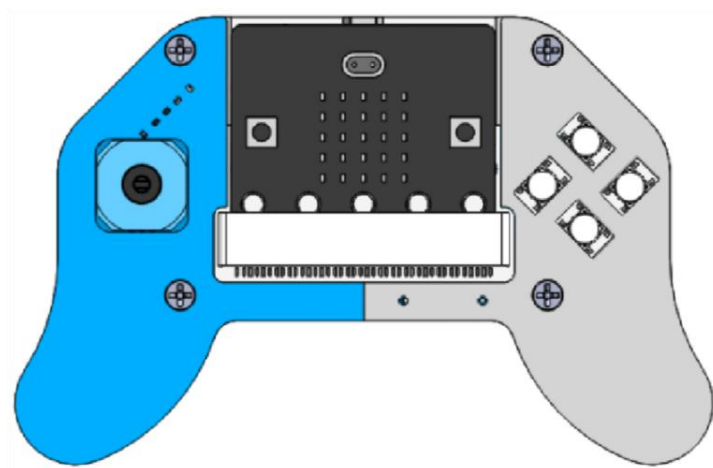
Alternatively, right-click the **HEX** file and select “Send to” → **MICROBIT drive**.



Wait for the upload to complete (during this process, the Micro:bit LED will blink).

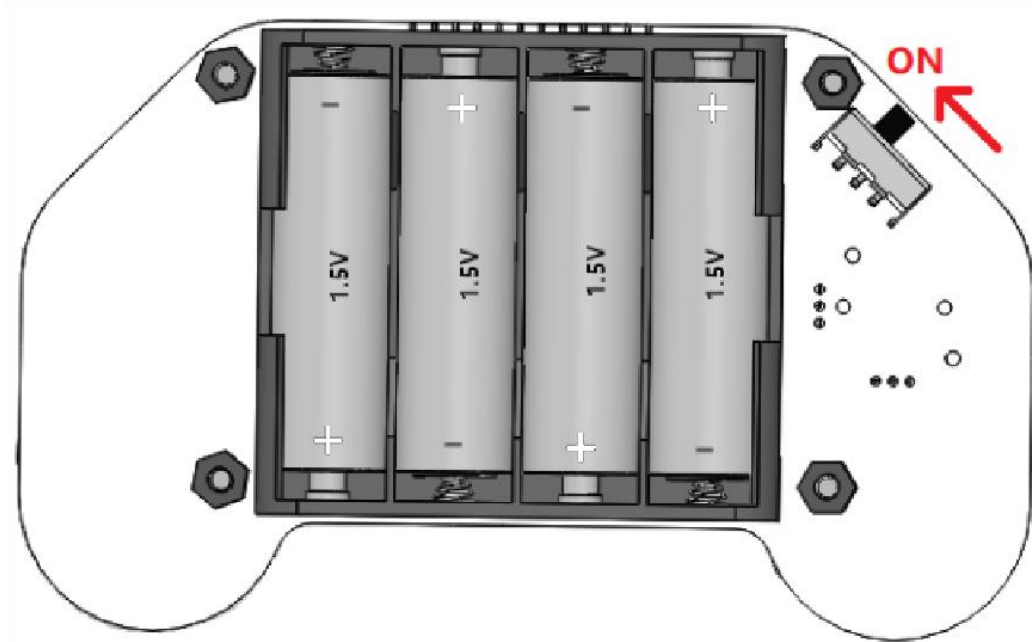


After the code has been uploaded successfully, insert the Micro:bit board into the handle slot.

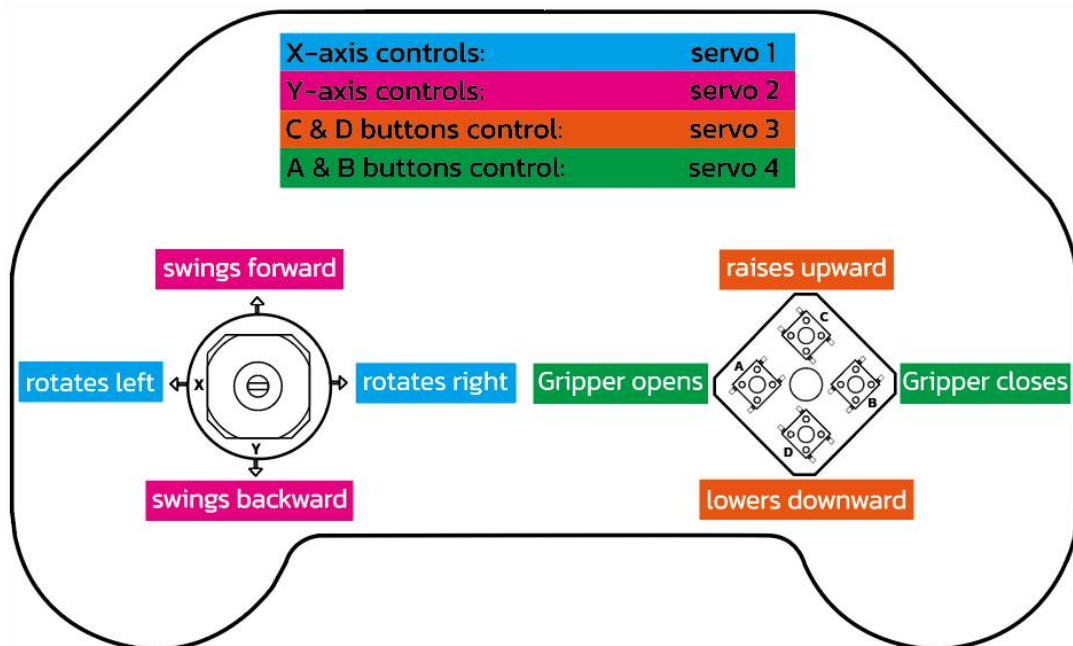


3.2 Quick Start Guide to Robotic Arm Control

Turn on the power switch of the robotic arm.



- After powering on, the micro:bit's LED matrix will continuously cycle through displaying the battery level (0%–100%). If the battery level drops below 30%, please replace all batteries immediately.
- You can use the joystick and four buttons to control the robotic arm's movements.
- Pressing the touch icon on the Micro:bit will activate the vibration motor on the controller.



4. Introduction to MicroPython

The Python editor is the perfect way to start MicroPython programming with the BBC Micro:bit. The Python editor is free and can be run in browsers on all platforms.

We recommend using Google Chrome or Microsoft Edge browser. WebUSB is a newly developed web feature that allows you to directly access the micro:bit from a web page. It also enables you to send data directly from the micro:bit to the Python editor. It is compatible with Google Chrome and Microsoft Edge browsers.

WebUSB supports micro:bit version

If you're not using the latest version of Google Chrome or Microsoft Edge, make sure you're using this version or a later one:

Google Chrome (version 10 and later) for Android, Chrome OS, Linux, macOS, and Windows 79.

Microsoft Edge (version 79 and later) for Android, Chrome OS, Linux, macOS, and Windows 10.

Link to download the latest Google Chrome browser:

<https://www.google.com/chrome/>

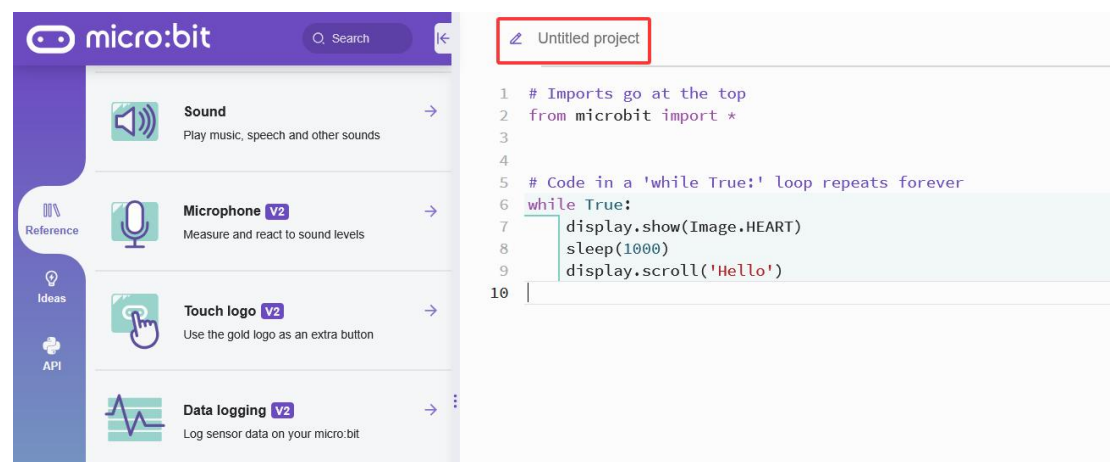
Link to download the latest Microsoft Edge:

<https://www.microsoft.com/en-us/edge/download>

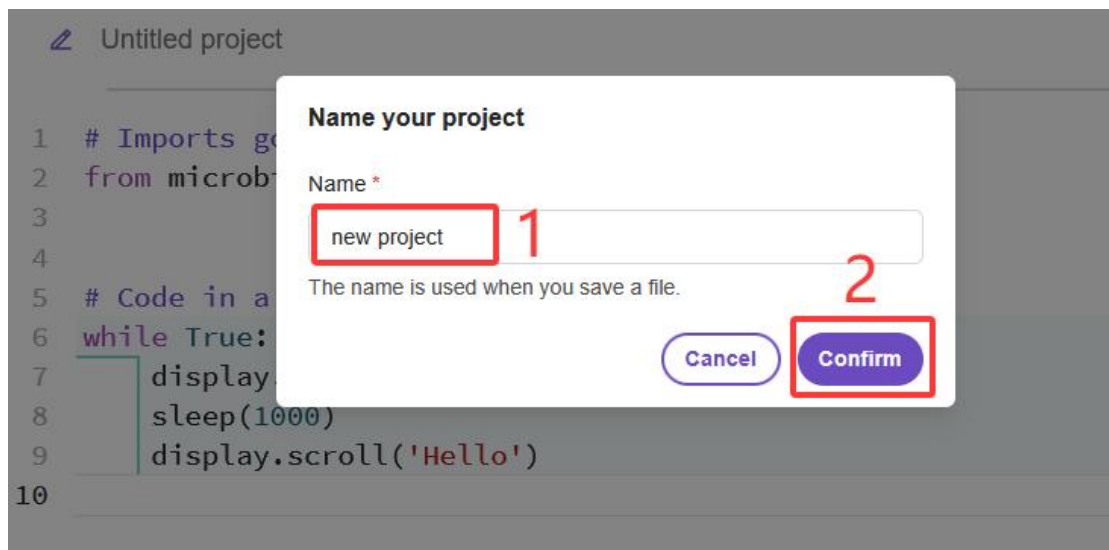
4.1 Create a New Project

Open the online programming page in Google Chrome on your PC:

<https://python.microbit.org/v/3>, and click the red box:

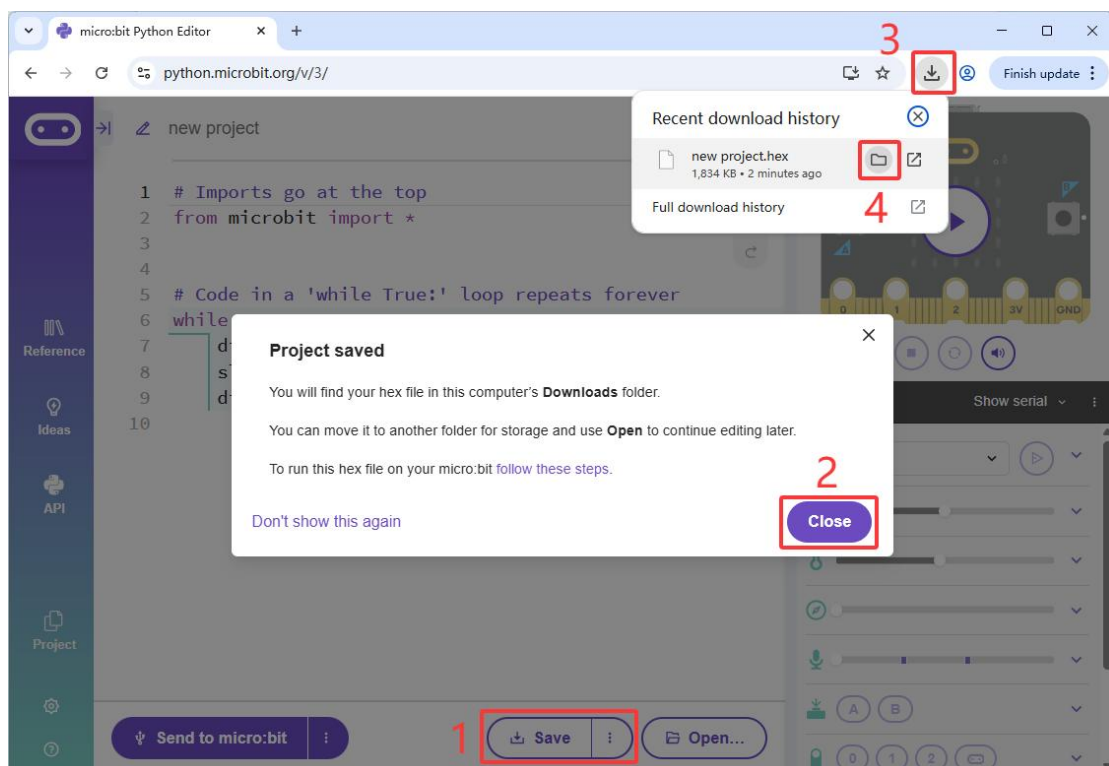


Then enter the project name and click **"Confirm"**:



4.2 Save the project locally

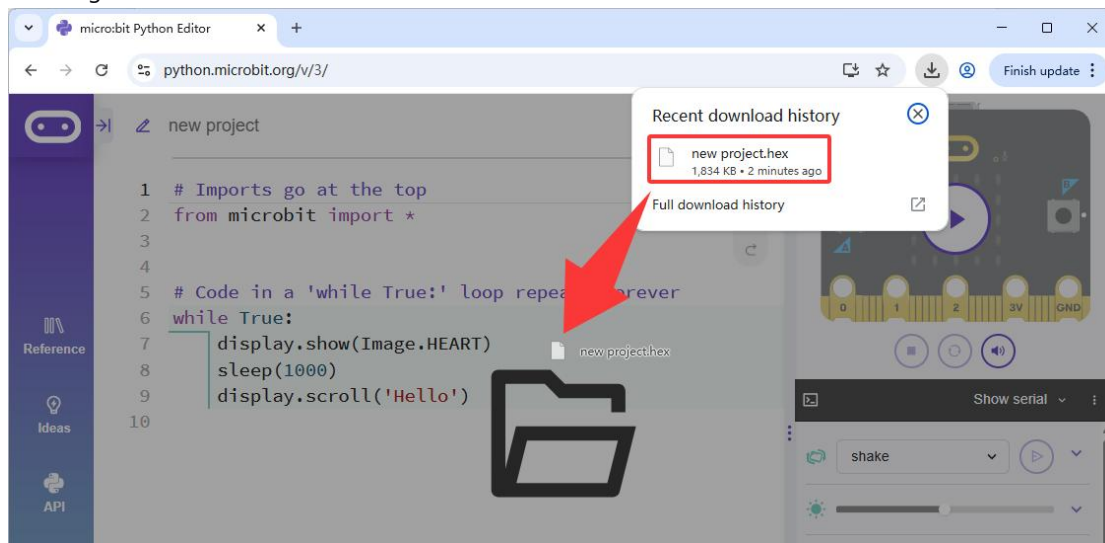
Click the **"Save"** button in **Step 1**, then click the **"Close"** button in **Step 2** to save the project locally. Next, follow **Steps 3 and 4** to locate the saved **.hex** file, as shown in the figure below:



Note: If you don't want the pop-up window to appear again, click **"Don't show this content again" in Step 2!**

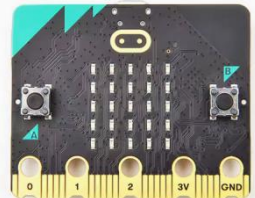
4.3 Importing a Local Project

Simply drag and drop the local “HEX” project file directly into the Python editor’s workspace, as shown in the figure below:

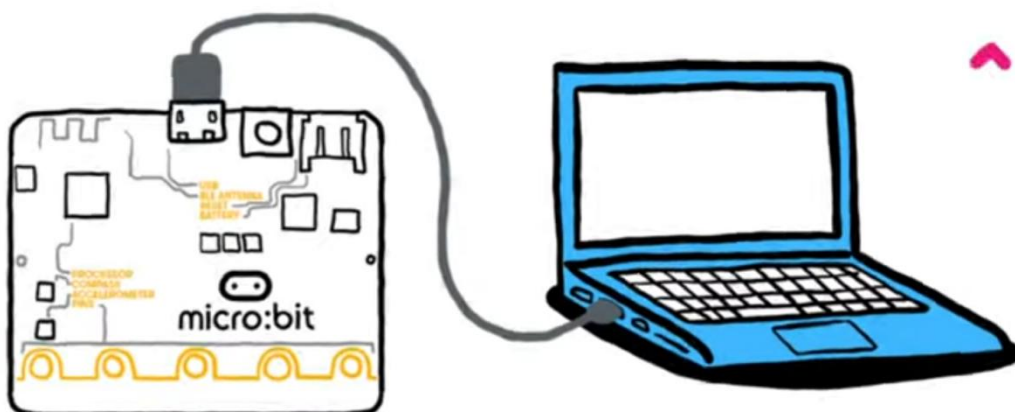


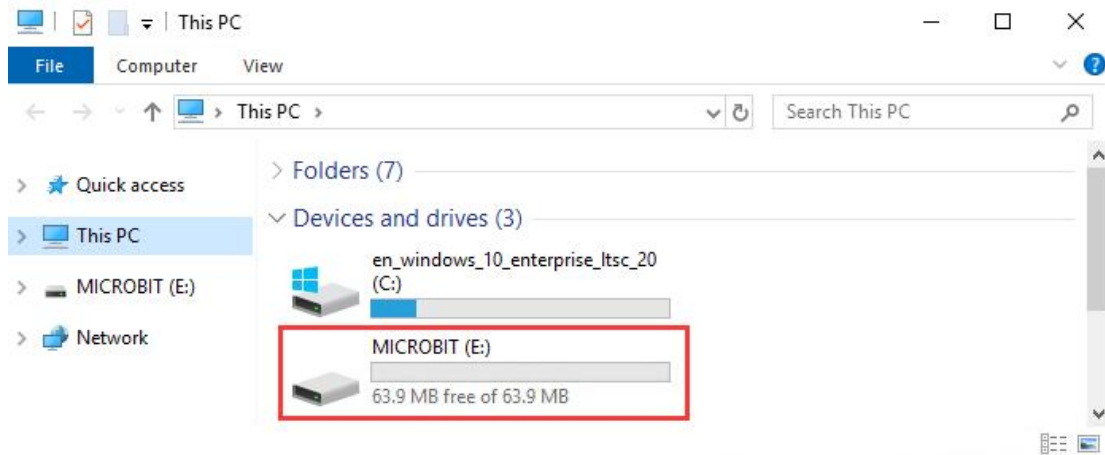
4.4 Upload Project

Tools:

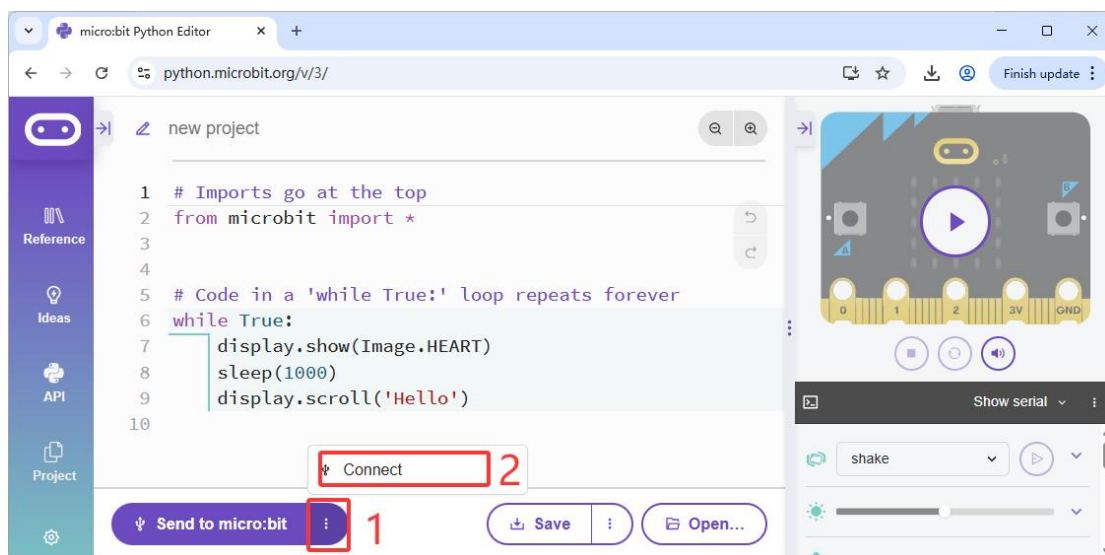
PC	micro:bit v2.x.x	Micro USB cable
		

Connect the micro:bit board to your PC using a Micro USB cable. A storage drive will appear on your PC:

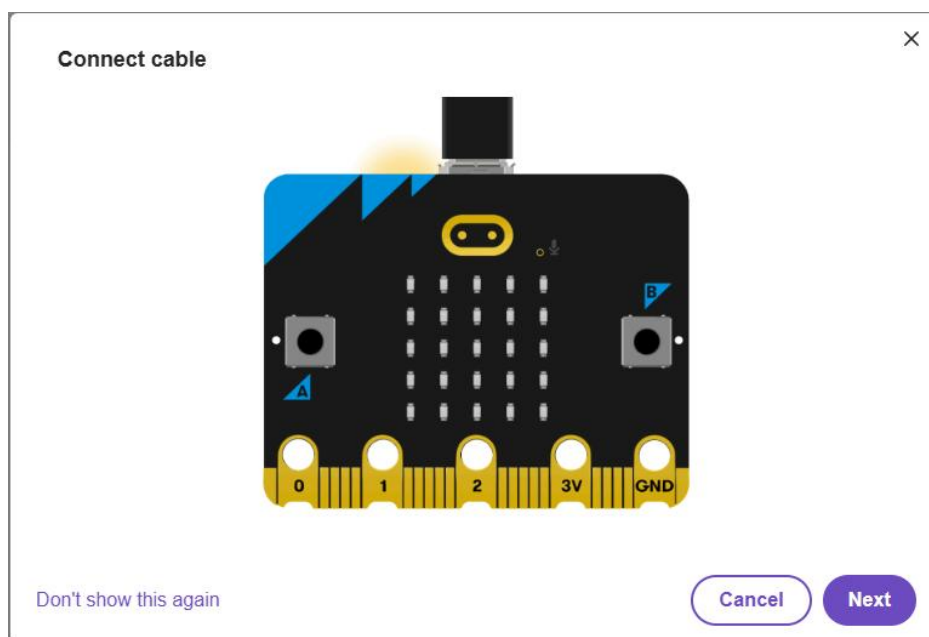




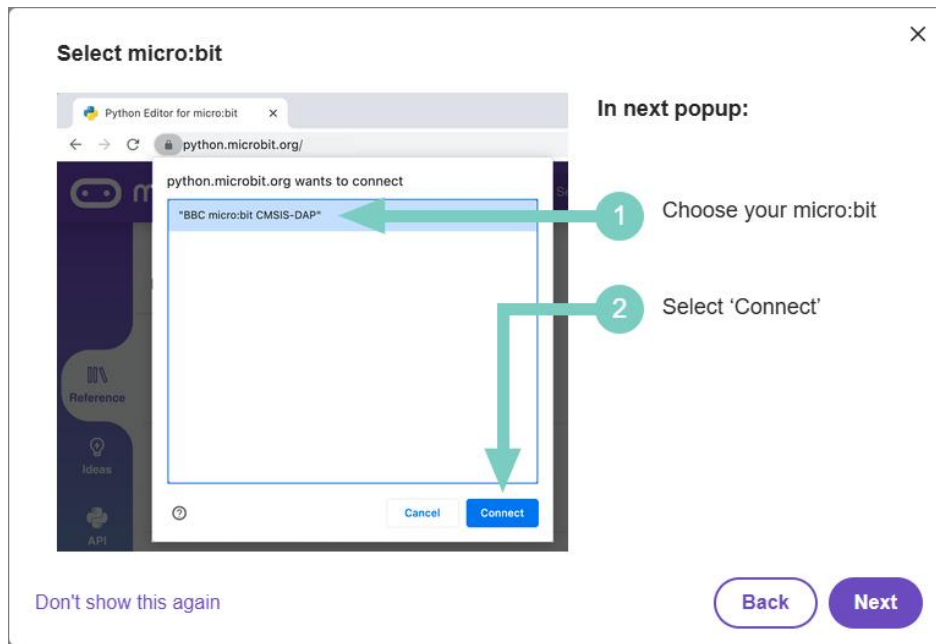
Click the **"Send to micro:bit"** -> **"Connect"** button:



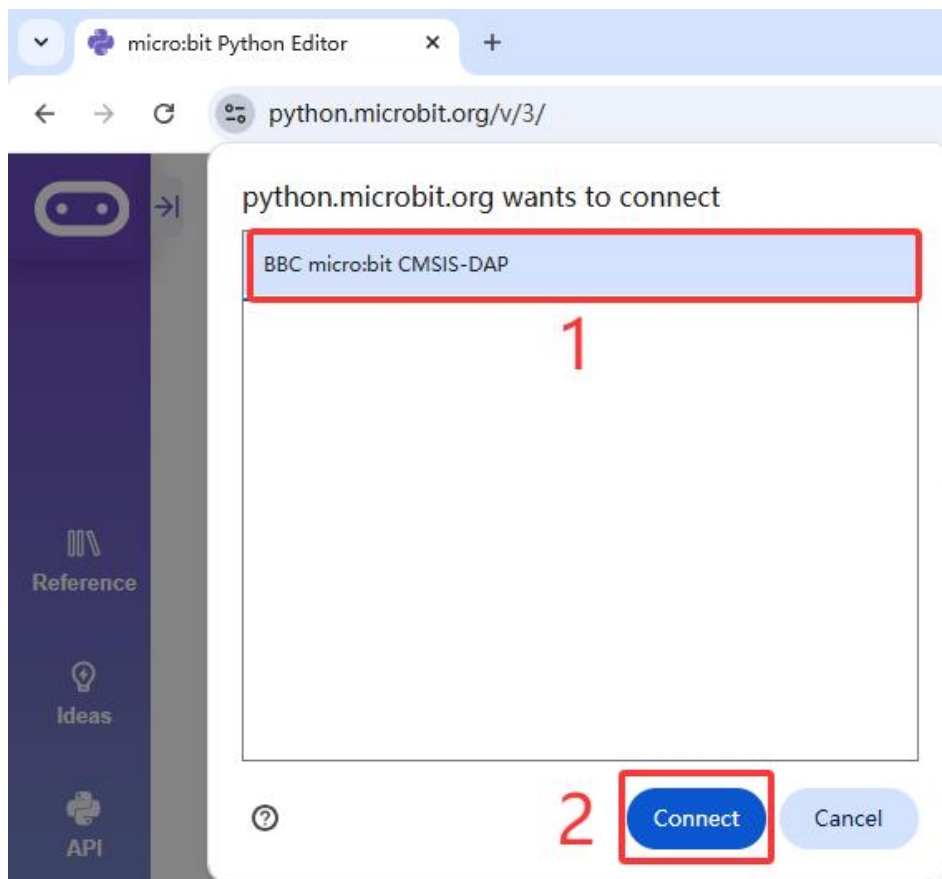
Click **"Next"**:



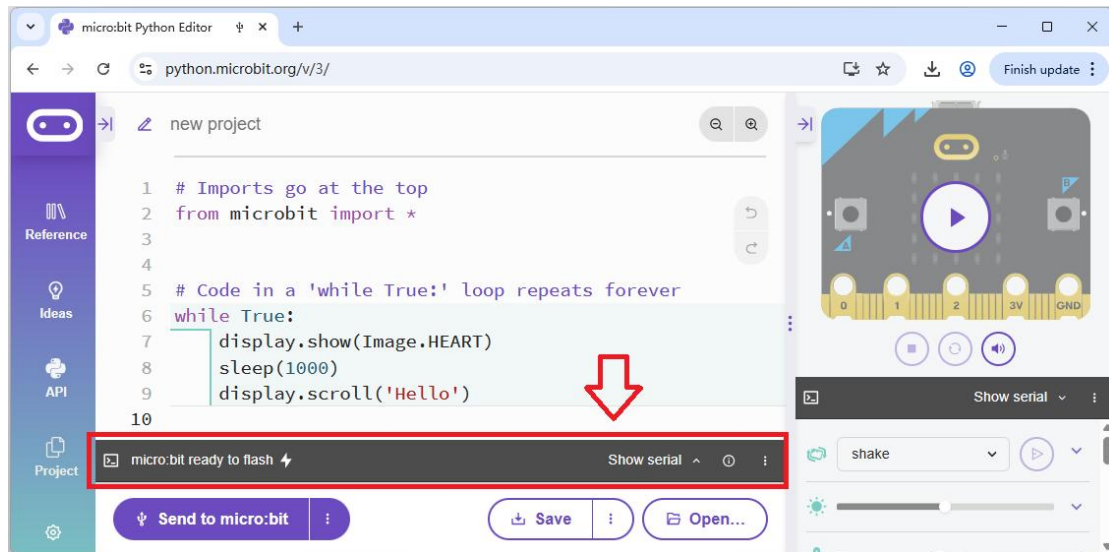
Click **"Next"**:



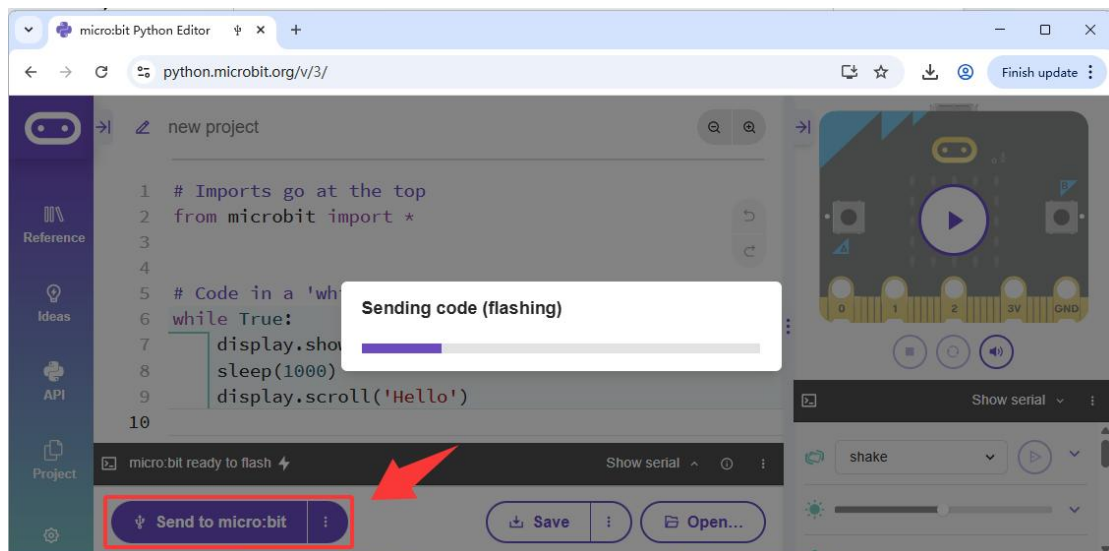
Then follow these steps:



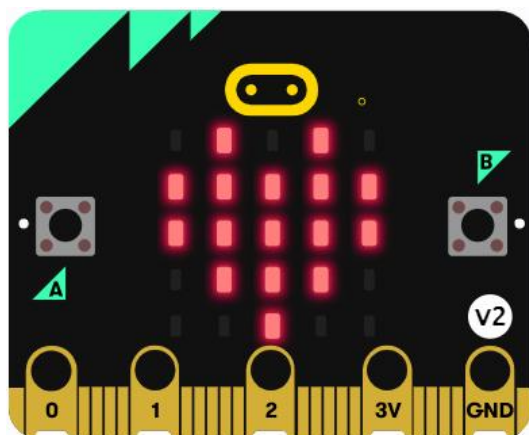
When the following appears, it indicates that the connection between the micro:bit and the Python editor has been successfully established:



Now click **"Send to micro:bit"** to upload the code from Python editor to micro:bit:

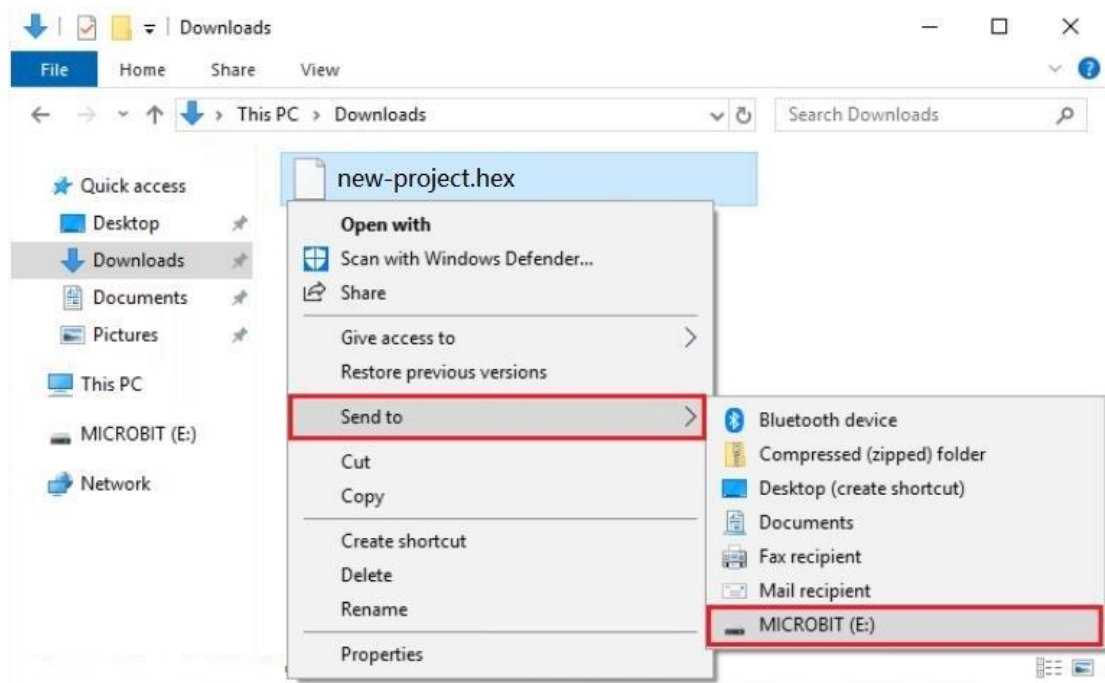


After the code is uploaded, the micro:bit board's dot matrix displays a heart shape and then scrolls "Hello":

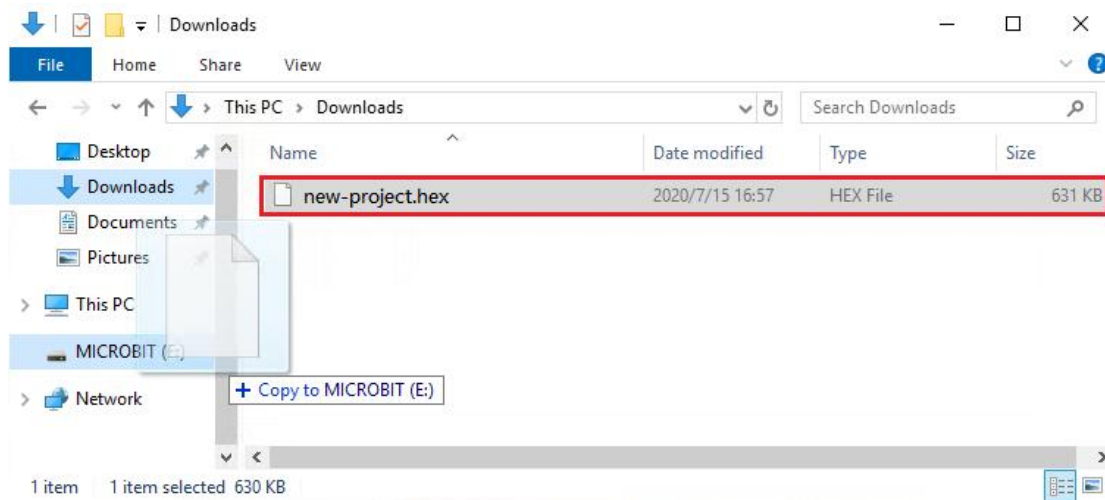


2 Other methods for uploading code

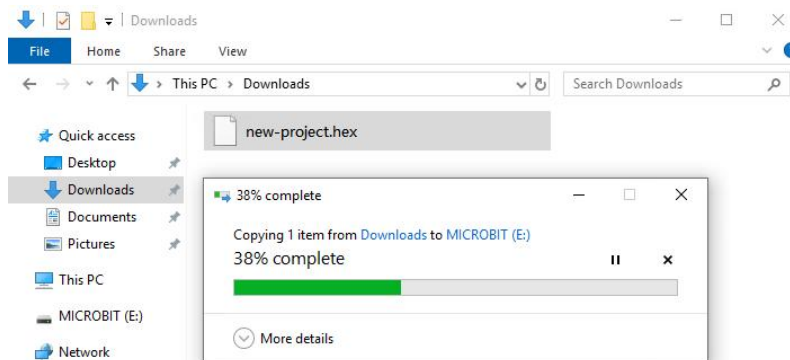
Select the "HEX" file, right-click, and send the "HEX" file to the micro:bit board:



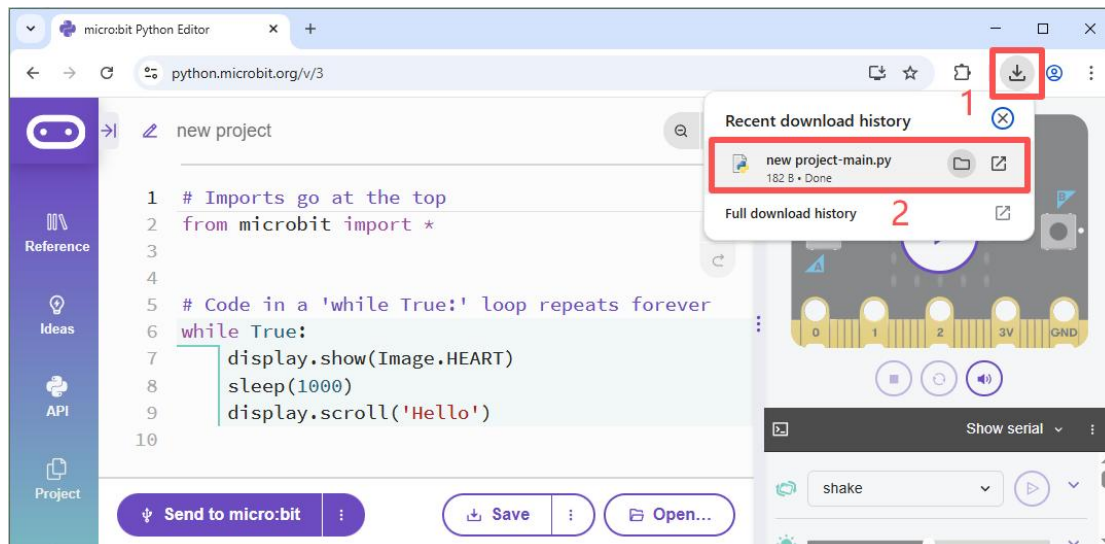
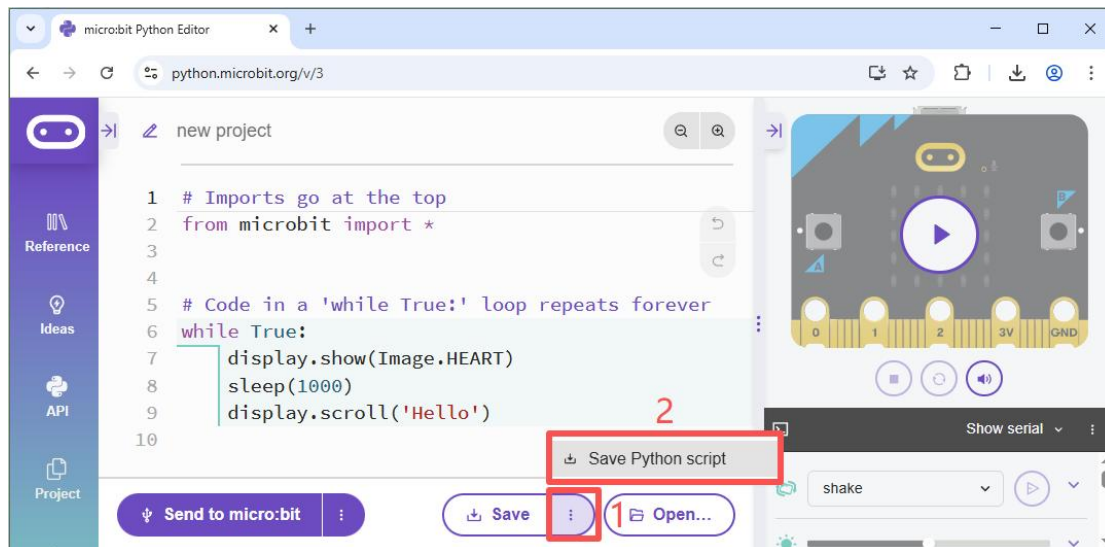
Or simply drag and drop it onto the micro:bit board:



The following interface indicates that the code is being uploaded:



4.5 Save the project as a MicroPython file



Note: MicroPython ".xx.py" file cannot be run directly on micro:bit; Only the project's ".xx.hex" file can be run on micro:bit. The ".xx.py" file can be opened with normal text editing software, but the ".xx.hex" file cannot!

4.6 Basic Syntax Learning for the Python Editor

The micro:bit platform provides official reference documentation for using the MicroPython editor.

User Guide:

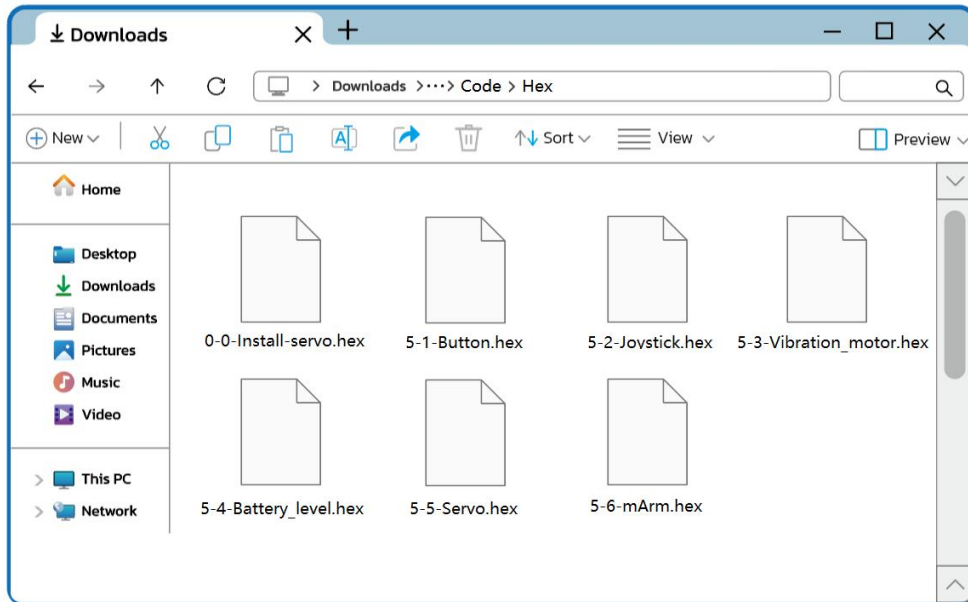
<https://microbit.org/get-started/user-guide/MicroPython-editor/>

MicroPython documentation:

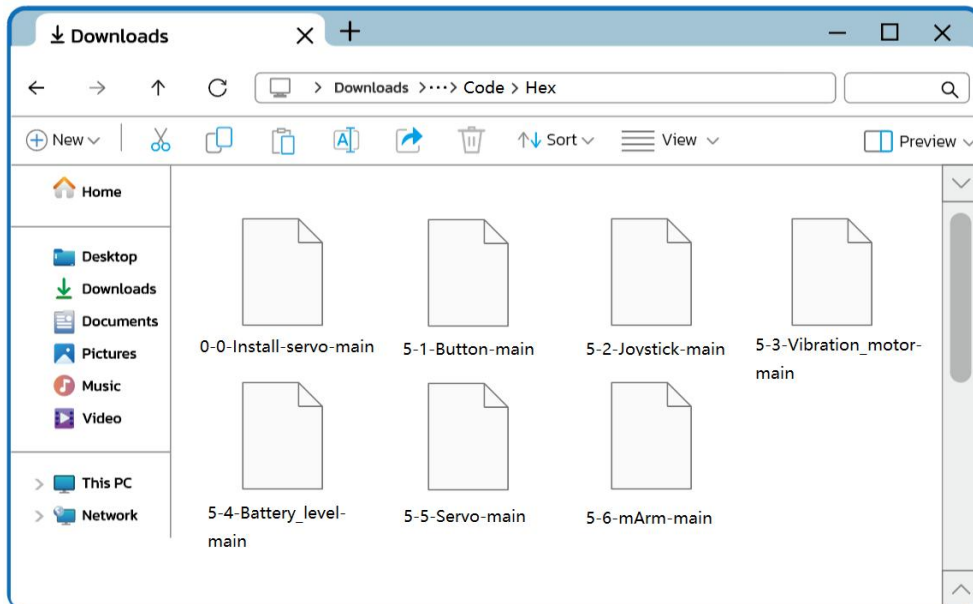
<https://microbit-microPython.readthedocs.io/en/v2-docs/>

5. Robotic arm programming

The example projects for the tutorial are all saved in the **"Micropython-> Code-> Hex"** folder:



The sample Python files for the tutorial are saved in the **"Micropython > Code > Python"** folder:



Warning: Flashing a new program onto the micro:bit will overwrite any existing code. The previous program will be lost. To regain control of the robotic arm after flashing the new code, make sure to flash the final program provided in this chapter.

5.1 Button

Code:

```
# Imports go at the top
from microbit import *

# Define the pins of the A, B, C, and D button
button_a = pin5
button_b = pin11
button_c = pin12
button_d = pin8

# I/O port with pull resistor
button_a.set_pull(button_a.PULL_UP)
button_b.set_pull(button_b.PULL_UP)
button_c.set_pull(button_c.PULL_UP)
button_d.set_pull(button_d.PULL_UP)

# Read button value function
def readButtonValue(button):
    # Return button value
    return button.read_digital()

# Code in a 'while True:' loop repeats forever
while True:
    # Determine whether button_a is pressed
    if readButtonValue(button_a) == 0:
        # If button_a is pressed, dot matrix displays character 'A'
        display.show('A')

    # Determine whether button_b is pressed
    if readButtonValue(button_b) == 0:
        # If button_b is pressed, dot matrix displays character 'B'
        display.show('B')

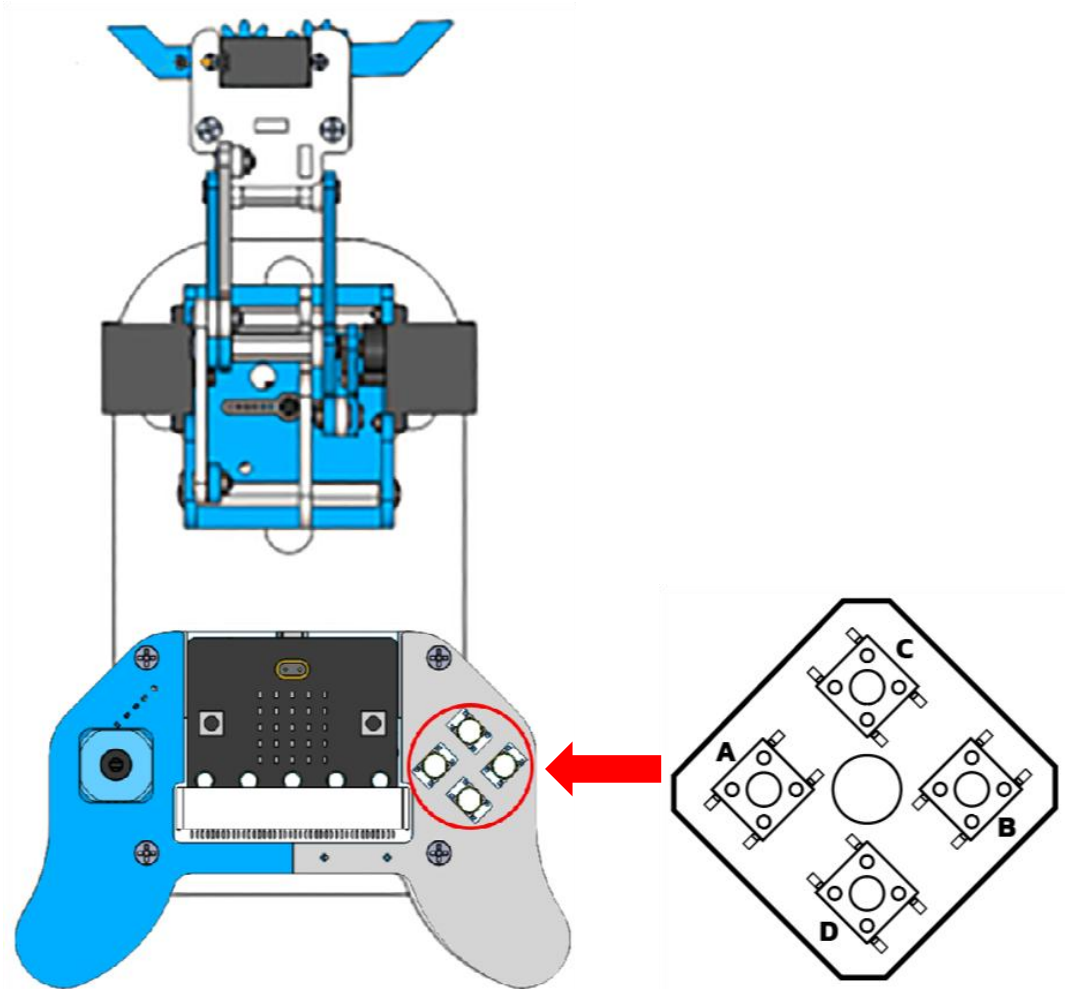
    # Determine whether button_c is pressed
    if readButtonValue(button_c) == 0:
        # If button_c is pressed, dot matrix displays character 'C'
        display.show('C')

    # Determine whether button_d is pressed
    if readButtonValue(button_d) == 0:
        # If button_d is pressed, dot matrix displays character 'D'
        display.show('D')
```

SIYEENOVE

```
# delay 100 millisecond  
sleep(100)
```

Result:



After powering on, the micro:bit matrix display shows a heart shape. When you press the A, B, C, or D button on the mJoystick, the micro:bit matrix display shows "A," "B," "C," or "D," respectively.

5.2 Joystick

Code:

```
# Imports go at the top
from microbit import *

# Define the pins of the joystick
x_axis = pin1
y_axis = pin0

# Read the joystick value function
def readJoystickValue(axis):
    # Map the values from 0 to 1023 to -100 to +100,
    # and then return the mapped value
    return scale(axis.read_analog(), from_=(0, 1023), to=(-100, 100))

# Code in a 'while True:' loop repeats forever
while True:
    # Determine whether the value of the X-axis is greater than 80
    if readJoystickValue(x_axis) > 80:
        # Dot matrix displays character 'L'
        display.show('L')

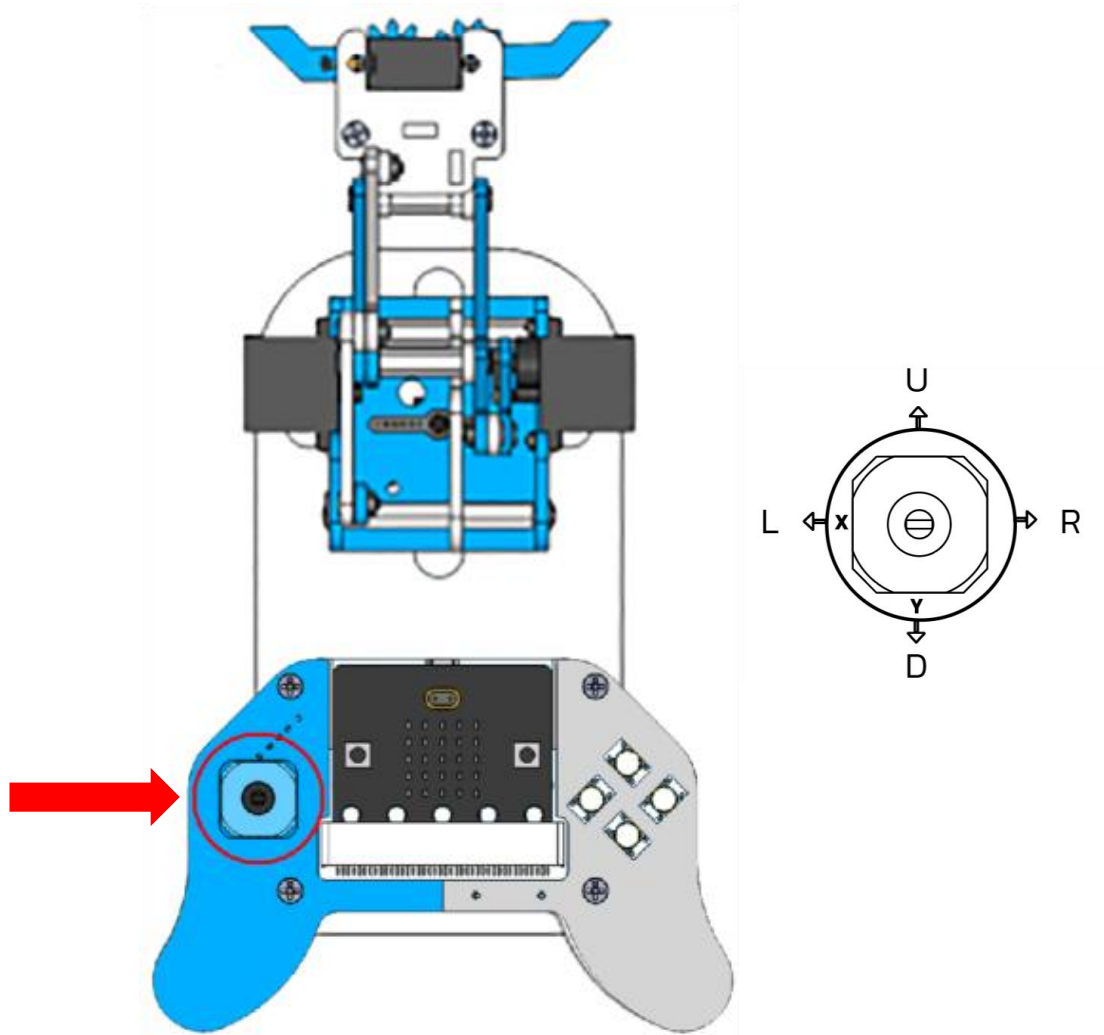
    # Determine whether the value of the X-axis is less than -80
    if readJoystickValue(x_axis) < -80:
        # Dot matrix displays character 'R'
        display.show('R')

    # Determine whether the value of the Y-axis is greater than 80
    if readJoystickValue(y_axis) > 80:
        # Dot matrix displays character 'U'
        display.show('U')

    # Determine whether the value of the Y-axis is less than -80
    if readJoystickValue(y_axis) < -80:
        # Dot matrix displays character 'D'
        display.show('D')

    # delay 100 millisecond
    sleep(100)
```


Result:



When the micro:bit is powered on, its dot matrix displays a heart shape. By moving the joystick on the mJoystick up, down, left, and right, the micro:bit's dot matrix displays L, R, U, and D respectively.

5.3 Vibration Motor

Code:

```
# Imports go at the top
from microbit import *

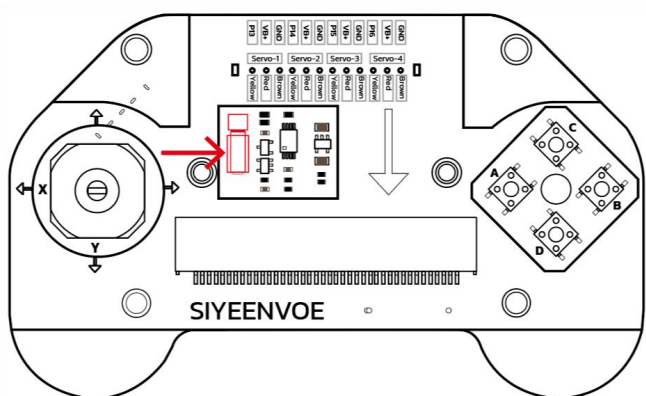
# Define the vibration motor state variables
ON = 1
OFF = 0

# Control the vibration motor function
def setVibrationMotor(OnOrOff):
    # Determine whether the value of the variable OnOrOff is equal to 1
    if OnOrOff == ON:
        # Turn on the vibration motor
        pin2.write_digital(1)
    # Determine whether the value of the variable OnOrOff is equal to 0
    if OnOrOff == OFF:
        # Turn off the vibration motor
        pin2.write_digital(0)

# Code in a 'while True:' loop repeats forever
while True:
    # Turn on the vibration motor
    setVibrationMotor(ON)
    sleep(1000) # delay 1000 millisecond

    # Turn off the vibration motor
    setVibrationMotor(OFF)
    sleep(1000) # delay 1000 millisecond
```

Result:



When the micro:bit is powered on, the handle's vibration motor vibrates once every 1 second.

5.4 Battery voltage

Code:

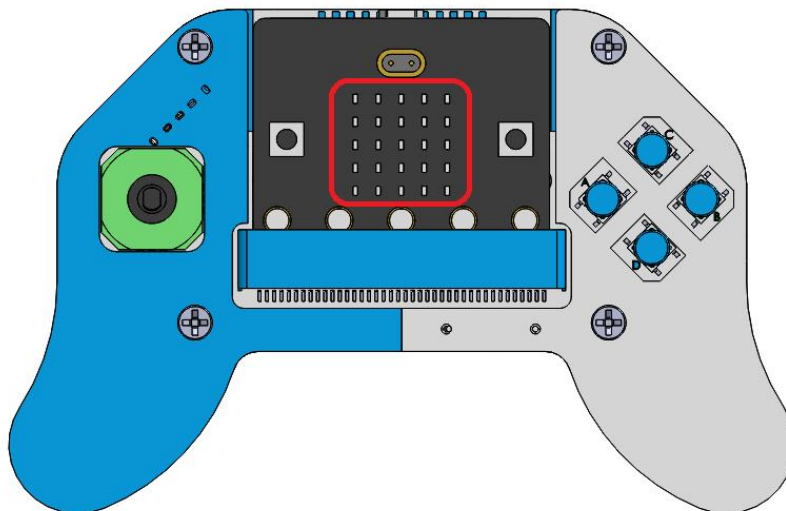
```
# Imports go at the top
from microbit import *

# Read the battery level
# 4 AA batteries
def readBatteryLevel():
    # Read the battery voltage value
    batLevel = pin2.read_analog()
    if batLevel > 310: # 310=6V/6/0.0032226, 100%
        batLevel = 310
    if batLevel < 232: # 232=4.5V/6/0.0032226, 0%
        batLevel = 232
    # Map 232-310 values to 0-100
    batLevel = scale(batLevel, from_=(232, 310), to=(0, 100))
    return batLevel # return battery level

# Code in a 'while True:' loop repeats forever
while True:
    # Dot matrix display of battery level
    display.show(readBatteryLevel())
    sleep(1000) # delay 1000 milliseconds
```

Result:

The micro:bit dot matrix displays the battery level, ranging from 0 to 100.



Warning!

The battery voltage reading and vibration motor control share the same P2 pin on the micro:bit. Frequent switching between these two functions is avoided as this may affect the accuracy of the battery voltage measurement. For best results, after controlling the vibration motor, multiple consecutive voltage readings are taken and the last recorded value is used.

Attention: The joystick must be equipped with 4 AA batteries and the power switch must be turned on!

5.5 Servo

Code:

```
# Imports go at the top
from microbit import *

# Servo class
class Servo:
    # The constructor of a class
    def __init__(self, pin):
        self.pin = pin
        # Servo parameters
        self.min_pulse = 500    # Pulse width at 0 degrees (microseconds)
        self.max_pulse = 2400    # Pulse width at 180 degrees (microseconds)
        self.period = 20000     # 20ms period (50Hz)
        self.pin.set_analog_period(20) # Set period to 20ms

    # Set the servo Angle function
    def set_angle(self, angle):
        # Limit angle range
        angle = max(0, min(180, angle))

        # Calculate pulse width
```

```

    pulse_width = self.min_pulse + (angle / 180) * (self.max_pulse - self.min_pulse)

    # Convert to analog value (0-1023)
    analog_value = int((pulse_width / self.period) * 1023)

    # Set PWM
    self.pin.write_analog(analog_value)

# Set the servo Angle function using pulse width
def set_pulse(self, pulse_width):
    # Limit the maximum pulse value
    pulse_width = max(self.min_pulse, min(self.max_pulse, pulse_width))

    # Calculate pulse
    analog_value = int((pulse_width / self.period) * 1023)

    # Set PWM
    self.pin.write_analog(analog_value)

# Show happy face
display.show(Image.HAPPY)
# delay 100 milliseconds
sleep(100)

# Create 4 servo objects and define the pins to use
servo1 = Servo(pin13) # servo1 uses pin13
servo2 = Servo(pin14) # servo2 uses pin14
servo3 = Servo(pin15) # servo3 uses pin15
servo4 = Servo(pin16) # servo4 uses pin16

# Initialize the angles of the 4 servos
servo1.set_angle(90) # Set the Angle of servo1
sleep(500)           # delay 500 milliseconds
servo2.set_angle(90) # Set the Angle of servo2
sleep(500)           # delay 500 milliseconds
servo3.set_angle(60) # Set the Angle of servo3
sleep(500)           # delay 500 milliseconds
servo4.set_angle(90) # Set the Angle of servo4
sleep(500)           # delay 500 milliseconds

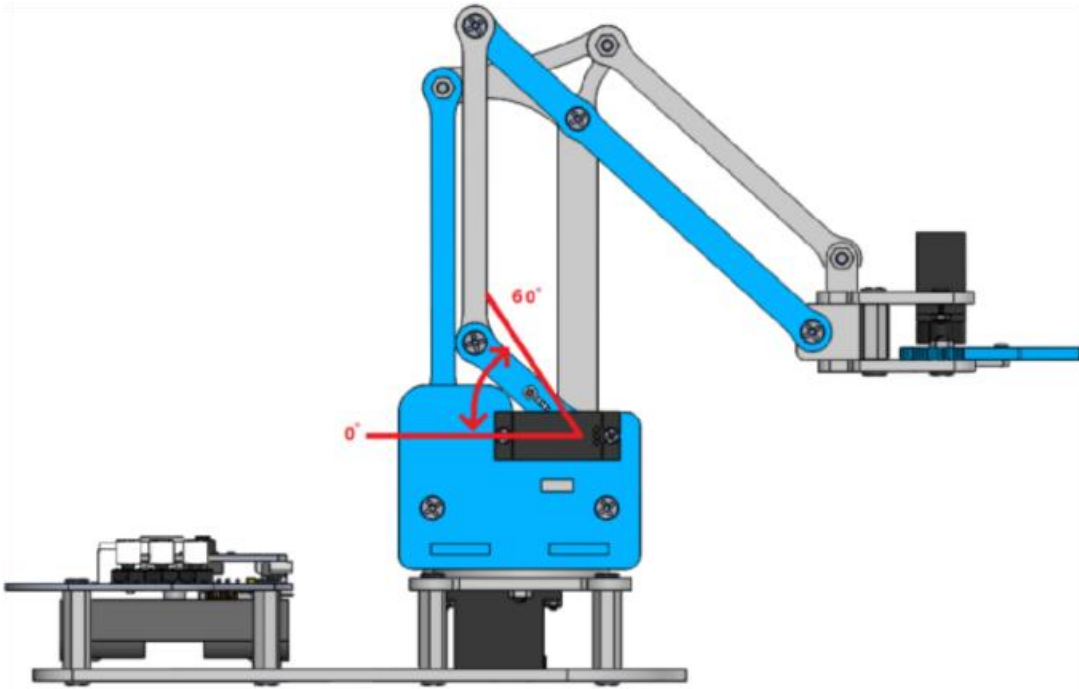
# Let servo 3 swing between 0 and 60 degrees.
while True:
    # The servo swings from 60 degrees to 0 degrees
    for i in range(60):

```

```
# Set the Angle of servo3
servo3.set_angle(60-i)
# delay 20 milliseconds
sleep(20)

# The servo swings from 0 degrees to 60 degrees
for i in range(60):
    # Set the Angle of servo3
    servo3.set_angle(i)
    # delay 20 milliseconds
    sleep(20)
```

Result:



The swing arm of Servo-3 swings between 120 and 180 degrees, and the robot repeatedly performs the lifting and lowering motion of the robotic arm.

Please note: The robotic arm must be equipped with 4 AA batteries and the power switch must be turned on!

5.6 mArm

This section will provide a detailed explanation of the code used in “3.2 Quick Start for Robotic Arm Control” If you wish to use this robotic arm again after uploading other programs, please upload this code.

Code:

```
# Imports go at the top
from microbit import *

# Define the pins of the A, B, C, and D button
button_a = pin5
button_b = pin11
button_c = pin12
button_d = pin8

# I/O port with pull resistor
button_a.set_pull(button_a.PULL_UP)
button_b.set_pull(button_b.PULL_UP)
button_c.set_pull(button_c.PULL_UP)
button_d.set_pull(button_d.PULL_UP)

# Read key value function
def readButtonValue(button):
    return button.read_digital()

# Define the pins of the joystick
x_axis = pin1
y_axis = pin0

# Read the joystick value function
def readJoystickValue(axis):
    # Map 0-1023 values to -100 - +100
    return scale(axis.read_analog(), from_=(0, 1023), to=(-100, 100))

# Define the vibration motor state variables
ON = 1
OFF = 0

# Control the vibration motor function
def setVibrationMotor(OnOrOff):
    if OnOrOff == ON:    # Turn on the vibration motor
        pin2.write_digital(1)
```



```

if OnOrOff == OFF: # Turn off the vibration motor
    pin2.write_digital(0)

# Read the battery level.
# 4 AA batteries
def readBatteryLevel():
    batLevel = pin2.read_analog()
    if batLevel > 310: # 310=6V/6/0.0032226, 100%
        batLevel = 310
    if batLevel < 232: # 232=4.5V/6/0.0032226, 0%
        batLevel = 232
    # Map 232-310 values to 0-100
    batLevel = scale(batLevel, from_=(232, 310), to=(0, 100))
    return batLevel

# Servo class
class Servo:
    def __init__(self, pin):
        self.pin = pin
        # Servo parameters
        self.min_pulse = 500 # Pulse width at 0 degrees (microseconds)
        self.max_pulse = 2400 # Pulse width at 180 degrees (microseconds)
        self.period = 20000 # 20ms period (50Hz)
        self.pin.set_analog_period(20) # Set period to 20ms

    def set_angle(self, angle):
        # Limit angle range
        angle = max(0, min(180, angle))

        # Calculate pulse width
        pulse_width = self.min_pulse + (angle / 180) * (self.max_pulse - self.min_pulse)

        # Convert to analog value (0-1023)
        analog_value = int((pulse_width / self.period) * 1023)

        # Set PWM
        self.pin.write_analog(analog_value)

    def set_pulse(self, pulse_width):
        # Set pulse width directly (microseconds)
        pulse_width = max(self.min_pulse, min(self.max_pulse, pulse_width))
        analog_value = int((pulse_width / self.period) * 1023)
        self.pin.write_analog(analog_value)

```

```
# Show happy face
display.show(Image.HAPPY)
sleep(100) # delay 100 milliseconds

# Define the Angle variables of the 4 servos
servo1_degree = 90
servo2_degree = 90
servo3_degree = 60
servo4_degree = 90

# Create 4 servo objects and define the pins to use
servo1 = Servo(pin13)
servo2 = Servo(pin14)
servo3 = Servo(pin15)
servo4 = Servo(pin16)

# Initialize the angles of the 4 servos
servo1.set_angle(servo1_degree) # Set the Angle of servo1
sleep(500) # delay 500 milliseconds
servo2.set_angle(servo2_degree)
sleep(500)
servo3.set_angle(servo3_degree)
sleep(500)
servo4.set_angle(servo4_degree)
sleep(500)

# Reads the current time when the program is running
last_time = running_time()
interval = 5000 # 5000ms
last_ticker = running_time()
ticker = 40000 # 40000ms

# An infinite loop statement.
while True:
    # Reads the current time when the program is running
    current_time = running_time()
    if current_time - last_time >= interval:
        # Display battery power value
        display.show(readBatteryLevel())
        last_time = current_time

    # Prevent the servo of the claws from overheating
    current_ticker = running_time()
    if current_ticker - last_ticker >= ticker:
```

```

    # Turn off the pulse of the servo
    servo4.set_pulse(0)

# Read the value of the joystick X-axis and drive the Angle
# of the servo1 according to the value.
if readJoystickValue(x_axis) > 60 and servo1_degree < 180:
    servo1_degree = servo1_degree + 1 # Variable increments by 1
    servo1.set_angle(servo1_degree)   # Set the Angle of servo1
    sleep(15)
if readJoystickValue(x_axis) < -60 and servo1_degree > 0:
    servo1_degree = servo1_degree - 1 # The variable decrements by 1
    servo1.set_angle(servo1_degree)
    sleep(15)

# Read the value of the joystick Y-axis and drive the Angle
# of the servo2 according to the value.
if readJoystickValue(y_axis) > 60 and servo2_degree < 180:
    servo2_degree = servo2_degree + 1
    servo2.set_angle(servo2_degree)
    sleep(15)
if readJoystickValue(y_axis) < -60 and servo2_degree > 0:
    servo2_degree = servo2_degree - 1
    servo2.set_angle(servo2_degree)
    sleep(15)

# Read the value of the button_a and drive the Angle
# of the servo4 according to the value.
if readButtonValue(button_a) == 0 and servo4_degree > 5:
    servo4_degree = servo4_degree - 1
    servo4.set_angle(servo4_degree)
    sleep(15)

# Read the value of the button_b and drive the Angle
# of the servo4 according to the value.
if readButtonValue(button_b) == 0 and servo4_degree < 180:
    servo4_degree = servo4_degree + 1
    servo4.set_angle(servo4_degree)
    sleep(15)

# Read the value of the button_c and drive the Angle
# of the servo3 according to the value.
if readButtonValue(button_c) == 0 and servo3_degree > 0:
    servo3_degree = servo3_degree - 1
    servo3.set_angle(servo3_degree)

```

```

sleep(15)

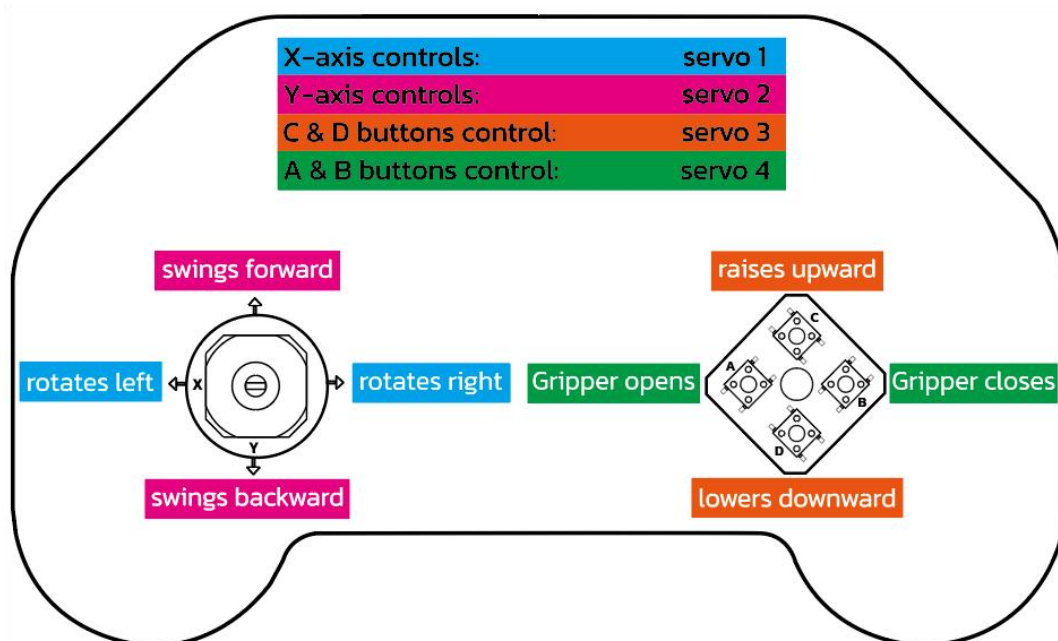
# Read the value of the button_d and drive the Angle
# of the servo3 according to the value.
if readButtonValue(button_d) == 0 and servo3_degree < 120:
    servo3_degree = servo3_degree + 1
    servo3.set_angle(servo3_degree)
    sleep(15)

# Executed when the micro:bit v2 logo is touched.
# Attention! Not compatible with microbit v1.
if pin_logo.is_touched():
    setVibrationMotor(ON)
    sleep(200)
    setVibrationMotor(OFF)

```

Result:

- After powering on, the micro:bit's LED matrix will continuously cycle through displaying the battery level (0%–100%). If the battery level drops below 30%, please replace all batteries immediately.
- You can use the joystick and four buttons to control the robotic arm's movements.
- Pressing the touch icon on the Micro:bit will activate the vibration motor on the controller.



Attention! micro:bit v1 does not support logo touch function, if you want to use this program in micro:bit v1, you need to delete the following code:

```
# Executed when the micro:bit v2 logo is touched.  
# Attention! Not compatible with microbit v1.  
if pin_logo.is_touched():  
    setVibrationMotor(ON)  
    sleep(200)  
    setVibrationMotor(OFF)
```

6. QA

6.1 micro:bit code upload failed

- 👉 Verify that the USB cable supports data transfer (charging-only cables won't work).
- 👉 Check the USB cable and port for physical damage or connection issues.

6.2 The robotic arm is not working.

- 👉 Confirm that the 4×AA batteries are installed correctly and the power switch is in the ON position.
- 👉 Check the battery voltage level (recommended minimum: 5.5V).
- 👉 Verify that the servo initialization sequence has been completed according to the tutorial guide.

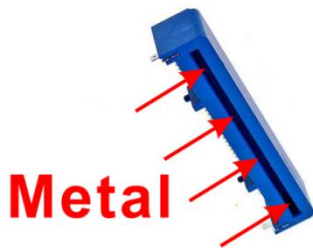
6.3 micro:bit driver is exhibiting abnormal behavior.

- 👉 Drives labeled “MAINTENANCE” (rather than “MICROBIT”) indicate an accidental entry into firmware recovery mode. Solution:

- Use the official tool to reflash the firmware:
<https://microbit.org/get-started/user-guide/firmware/>
- Prevent recurrence by avoiding pressing the reset button during power cycling.

6.4 The button or joystick is unresponsive

- 👉 Make sure the micro:bit is fully inserted into the mJoystick slot.
- 👉 Make sure the micro:bit's edge connector is clean.
- 👉 Make sure the mJoystick slot is clean.



6.5 Errors when using micro:bit v1

If you encounter an error when using the sample code "5-6-mArm.hex" from Chapter 5.6 on the micro:bit v1, this is because the code uses the statement "`pin_logo.is_touched()`", which is available only on the micro:bit v2.

- 👉 The micro:bit v1 lacks a logo touch function, which causes an error. Simply delete the "`pin_logo.is_touched()`" statement, regenerate the .hex file, and then upload it to the micro:bit v1 board.

6.6 Other Issues

- 👉 Please check whether the assembly is correct.
- 👉 Please check whether the batteries used meet the specifications.

7. Contact Us

If you can't find the solutions mentioned above, please contact our support team.

To help us assist you quickly, please have the following information ready:

Your order number.

Product model (e.g., robotic arm kit M1R0000) and software version (e.g., MicroPython v3.X.X).

The question or a detailed description of your issue.

The steps you have already tried.

Any screenshots, photos, or code snippets of related error messages.

Other queries?

We value your feedback and are always looking for ways to improve.

Please feel free to contact us:

Tutorial errors and feedback: Help us improve the documentation.

Product Ideas and Suggestions: We'd love to hear your great ideas.

Partnerships and Collaboration: Interested in collaborating? Let's talk.

Discounts and promotions: Inquire about educational or bulk pricing.

Any other content: For all other non-technical questions.

Support channels



support@siyeenove.com



<https://siyeenove.com>